

Pruebas de entrada: Aviónica

Faraday Rocketry UPV

Ejercicio 3: Especialización

Opción II: Fusión sensorial

Enunciado

En un vuelo real al espacio, ciertas variables críticas no pueden obtenerse directamente de los sensores de la aviónica. Para remediar esto, se emplean algoritmos de fusión sensorial para aprovechar los datos que sí se pueden medir y obtener los que no.

- a. ¿Qué algoritmos de esta índole crees que deben implementarse en un cohete sonda que llegue al espacio?
 - b. Diseña e implementa en C o C++ estos algoritmos, justificando claramente el tipo de filtro que son, las medidas usadas en cada paso y la confianza que le das a cada una de ellas.
-

Índice

1. Algoritmos de fusión sensorial propuestos y planteamiento inicial	3
1.1. Estimación de la orientación	4
1.2. Estimación de la posición y la velocidad	4
1.3. Comprobación de las medidas y confianza en los sensores	5
2. Diseño, implementación y simulación de los algoritmos	6
2.1. Sistemas de referencia y unidades	6
2.2. Variables del estimador	7
2.3. Inicialización del estimador	8
2.4. Predicción mediante la IMU	9
2.5. Propagación de la incertidumbre	14
2.6. Corrección mediante el GNSS	18
2.7. Corrección mediante los barómetros	23
2.8. Corrección mediante el magnetómetro	25
2.9. Integración de la predicción y las correcciones	29
2.10. Simulación y evaluación del estimador	35
2.11. Trayectoria de referencia 1: ascenso vertical	37
2.12. Trayectoria de referencia 2: movimiento tridimensional y orientación variable . .	48
2.13. Resultados e interpretación de las simulaciones	50
2.14. Comparación y conclusiones	73
Referencias	76
A. Fundamento del filtro de Kalman extendido	77
A.1. Modelo del estado y de las medidas	77
A.2. Representación de la incertidumbre	77
A.3. Predicción y linealización	78
A.4. Corrección con una medida	78
A.5. Comprobación de la innovación	79
B. Transformación de los ejes del cuerpo al sistema NED	79
B.1. Convención y matriz de rotación	79
B.2. Implementación en C	79
B.3. Integración en la predicción	80

1. Algoritmos de fusión sensorial propuestos y planteamiento inicial

Para analizar el vuelo necesitamos conocer la orientación, la posición y la velocidad del cohete. Ningún sensor aporta por sí solo toda esa información, por lo que combinaremos sus medidas mediante fusión sensorial. Partiremos de la IMU, los dos barómetros y el receptor GNSS seleccionados en los ejercicios anteriores, y añadiremos un magnetómetro. La IMU mide velocidad angular y fuerza específica; los barómetros, presión; el GNSS aporta posición y velocidad; y el magnetómetro mide el campo magnético. Con estas referencias estimaremos el movimiento y la orientación.

Utilizaremos un filtro de Kalman extendido de estado de error (ESKF). La IMU permitirá predecir el movimiento entre medidas externas y el filtro corregirá esa predicción según la incertidumbre de cada sensor. La orientación interviene en la transformación de las medidas del acelerómetro y hace que el modelo sea no lineal [1].

Distinguiremos la estimación de la orientación y la estimación de la posición y velocidad, aunque implementaremos ambas en el mismo estimador para poder observar cómo los errores de orientación afectan al cálculo de la velocidad y posición. El estimador también incluirá los sesgos del acelerómetro y del giróscopo.

Utilizaremos el acelerómetro y el giróscopo para actualizar con frecuencia la predicción, mientras que el GNSS, el magnetómetro y los barómetros se encargarán de corregir estas predicciones con sus propias medidas, de forma independiente entre ellos. Así, entre dos soluciones GNSS seguiremos estimando el movimiento con la IMU, sin esperar a que todos los sensores se actualicen a la vez [1].

Antes de utilizar una medida comprobaremos si el sensor puede proporcionarla de forma fiable en esa fase del vuelo. Una lectura fuera de rango o afectada por una perturbación no debe corregir el estado como si fuese válida. Si faltan referencias externas, continuaremos con la IMU, aunque podremos acumular deriva. Esto será especialmente relevante a gran altitud, donde el barómetro puede salir de rango y la disponibilidad del GNSS debe comprobarse para la misión. La adaptación al vuelo espacial se discutirá al final, pero no se implementará en estas simulaciones.

El funcionamiento se organizará en dos operaciones: la predicción, que utiliza la IMU para avanzar la estimación entre instantes consecutivos, y la corrección, que incorpora las referencias de los demás sensores para limitar los errores acumulados.

El desarrollo seguirá esta secuencia:

1. Definir los sistemas de referencia, las variables del estado y su inicialización.
2. Implementar la predicción del movimiento y de la orientación mediante la IMU, junto con la propagación de la incertidumbre.
3. Incorporar las correcciones del GNSS para posición y velocidad, de los barómetros para altura y del magnetómetro como referencia de orientación.
4. Integrar estas operaciones, gestionando la disponibilidad, la validez y los instantes de adquisición de las medidas.
5. Comprobar las funciones con casos sencillos y evaluar el estimador completo mediante una simulación con ruido artificial.

1.1. Estimación de la orientación

La estimación de orientación permitirá conocer cómo están orientados los ejes del cohete respecto al sistema de navegación. Por ejemplo, si el cohete se inclina, una lectura en su eje longitudinal ya no corresponde únicamente al movimiento vertical.

El EKF utilizará las medidas del giróscopo para calcular los cambios de orientación. También estimará sus sesgos, es decir, los pequeños errores sistemáticos que pueden hacer que el sensor indique un giro distinto del real. Al integrar las medidas, estos errores se acumulan; por eso necesitamos otras referencias con las que corregir el resultado.

El acelerómetro puede ayudar a identificar la vertical cuando el cohete está en reposo o cuando la aceleración de su movimiento es suficientemente pequeña. Durante el ascenso con motor no podemos confundir la lectura producida por el empuje con una referencia de la gravedad. Tampoco podemos utilizarlo directamente como referencia de gravedad durante el vuelo sin propulsión. En caída libre ideal, sin resistencia aerodinámica, mediría aproximadamente cero; si existe resistencia, medirá la fuerza específica asociada a esa acción [4].

El magnetómetro aportará una referencia del campo magnético terrestre. Compararemos el campo que debería medir según la orientación estimada con el que mide realmente. Para que esta comparación resulte útil, el sensor debe estar calibrado y las medidas no deben estar dominadas por campos de motores, cables u otros elementos cercanos. El magnetómetro no permite corregir por sí solo toda la orientación. Un giro alrededor de la dirección del campo magnético no cambia su lectura ideal. Mientras no exista otra referencia válida, ese giro se seguirá mediante el giróscopo y podrá acumular error.

Una misma lectura del acelerómetro corresponde a direcciones distintas del movimiento según la orientación del cohete. El filtro calcula cómo afectan pequeños errores de orientación a esa transformación y actualiza ese cálculo durante el vuelo.

1.2. Estimación de la posición y la velocidad

Para estimar la posición y la velocidad combinaremos la información inercial con el GNSS y los barómetros en tres dimensiones. Tendremos en cuenta los sesgos de los sensores y el error de orientación, ya que una orientación incorrecta hace que atribuyamos la aceleración a una dirección equivocada [1].

Para predecir el movimiento, transformaremos la fuerza específica medida por el acelerómetro desde los ejes del cohete al sistema de navegación. Después incorporaremos la gravedad para obtener la aceleración del movimiento. Integrando esta aceleración actualizaremos la velocidad y, a partir de ella, la posición. La gravedad deberá representarse de acuerdo con la altura y el sistema de referencia que adoptemos. Tendremos que tener cuidado porque la integración acumula errores del acelerómetro y de la orientación, lo que intentaremos corregir mediante el GNSS y los barómetros mientras estén disponibles.

Cuando llegue una solución GNSS válida, compararemos su posición y velocidad con las que habíamos predicho. El filtro corregirá la estimación según el error observado y la incertidumbre de cada medida. Antes de combinarlas, expresaremos las magnitudes en el mismo sistema de referencia y tendremos en cuenta el instante al que corresponden, para evitar comparar datos de momentos diferentes.

Los barómetros aportarán una referencia de altura a partir de la presión atmosférica. La utilizaremos dentro de su rango válido, considerando que el resultado depende del modelo atmosférico y de cómo se toma la presión en el cohete. Además, la altura barométrica y la del GNSS deben emplear una referencia vertical compatible.

Recordamos de los ejercicios anteriores que disponemos de dos barómetros, lo que nos permitirá comparar las lecturas y utilizar una de ellas cuando la otra no sea válida. Si difieren, no podremos decidir automáticamente cuál es correcta solo por tener dos medidas. Si coinciden, tampoco descartaremos errores comunes, como los debidos a la instalación o al modelo atmosférico. Estos efectos deberán considerarse al asignarles confianza.

Si se pierde el GNSS o la presión queda fuera del rango útil, continuaremos calculando el movimiento con la IMU. Sin embargo, nos exponemos a que el error y la incertidumbre aumenten hasta recuperar una referencia externa válida.

1.3. Comprobación de las medidas y confianza en los sensores

Además de estimar la orientación y el movimiento, necesitamos comprobar que las medidas utilizadas sean adecuadas. Una lectura puede haberse recibido correctamente y, aun así, no servir para corregir el estado: el sensor puede estar saturado, sufrir una perturbación o entregar información demasiado antigua. Primero comprobaremos que cada medida sea nueva, corresponda a un instante conocido y esté dentro de los límites del sensor.

La confianza en las medidas se representará mediante su incertidumbre. Para una medida individual utilizaremos la varianza:

$$R = \sigma^2, \quad (1)$$

donde σ es la desviación típica del error. Una varianza mayor significa que esperamos más error y, en general, que la medida tendrá menos influencia en la corrección.

Cuando se utilicen varias componentes, sus incertidumbres se recogerán en una matriz. Por ejemplo, para la posición GNSS, si consideramos que los errores de los tres ejes NED no están correlacionados:

$$\mathbf{R}_{\text{GNSS},p} = \begin{bmatrix} \sigma_{p_N}^2 & 0 & 0 \\ 0 & \sigma_{p_E}^2 & 0 \\ 0 & 0 & \sigma_{p_D}^2 \end{bmatrix}. \quad (2)$$

Los valores se justificarán mediante las características del receptor, la calidad de la solución y las pruebas. La ausencia de correlación es una hipótesis que deberá comprobarse para la aplicación.

La matriz de covarianza P representa la incertidumbre del estimador. Durante la predicción evoluciona según el movimiento y los errores considerados. Sin referencias externas, la incertidumbre de posición y velocidad tenderá a crecer. Los sesgos se calibrarán o estimarán dentro del filtro: aumentar la incertidumbre de una medida reduce su influencia, pero no elimina un error sistemático.

Al recibir una medida, calculamos la *innovación*: la diferencia entre la lectura recibida y la que esperaba el estimador. Por ejemplo, para el barómetro comparamos la altura obtenida de la presión con la altura predicha.

La ganancia de Kalman determina cómo se utiliza esa diferencia para corregir el estado. Su cálculo considera tanto la incertidumbre de la medida como la de la predicción. Por tanto, la confianza no se asigna mediante un porcentaje fijo para cada sensor: depende de las incertidumbres y de las relaciones entre las variables estimadas.

Antes de aplicar la corrección evaluaremos si la innovación es razonable para la incertidumbre calculada. Una discrepancia importante permitirá detectar una posible anomalía, aunque no demuestra por sí sola un fallo del sensor.

El desarrollo teórico se recoge en el Apéndice A, donde se presentan las ecuaciones de predicción y corrección, el tratamiento de las incertidumbres y el criterio para detectar medidas incompatibles con la estimación [2].

El uso de cada referencia dependerá de la fase de vuelo. Antes del lanzamiento, el acelerómetro podrá ayudar a establecer la vertical. Durante el ascenso propulsado y el vuelo balístico no se utilizará automáticamente con esa finalidad, aunque sus medidas seguirán siendo necesarias para predecir el movimiento. El magnetómetro se utilizará cuando no existan perturbaciones importantes; los barómetros, dentro de su rango útil; y el GNSS cuando proporcione soluciones válidas.

2. Diseño, implementación y simulación de los algoritmos

Implementaremos la fusión en C, separando los cálculos de las funciones que comunican con los sensores. El estimador recibirá las medidas, su instante de adquisición y su indicador de validez. Esta organización permitirá ejecutar el mismo algoritmo con datos simulados y, posteriormente, conectarlo al programa STM32 del ejercicio anterior. La conversión de coordenadas GNSS, la transformación de presión en altura y las comprobaciones de rango, saturación y calidad del sensor pertenecen a una capa previa de preparación de medidas. Las funciones de fusión desarrolladas reciben esas magnitudes ya preparadas y realizan las comprobaciones numéricas, temporales y de innovación descritas a continuación.

2.1. Sistemas de referencia y unidades

Utilizaremos dos sistemas de referencia. El sistema del cuerpo estará unido al cohete: su eje x_b apuntará hacia la punta, y los ejes y_b y z_b serán transversales y completarán un sistema de mano derecha. Fijaremos su orientación en el montaje para que las medidas de los sensores puedan expresarse en estos mismos ejes. Para describir el movimiento utilizaremos un sistema de navegación local NED: norte, este y abajo. Ambos sistemas de referencia se pueden apreciar en la siguiente figura. Su origen se situará en el punto de lanzamiento. Las posiciones se expresarán como desplazamientos respecto a ese origen, y las velocidades seguirán las mismas direcciones.

En este sistema, la componente vertical es positiva hacia abajo. Por tanto, la altura relativa al lanzamiento y la velocidad de ascenso serán:

$$h = -p_D, \quad v_{\text{ascenso}} = -v_D, \quad (3)$$

donde p_D y v_D son la posición y la velocidad en la dirección abajo.

Las medidas del acelerómetro, del giróscopo y del magnetómetro se expresarán en los ejes del cuerpo antes de entrar en el estimador. Si los sensores están montados con otra orientación, aplicaremos una transformación fija para alinear sus lecturas con los ejes definidos.

Al trabajar con SI, las lecturas del ejercicio anterior deberán convertirse:

$$f_{\text{m/s}^2} = f_g g_0, \quad \omega_{\text{rad/s}} = \omega_{\text{grados/s}} \frac{\pi}{180}, \quad (4)$$

con $g_0 = 9,80665 \text{ m/s}^2$. Este factor convierte la lectura expresada en múltiplos de g_0 a unidades del estimador; no obliga a modelar una gravedad constante.

El sistema NED fijo al lanzamiento constituye una aproximación local. Para aplicarlo a un vuelo que alcance el espacio, justificaremos su alcance según la distancia recorrida y la precisión requerida. Si la curvatura o la rotación terrestre tienen un efecto relevante, deberán incorporarse al modelo de navegación.

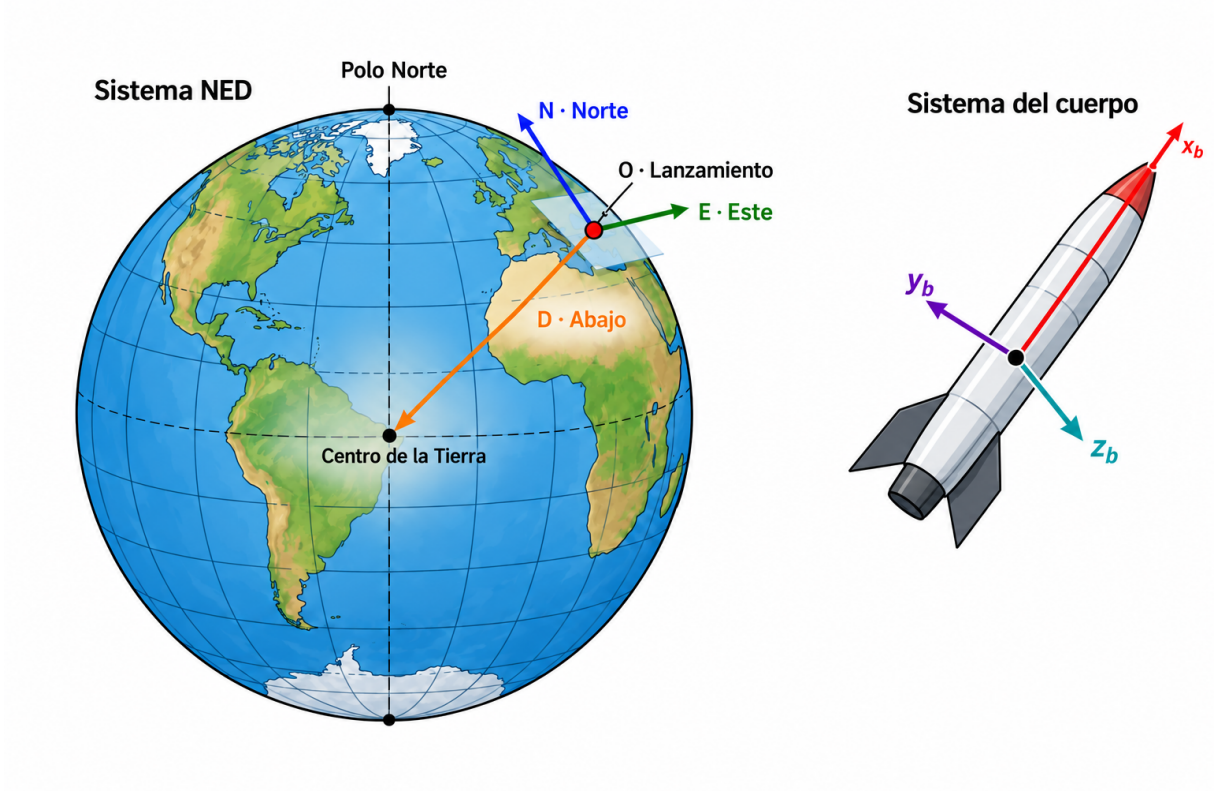


Figura 1: Sistema de navegación local NED, con origen en el punto de lanzamiento, y sistema del cuerpo, unido al cohete. En la representación esférica de la Tierra, el eje D apunta hacia su centro.

2.2. Variables del estimador

El estimador mantendrá la posición, la velocidad, la orientación y los sesgos del acelerómetro y del giróscopo. Estas variables forman el estado:

$$\mathbf{x} = \begin{bmatrix} \mathbf{p}^n \\ \mathbf{v}^n \\ \mathbf{q}_b^n \\ \mathbf{b}_a \\ \mathbf{b}_g \end{bmatrix}. \quad (5)$$

La posición $\mathbf{p}^n$ y la velocidad $\mathbf{v}^n$ tienen tres componentes, correspondientes a norte, este y abajo. La orientación $\mathbf{q}_b^n$ permite transformar vectores desde los ejes del cohete al sistema de navegación.

Representaremos la orientación mediante un cuaternión unitario de cuatro componentes:

$$\mathbf{q}_b^n = [q_0 \ q_1 \ q_2 \ q_3]^T, \quad q_0^2 + q_1^2 + q_2^2 + q_3^2 = 1. \quad (6)$$

Utilizaremos la convención de componente escalar primero. El cuaternión permite representar la orientación sin las singularidades de los ángulos de Euler. Para interpretar las gráficas podremos convertirlo a ángulos, pero los cálculos internos utilizarán el cuaternión [1].

Los vectores $\mathbf{b}_a$ y $\mathbf{b}_g$ representan los sesgos de los tres ejes del acelerómetro y del giróscopo. Los incluimos porque un pequeño error persistente puede acumularse al integrar las medidas. Antes de utilizarlas en la predicción, descontaremos los sesgos estimados:

$$\mathbf{f}_c^b = \mathbf{f}_m^b - \hat{\mathbf{b}}_a, \quad \boldsymbol{\omega}_c^b = \boldsymbol{\omega}_m^b - \hat{\mathbf{b}}_g. \quad (7)$$

Los subíndices m y c indican las medidas recibidas y las corregidas. Esta operación reduce el efecto del sesgo estimado, pero no elimina el ruido ni garantiza que el sesgo se conozca exactamente.

Para aplicar las correcciones utilizaremos un EKF de estado de error. El estado nominal almacena los valores estimados. Definimos el estado de error como:

$$\delta \mathbf{x} = \begin{bmatrix} \delta \mathbf{p}^n \\ \delta \mathbf{v}^n \\ \delta \boldsymbol{\theta} \\ \delta \mathbf{b}_a \\ \delta \mathbf{b}_g \end{bmatrix}. \quad (8)$$

Este error no se conoce directamente. El filtro calcula su estimación $\widehat{\delta \mathbf{x}}$, que utilizaremos para corregir el estado nominal. El error de orientación $\delta \boldsymbol{\theta}$ tiene tres componentes y representa una pequeña rotación. Esta formulación permite corregir la orientación mediante una rotación, respetando la condición de cuaternión unitario.

Guardaremos 16 valores para el estado: tres de posición, tres de velocidad, cuatro del cuaternión y seis de sesgos. La incertidumbre se representará mediante una matriz P de 15×15 , porque el error de orientación tiene tres componentes. Esta matriz permitirá relacionar los errores de orientación con los de velocidad y posición. El Listado 1 muestra cómo empezamos a almacenar las variables del estimador. Esta estructura conserva la posición, la velocidad, la orientación y los sesgos estimados de la IMU, junto con la matriz de incertidumbre.

Los sensores no generan estimaciones independientes dentro de esta estructura. Todos contribuyen a corregir una misma estimación del movimiento: la IMU permite predecir su evolución, mientras que el GNSS, los barómetros y el magnetómetro aportan las referencias descritas anteriormente.

Listado 1: Estructura que almacena el estado del estimador.

```
#define FUSION_DIM_ERROR 15U

typedef struct
{
    /* Movimiento en NED: IMU, GNSS y barometros. */
    float posicion_ned_m[3];      /* m */
    float velocidad_ned_m_s[3];   /* m/s */

    /* Orientacion cuerpo -> NED: escalar, x, y, z. */
    float q_cuerpo_a_ned[4];

    /* Sesgos de la IMU en los ejes del cuerpo. */
    float sesgo_acc_m_s2[3];      /* m/s^2 */
    float sesgo_giro_rad_s[3];    /* rad/s */

    /* Covarianza de los 15 componentes del error. */
    float P[FUSION_DIM_ERROR][FUSION_DIM_ERROR];
} FusionEstado;
```

2.3. Inicialización del estimador

Antes de procesar las medidas, el programa debe establecer el estado inicial y su incertidumbre. Se toma la posición de lanzamiento como origen y se inicia la velocidad a cero, bajo la condición

de que el vehículo esté en reposo, y la orientación inicial se proporciona mediante un cuaternión que transforma los ejes del cuerpo al sistema NED, por una alineación inicial respecto al sistema de navegación.

Al no haber calibrado nuestros sensores, los sesgos del acelerómetro y del giróscopo se inicializan a cero. Esto no supone que los sensores estén libres de error: se asigna una incertidumbre inicial a esos sesgos, que el filtro podrá corregir cuando las medidas y el movimiento permitan estimarlos.

La función recibe las desviaciones típicas iniciales de los 15 componentes del error que hemos definido, y sus cuadrados se introducen en la diagonal de la matriz de covarianza:

$$P_{0,ii} = \sigma_{0,i}^2, \quad i = 1, \dots, 15. \quad (9)$$

Los elementos restantes se inicializan a cero, suponiendo que los errores iniciales no están correlacionados. Esta hipótesis se aplica únicamente a la inicialización: durante la predicción y las correcciones pueden aparecer covarianzas no nulas entre los distintos componentes del error.

El Listado 2 muestra las operaciones principales de `fusion_inicializar`: establecer el origen y el reposo inicial, normalizar la orientación y asignar las varianzas iniciales.

El fragmento se ejecuta después de comprobar que los argumentos sean válidos, que el cuaternión tenga una norma finita y no nula, y que las desviaciones típicas y sus cuadrados sean positivos y finitos. La función completa devuelve cero si alguna comprobación falla, sin modificar el estado, y uno si termina correctamente.

Listado 2: Operaciones principales de inicialización, tras validar los argumentos.

```
/* Origen, velocidad y sesgos inicialmente nulos. */
FusionEstado inicial = {0};

/* Orientacion inicial: normalizacion del cuaternion.
   norma2 se ha calculado y comprobado previamente. */
float norma = sqrtf(norma2);

for (unsigned int i = 0U; i < 4U; ++i)
{
    inicial.q_cuerpo_a_ned[i] = q_inicial[i] / norma;
}

/* Incertidumbre: varianzas en la diagonal de P. */
for (unsigned int i = 0U; i < FUSION_DIM_ERROR; ++i)
{
    inicial.P[i][i] =
        sigma_inicial[i] * sigma_inicial[i];
}

/* Guardamos el estado inicial preparado. */
*estado = inicial;
```

2.4. Predicción mediante la IMU

Utilizaremos la IMU para la estimación del cohete entre dos instantes consecutivos. Partiremos del estado estimado anterior, y utilizaremos sus medidas para actualizar orientación, velocidad y posición de forma independiente al resto de sensores, que influirán más tarde en la corrección.

Empezaremos restando a nuestras lecturas los sesgos estimados y actualizando la orientación con la velocidad angular medida por el giróscopo, para poder transformar la fuerza específica

medida por el acelerómetro al sistema NED. Añadiendo la gravedad obtenemos la aceleración total, con la que propagamos la velocidad y la posición durante el intervalo Δt .

Además del estado estimado, propagamos su matriz de covarianza P . Los errores se transmiten entre variables: una orientación incorrecta altera la transformación de la fuerza específica, lo que afecta a la velocidad y después a la posición. El ruido de la IMU también añade incertidumbre durante el intervalo.

Las correcciones utilizarán esta incertidumbre y la de las medidas para determinar cuánto modificar la estimación. Cada muestra de la IMU permite avanzar desde el instante anterior, utilizando el tiempo transcurrido entre muestras:

$$\Delta t_k = t_k - t_{k-1}. \quad (10)$$

Así, el cálculo dependerá del intervalo real y no únicamente de la frecuencia configurada.

Primero convertiremos las medidas de la IMU a las unidades del estimador y descontaremos los sesgos estimados, según las ecuaciones 4 y 7. El Listado 3 muestra las operaciones realizadas por `fusion_preparar_imu` para cada eje. Las entradas deben ser válidas y estar expresadas en los ejes del cuerpo.

Listado 3: Conversión de unidades y corrección de sesgos de la IMU.

```
const float g0 = 9.80665f;
const float grados_a_radianes = 0.01745329252f;

for (unsigned int i = 0U; i < 3U; ++i)
{
    /* Acelerometro: fuerza especifica en m/s^2. */
    fuerza_corregida[i] =
        aceleracion_g[i] * g0 - estado->sesgo_acc_m_s2[i];

    /* Giroscopio: velocidad angular en rad/s. */
    giro_corregido[i] =
        giro_dps[i] * grados_a_radianes
        - estado->sesgo_giro_rad_s[i];
}
```

Con la velocidad angular corregida calcularemos la rotación producida durante el intervalo:

$$\Delta \theta_k = \omega_{c,k}^b \Delta t_k. \quad (11)$$

Esta expresión supone que la velocidad angular puede considerarse constante durante ese intervalo.

Para representar este giro mediante un cuaternión calculamos el ángulo recorrido:

$$\alpha_k = \|\omega_{c,k}^b\| \Delta t_k. \quad (12)$$

Si la velocidad angular no es nula, el cuaternión del incremento será:

$$\Delta \mathbf{q}_k = \begin{bmatrix} \cos(\alpha_k/2) \\ \frac{\omega_{c,k}^b}{\|\omega_{c,k}^b\|} \sin(\alpha_k/2) \end{bmatrix}. \quad (13)$$

La primera componente es escalar y las tres restantes forman la parte vectorial, cuya dirección indica el eje de giro y cuyo módulo es el seno del semiángulo. Si la velocidad angular es cero, utilizamos el cuaternión identidad $[1, 0, 0, 0]^T$, que representa un incremento nulo.

El Listado 4 muestra la conversión realizada por `fusion_incremento_rotacion` para el caso de velocidad angular no nula, tras validar las entradas. La función supone velocidad angular constante durante el intervalo.

Listado 4: Cuaternión del incremento de giro medido por el giróscopo.

```
float velocidad = hypotf(
    hypotf(giro_rad_s[0], giro_rad_s[1]),
    giro_rad_s[2]
);
float angulo = velocidad * dt;
float factor = sinf(0.5f * angulo) / velocidad;

dq[0] = cosf(0.5f * angulo);
dq[1] = factor * giro_rad_s[0];
dq[2] = factor * giro_rad_s[1];
dq[3] = factor * giro_rad_s[2];
```

Este incremento se combinará con la orientación anterior:

$$\mathbf{q}_k^- = \mathbf{q}_{k-1}^+ \otimes \Delta \mathbf{q}_k. \quad (14)$$

Los superíndices $-$ y $+$ indican, respectivamente, la estimación predicha y la obtenida tras incorporar las correcciones disponibles. No representan signos algebraicos. El sombrero identifica una magnitud estimada; cuando no exista ambigüedad, lo omitiremos en las expresiones de propagación del estado nominal. El producto se implementa mediante la función `fusion_producto_cuaterniones`, mostrada en el Listado 5. Se calcula primero en un vector auxiliar para permitir que la salida ocupe la misma memoria que una entrada.

Listado 5: Producto de Hamilton para combinar la orientación y el incremento de giro.

```
float producto[4];

producto[0] =
    a[0]*b[0] - a[1]*b[1] - a[2]*b[2] - a[3]*b[3];

producto[1] =
    a[0]*b[1] + a[1]*b[0] + a[2]*b[3] - a[3]*b[2];

producto[2] =
    a[0]*b[2] - a[1]*b[3] + a[2]*b[0] + a[3]*b[1];

producto[3] =
    a[0]*b[3] + a[1]*b[2] - a[2]*b[1] + a[3]*b[0];

for (unsigned int i = 0U; i < 4U; ++i)
{
    resultado[i] = producto[i];
}
```

En la predicción, $\mathbf{a}$ será la orientación anterior y $\mathbf{b}$ el cuaternión del incremento. Después del producto normalizamos el cuaternión:

$$\mathbf{q} \leftarrow \frac{\mathbf{q}}{\|\mathbf{q}\|}. \quad (15)$$

La función `fusion_normalizar_cuaternion` comprueba que sus componentes sean finitas y que la norma no sea demasiado próxima a cero. Tras estas comprobaciones aplica la operación del Listado 6. La normalización conserva la rotación representada, pero no elimina los errores de orientación acumulados por el giróscopo.

Listado 6: Normalización del cuaternión tras comprobar su validez.

```
float norma = hypotf(
    hypotf(q[0], q[1]),
    hypotf(q[2], q[3])
);

/* Tras comprobar que la norma es válida: */
for (unsigned int i = 0U; i < 4U; ++i)
{
    q[i] /= norma;
}
```

A partir del cuaternión normalizado obtendremos la matriz de rotación $\mathbf{C}_b^n$ mediante `fusion_matriz_rotacion`. Esta matriz permite expresar un vector del cuerpo en los ejes NED:

$$\mathbf{f}_c^n = \mathbf{C}_b^n(\mathbf{q}) \mathbf{f}_c^b. \quad (16)$$

Las filas de la matriz corresponden a norte, este y abajo; sus columnas, a los ejes X, Y y Z del cuerpo. Por ejemplo, su primera columna representa la dirección del eje X del cohete expresada en NED.

El Listado 7 muestra la llamada que utilizaremos para obtenerla. Las expresiones utilizadas, la implementación de la función y su integración en la predicción se recogen en el Apéndice B.

Listado 7: Obtención de la matriz de rotación a partir de la orientación normalizada.

```
float C[3][3];
fusion_matriz_rotacion(q, C);
```

La aceleración del movimiento se calculará sumando la gravedad, ya en ejes NED (recordamos que apunta hacia el centro de la Tierra, bajo la aproximación esférica adoptada):

$$\mathbf{a}_k^n = \mathbf{C}_b^n(\mathbf{q}) \mathbf{f}_{c,k}^b + \mathbf{g}^n. \quad (17)$$

Para transformar la fuerza específica utilizaremos la orientación estimada en el punto medio del intervalo. La obtendremos aplicando a la orientación anterior el incremento de giro calculado con la velocidad angular corregida y un tiempo $\Delta t_k/2$. La orientación al final del intervalo se calculará también a partir de la orientación anterior, pero utilizando el intervalo completo Δt_k . La función del Listado 8 aplica esta operación. Primero transforma la fuerza específica corregida a los ejes NED mediante la matriz de orientación. Después suma la gravedad a la componente vertical, cuyo sentido positivo es hacia abajo.

La gravedad se recibe como argumento para permitir su actualización según la altura. El cuaternión utilizado debe estar normalizado y representar la orientación empleada para transformar la medida.

Listado 8: Cálculo de la aceleración en el sistema NED.

```
float C[3][3];
fusion_matriz_rotacion(q, C);

/* Fuerza especifica del cuerpo a norte, este y abajo. */
```

```

for (unsigned int i = 0U; i < 3U; ++i)
{
    aceleracion[i] =
        C[i][0] * fuerza_corregida[0]
        + C[i][1] * fuerza_corregida[1]
        + C[i][2] * fuerza_corregida[2];
}

/* Sumar la gravedad en el sentido positivo de abajo. */
aceleracion[2] += gravedad_m_s2;

```

Como comprobación, un vehículo en reposo con sus ejes alineados con NED presenta una fuerza específica vertical de $-g$. Al sumar la gravedad, la aceleración resultante es cero, como corresponde a esa situación. Omitiremos por el momento efectos asociados a rotación terrestre y desplazamiento del sistema de navegación.

Suponiendo una aceleración constante durante el intervalo, actualizaremos la velocidad y la posición:

$$\mathbf{v}_k^- = \mathbf{v}_{k-1}^+ + \mathbf{a}_k^n \Delta t_k, \quad (18)$$

$$\mathbf{p}_k^- = \mathbf{p}_{k-1}^+ + \mathbf{v}_{k-1}^+ \Delta t_k + \frac{1}{2} \mathbf{a}_k^n \Delta t_k^2. \quad (19)$$

El Listado 9 implementa la actualización de posición y velocidad para cada eje NED. Se aproxima la aceleración como constante durante el intervalo entre muestras. La posición se calcula utilizando la velocidad al comienzo de ese intervalo.

Listado 9: Integración de la aceleración para actualizar el movimiento.

```

for (unsigned int i = 0U; i < 3U; ++i)
{
    float p = estado->posicion_ned_m[i];
    float v = estado->velocidad_ned_m_s[i];
    float a = aceleracion_ned[i];

    nueva_posicion[i] =
        p + v * dt + 0.5f * a * dt * dt;

    nueva_velocidad[i] = v + a * dt;
}

```

La función completa comprueba que el intervalo sea positivo y que los datos y resultados sean finitos antes de modificar el estado.

La función `fusion_prededir_nominal` condensa las operaciones anteriores, pero todavía no propaga la matriz de covarianza. Recordamos que se aproxima la velocidad angular y la fuerza específica como constantes entre muestras. La implementación trabaja sobre una copia del estado y solo guarda el resultado si todas las operaciones terminan correctamente. La propagación de la matriz de incertidumbre se realiza por separado y se coordina con esta operación mediante `fusion_prededir`.

Los sesgos conservarán su valor estimado durante la predicción. Su posible variación se representará mediante el ruido del proceso. También actualizaremos la matriz de incertidumbre P con el modelo del estado de error: seguir calculando posición y velocidad no implica que su precisión permanezca constante.

Esta predicción se realizará con muestras nuevas y válidas. Si la IMU deja de entregar datos o alguno de sus canales se satura, no trataremos la última lectura como si fuera una muestra

nueva. El tratamiento de esos intervalos deberá reflejar la pérdida de información y el aumento de incertidumbre.

2.5. Propagación de la incertidumbre

Además de predecir el movimiento, debemos actualizar la incertidumbre de la estimación. Las relaciones entre los errores y su evolución se representan mediante el modelo linealizado del estado de error:

$$\delta\dot{\mathbf{x}} = A \delta\mathbf{x} + G \mathbf{n}, \quad (20)$$

El vector $\delta\mathbf{x}$ contiene los errores de posición, velocidad y orientación, así como las diferencias entre los sesgos reales y estimados. Su derivada temporal, $\delta\dot{\mathbf{x}}$, expresa cómo cambian esos errores.

El término $A \delta\mathbf{x}$ representa cómo los errores existentes afectan a su evolución. Por ejemplo, un error de velocidad produce una variación del error de posición, mientras que un error de orientación altera la transformación de la fuerza específica y afecta al error de velocidad. La matriz A recoge estas relaciones mediante una aproximación lineal alrededor del estado estimado.

El término $G \mathbf{n}$ representa la contribución de los ruidos. El vector $\mathbf{n}$ agrupa el ruido del acelerómetro y del giróscopo, junto con los ruidos que modelan la posible variación de sus sesgos. La matriz G determina sobre qué componentes del error actúa cada ruido y en qué sistema de referencia.

Los errores de posición, velocidad y sesgos se definen como valor real menos estimado. Para la orientación utilizamos una pequeña rotación expresada en los ejes del cuerpo y aplicada a la derecha:

$$\mathbf{q}_{\text{real}} = \hat{\mathbf{q}} \otimes \delta\mathbf{q}, \quad \delta\mathbf{q} \approx \begin{bmatrix} 1 \\ \frac{1}{2}\delta\boldsymbol{\theta} \end{bmatrix}. \quad (21)$$

Con esta convención y el orden de la ecuación (8), la linealización resulta [5]:

$$A = \begin{bmatrix} 0_3 & I_3 & 0_3 & 0_3 & 0_3 \\ 0_3 & 0_3 & -\hat{\mathbf{C}}_b^n [\mathbf{f}_c^b]_{\times} & -\hat{\mathbf{C}}_b^n & 0_3 \\ 0_3 & 0_3 & -[\boldsymbol{\omega}_c^b]_{\times} & 0_3 & -I_3 \\ 0_3 & 0_3 & 0_3 & 0_3 & 0_3 \\ 0_3 & 0_3 & 0_3 & 0_3 & 0_3 \end{bmatrix}, \quad (22)$$

donde $\hat{\mathbf{C}}_b^n = \mathbf{C}_b^n(\hat{\mathbf{q}})$ es la matriz de rotación calculada a partir de la orientación estimada y transforma vectores del cuerpo a NED y $[\mathbf{u}]_{\times}$ es la matriz que cumple $[\mathbf{u}]_{\times} \mathbf{v} = \mathbf{u} \times \mathbf{v}$. Cada bloque tiene dimensiones 3×3 .

Las tres primeras filas de bloques describen la evolución de los errores de posición, velocidad y orientación. Las filas de los sesgos son nulas porque su valor lo consideramos constante; su posible variación se incorporará mediante el ruido del proceso. Este modelo omite la rotación terrestre y la sensibilidad de la gravedad a la posición.

En C, `fusion_matriz_producto_vectorial` construye la matriz del producto vectorial;

`fusion_jacobianos_aceleracion` calcula las sensibilidades de la aceleración; y

`fusion_matriz_error_continuo` reúne los bloques anteriores en A .

A partir de A obtendremos la matriz de transición Φ_k , que describe cómo un error en un instante se transmite en el siguiente. Para calcular la transición se aproxima A como constante durante

cada intervalo y se utiliza el desarrollo de segundo orden de la exponencial matricial:

$$\Phi_k = e^{A\Delta t_k} \approx I_{15} + A\Delta t_k + \frac{1}{2}A^2\Delta t_k^2. \quad (23)$$

Aquí, A^2 representa el producto matricial AA . El término de segundo orden recoge también efectos que se transmiten a través de una variable intermedia, como la influencia del sesgo del acelerómetro sobre la posición a través de la velocidad. El Listado 10 muestra el cálculo principal de la transición. Previamente se ha calculado $B = A\Delta t$ y se han validado los argumentos. Los índices i y j seleccionan el elemento de salida; el índice k realiza el producto de una fila de B por una columna de B .

Listado 10: Calculo de la transicion del error mediante una aproximacion de segundo orden.

```
/* B = A * dt, calculada previamente. */

for (unsigned int i = 0U; i < FUSION_DIM_ERROR; ++i)
{
    for (unsigned int j = 0U; j < FUSION_DIM_ERROR; ++j)
    {
        /* Elemento (i,j) del producto matricial B*B. */
        float producto = 0.0f;

        for (unsigned int k = 0U; k < FUSION_DIM_ERROR; ++k)
        {
            producto += B[i][k] * B[k][j];
        }

        /* Identidad: uno en la diagonal y cero fuera. */
        float identidad = (i == j) ? 1.0f : 0.0f;

        resultado[i][j] =
            identidad + B[i][j] + 0.5f * producto;
    }
}
```

La covarianza se propagará mediante:

$$P_k^- = \Phi_k P_{k-1}^+ \Phi_k^T + Q_{d,k}. \quad (24)$$

El primer término calcula cómo llega al nuevo instante la incertidumbre que ya teníamos, incluyendo cómo se transmite entre posición, velocidad, orientación y sesgos. La matriz $Q_{d,k}$ añade la incertidumbre introducida durante ese intervalo por el ruido de la IMU y por la posible variación de sus sesgos.

Configuramos el ruido del proceso mediante cuatro parámetros: q_a y q_g para el ruido del acelerómetro y del giróscopo, y q_{b_a} y q_{b_g} para la posible variación de sus sesgos. Estos parámetros determinan la incertidumbre que se añade durante la predicción.

En esta implementación se supone igual intensidad en los tres ejes de cada grupo e independencia entre las fuentes. Su matriz de intensidad es:

$$Q_c = \text{diag} \left(q_a I_3, q_g I_3, q_{b_a} I_3, q_{b_g} I_3 \right). \quad (25)$$

Esta matriz tiene dimensiones 12×12 : agrupa las doce componentes de ruido, no los quince componentes del estado de error.

Los parámetros se almacenan en la estructura `FusionRuido`. Son intensidades continuas, no varianzas de muestras individuales. Por ejemplo, la contribución directa a la varianza de cada componente del sesgo del acelerómetro durante Δt es $q_{b_a} \Delta t$.

Los sesgos nominales permanecen constantes durante la predicción; el paseo aleatorio describe su incertidumbre y no exige introducir perturbaciones aleatorias en el estado estimado.

La matriz G relaciona las doce fuentes de ruido con los quince componentes del estado de error. Ordenando las fuentes como ruido del acelerómetro, del giróscopo y de variación de sus respectivos sesgos, resulta:

$$G = \begin{bmatrix} 0_3 & 0_3 & 0_3 & 0_3 \\ -C_b^n & 0_3 & 0_3 & 0_3 \\ 0_3 & -I_3 & 0_3 & 0_3 \\ 0_3 & 0_3 & I_3 & 0_3 \\ 0_3 & 0_3 & 0_3 & I_3 \end{bmatrix}. \quad (26)$$

Sus filas de bloques corresponden a posición, velocidad, orientación y los dos grupos de sesgos. El bloque $-C_b^n$ transforma el ruido del acelerómetro desde el cuerpo a NED. En la implementación no se almacena G : se construye directamente el producto GQ_cG^T , aprovechando las hipótesis adoptadas para el ruido.

La intensidad del ruido expresada en las coordenadas del estado de error se obtiene mediante:

$$W = GQ_cG^T = \text{diag} \left(0_3, q_a I_3, q_g I_3, q_{b_a} I_3, q_{b_g} I_3 \right). \quad (27)$$

El bloque de posición es nulo porque no se introduce una fuente independiente directamente en su derivada. El ruido alcanza la posición a través de la velocidad. Por tanto, este bloque nulo no implica ausencia de incertidumbre en la posición.

La función `fusion_intensidad_ruido_estado` valida los cuatro parámetros y construye W . El Listado 11 muestra la asignación principal, después de inicializar la matriz a cero.

Listado 11: Asignacion de las intensidades al estado de error.

```
for (unsigned int i = 0U; i < 3U; ++i)
{
    W[3U + i][3U + i] = intensidades[0];
    W[6U + i][6U + i] = intensidades[1];
    W[9U + i][9U + i] = intensidades[2];
    W[12U + i][12U + i] = intensidades[3];
}
```

La matriz W representa una intensidad continua. Para obtener $Q_{d,k}$ debemos considerar cómo se acumula y se propaga su efecto durante el intervalo entre muestras. Manteniendo el modelo constante durante el intervalo, la covarianza del ruido acumulado se expresa como:

$$Q_{d,k} = \int_0^{\Delta t_k} \Phi(\tau) W \Phi(\tau)^T d\tau \approx \frac{\Delta t_k}{6} \left[W + 4\Phi_m W \Phi_m^T + \Phi_f W \Phi_f^T \right], \quad (28)$$

aproximándola mediante la regla de Simpson con la función `fusion_ruido_discreto`. La transición dentro de la integral considera cómo el ruido se transmite entre las variables durante el intervalo, donde $\Phi_m = \Phi(\Delta t_k/2)$ y $\Phi_f = \Phi(\Delta t_k)$ se calculan mediante la aproximación de segundo orden implementada.

Como W es diagonal bajo las hipótesis adoptadas, cada elemento del producto se obtiene mediante:

$$(\Phi W \Phi^T)_{ij} = \sum_{\ell=1}^{15} \Phi_{i\ell} W_{\ell\ell} \Phi_{j\ell}. \quad (29)$$

Listado 12: Elemento del ruido propagado aprovechando que W es diagonal.

```
float suma = 0.0f;
```

```

for (unsigned int k = 0U; k < FUSION_DIM_ERROR; ++k)
{
    suma += Phi[i][k] * W[k][k] * Phi[j][k];
}

```

La implementación calcula un triángulo de la matriz y refleja sus elementos para conservar la simetría. Solo entrega el resultado si los cálculos son finitos.

Una vez calculadas Φ_k y $Q_{d,k}$, la función `fusion_propagar_covarianza` actualiza P mediante la ecuación (24). El producto se realiza en dos etapas:

$$M = \Phi_k P_{k-1}^+, \quad P_k^- = M \Phi_k^T + Q_{d,k}. \quad (30)$$

La función utiliza matrices auxiliares y solo sustituye P si los cálculos terminan correctamente. Comprueba que las entradas y resultados sean finitos y rechaza varianzas finales negativas.

Para limitar pequeñas asimetrías de redondeo, se aplica:

$$P \leftarrow \frac{1}{2}(P + P^T). \quad (31)$$

La simetría y las varianzas no negativas no bastan para garantizar una covarianza válida, ni demuestran que represente bien los errores observados.

La función modifica únicamente la covarianza. Su ejecución deberá coordinarse con la predicción nominal, utilizando el mismo intervalo y una linealización correspondiente a ese paso. La función `fusion_predecir` coordina la predicción nominal y la propagación de la covarianza para una misma muestra de la IMU y un mismo intervalo.

Primero convierte las medidas, resta los sesgos y calcula la orientación a mitad del intervalo. Con esa orientación y las medidas corregidas construye A , que se aproxima como constante durante el paso. Después calcula Φ_k , $Q_{d,k}$ y la covarianza predicha.

Finalmente llama a `fusion_predecir_nominal` para actualizar posición, velocidad y orientación. Esta última función conserva la covarianza calculada previamente. El Listado 13 resume la secuencia principal.

Listado 13: Secuencia principal de la predicción conjunta.

```

fusion_matriz_error_continuo(
    q_mitad, fuerza, giro, A
);

if (!fusion_matriz_transicion(A, dt, Phi))
    return 0;

if (!fusion_ruido_discreto(A, ruido, dt, Qd))
    return 0;

if (!fusion_propagar_covarianza(&siguiente, Phi, Qd))
    return 0;

if (!fusion_predecir_nominal(
    &siguiente, aceleracion_g, giro_dps,
    dt, gravedad_m_s2))
    return 0;

*estado = siguiente;

```

Todas las operaciones se realizan sobre una copia. El estado original solo se sustituye cuando la predicción nominal y la de covarianza terminan correctamente.

Con `fusion_predecir` obtenemos el estado y la covarianza al final del intervalo. La estimación queda preparada para incorporar las medidas externas de ese instante. Su comportamiento se comprobará primero en reposo y después con las trayectorias simuladas. La validación con sensores simulados ruidosos se realizará posteriormente.

2.6. Corrección mediante el GNSS

La IMU permite seguir el movimiento entre soluciones GNSS, pero sus errores se acumulan al integrar las medidas. El GNSS aporta una referencia externa para corregir la posición y la velocidad estimadas. Antes de aplicar la corrección, comprobaremos que la solución sea nueva y válida. También necesitaremos su instante de adquisición y una estimación de su precisión.

La posición GNSS se transformará al sistema de navegación definido para el estimador. Para obtener desplazamientos respecto al lanzamiento, convertiremos las coordenadas geográficas a cartesianas terrestres y, después, al sistema NED local. La velocidad se expresará en esos mismos ejes. Así evitaremos comparar directamente latitud y longitud con posiciones calculadas en metros.

La medida utilizada será:

$$\mathbf{z}_{\text{GNSS}} = \begin{bmatrix} \mathbf{p}_{\text{GNSS}}^n \\ \mathbf{v}_{\text{GNSS}}^n \end{bmatrix}. \quad (32)$$

La innovación recogerá las diferencias entre las medidas y la predicción:

$$\mathbf{r}_{\text{GNSS}} = \begin{bmatrix} \mathbf{p}_{\text{GNSS}}^n - \hat{\mathbf{p}}^- \\ \mathbf{v}_{\text{GNSS}}^n - \hat{\mathbf{v}}^- \end{bmatrix}. \quad (33)$$

Ambas deben corresponder al mismo instante. Si existe un retraso relevante en la recepción, no aplicaremos la medida como si describiera el estado actual.

La matriz de medida H describe cómo se refleja cada componente del error del estado en esa innovación:

$$\mathbf{r} \approx H \delta \mathbf{x} + \mathbf{n}, \quad (34)$$

donde $\mathbf{n}$ representa el ruido de la medida. Cada fila de H corresponde a una componente medida y cada columna a una componente del estado de error. Por tanto, H permite expresar qué errores afectan directamente a la medida y con qué signo y magnitud.

En el caso del GNSS, las medidas proporcionan directamente la posición y la velocidad. Suponiendo que la antena y la IMU están en el mismo punto, y que las medidas están expresadas en el sistema NED del estimador, tendremos:

$$\mathbf{r}_{\text{GNSS}} = \begin{bmatrix} \delta \mathbf{p} \\ \delta \mathbf{v} \end{bmatrix} + \mathbf{n}_{\text{GNSS}}. \quad (35)$$

Para el orden de componentes del estado de error definido en la ecuación (8), esta relación se representa mediante:

$$H_{\text{GNSS}} = \begin{bmatrix} I_3 & 0_3 & 0_3 & 0_3 & 0_3 \\ 0_3 & I_3 & 0_3 & 0_3 & 0_3 \end{bmatrix}. \quad (36)$$

Aquí, I_3 es la identidad de 3×3 y 0_3 una matriz de ceros del mismo tamaño. La matriz completa tiene dimensiones 6×15 : relaciona los quince componentes del error con las seis componentes medidas por el GNSS.

La primera fila de bloques selecciona el error de posición y la segunda el error de velocidad. Los bloques restantes son nulos porque, bajo estas hipótesis, la medida no depende directamente de la orientación ni de los sesgos. Esto no impide corregir esas variables indirectamente a través de las correlaciones almacenadas en P .

Si la separación entre la antena y la IMU resulta relevante, deberá incluirse en el modelo de medida y adaptar la matriz H en consecuencia.

La confianza se recogerá en una matriz R_{GNSS} de 6×6 . Si suponemos errores no correlacionados:

$$R_{\text{GNSS}} = \text{diag} \left(\sigma_{p_N}^2, \sigma_{p_E}^2, \sigma_{p_D}^2, \sigma_{v_N}^2, \sigma_{v_E}^2, \sigma_{v_D}^2 \right). \quad (37)$$

Para implementar la corrección GNSS, comenzamos por una componente escalar, como la posición norte. La función de cálculo de la ganancia será común a otras medidas escalares. Para cada componente, H será la fila correspondiente de H_{GNSS} y R su varianza. Para esta posición:

$$r = p_{\text{GNSS},N} - \hat{p}_N^-, \quad H = \begin{bmatrix} 1 & \underbrace{0 \cdots 0}_{14} \end{bmatrix}, \quad R = \sigma_{p_N}^2. \quad (38)$$

A partir de la innovación y de las incertidumbres, calculamos:

$$S = HP^-H^T + R, \quad K = \frac{P^-H^T}{S}. \quad (39)$$

Se supone que el ruido de medida no está correlacionado con el error de predicción. S es la varianza escalar de la innovación y K es un vector de 15 componentes. Cada componente de K determina cómo influye la innovación en la corrección de un componente del estado de error.

La discrepancia se valora mediante la innovación normalizada al cuadrado:

$$\text{NIS} = \frac{r^2}{S}. \quad (40)$$

Este valor permite comparar la diferencia observada con la incertidumbre prevista. Se utilizará para comprobar la compatibilidad de la medida antes de aplicar la corrección, conforme al criterio descrito en el Apéndice A. Una discrepancia elevada no identifica por sí sola un fallo del receptor.

La función `fusion_preparar_correccion_escalar` calcula S , K y la NIS sin modificar el estado ni su covarianza. Recibe la innovación, la fila H y la varianza R ; comprueba que los argumentos sean válidos y que R y S sean positivas y finitas.

Primero obtiene el vector auxiliar PH^T y después realiza las operaciones principales del Listado 14. Los resultados se almacenan en la estructura `FusionCorreccionEscalar`.

Listado 14: Varianza de innovacion y ganancia para una medida escalar.

```
/* PHt contiene el producto P * H^T. */
float S = R;

for (unsigned int i = 0U; i < FUSION_DIM_ERROR; ++i)
{
    S += H[i] * PHt[i];
}

/* Tras comprobar que S sea positiva y finita: */
for (unsigned int i = 0U; i < FUSION_DIM_ERROR; ++i)
{
    resultado.ganancia[i] = PHt[i] / S;
```

```

}

/* Innovacion normalizada al cuadrado. */
float innovacion_normalizada = innovacion / sqrtf(S);

resultado.nis =
    innovacion_normalizada * innovacion_normalizada;

```

La corrección propuesta del estado de error será:

$$\delta \hat{\mathbf{x}} = K r. \quad (41)$$

La función `fusion_inyectar_error_nominal` aplica las componentes de $\delta \hat{\mathbf{x}}$ al estado nominal. La posición, la velocidad y los sesgos se actualizan de la siguiente manera:

$$\hat{\mathbf{p}}^+ = \hat{\mathbf{p}}^- + \delta \hat{\mathbf{p}}, \quad (42)$$

$$\hat{\mathbf{v}}^+ = \hat{\mathbf{v}}^- + \delta \hat{\mathbf{v}}, \quad (43)$$

$$\hat{\mathbf{b}}_a^+ = \hat{\mathbf{b}}_a^- + \delta \hat{\mathbf{b}}_a, \quad (44)$$

$$\hat{\mathbf{b}}_g^+ = \hat{\mathbf{b}}_g^- + \delta \hat{\mathbf{b}}_g. \quad (45)$$

Para la orientación, el vector $\delta \hat{\boldsymbol{\theta}}$ se convierte en un cuaternión de corrección. Definiendo $\alpha = \|\delta \hat{\boldsymbol{\theta}}\|$:

$$\delta \hat{q} = \begin{bmatrix} \cos(\alpha/2) \\ \frac{\sin(\alpha/2)}{\alpha} \delta \hat{\boldsymbol{\theta}} \end{bmatrix}, \quad \hat{q}^+ = \hat{q}^- \otimes \delta \hat{q}. \quad (46)$$

Para α próximo a cero se utiliza $\sin(\alpha/2)/\alpha \approx 1/2$. La corrección se aplica a la derecha, de acuerdo con la convención local adoptada, y el cuaternión resultante se normaliza. El modelo linealizado sigue requiriendo errores de orientación pequeños.

El Listado 15 recoge las actualizaciones principales, omitiendo las comprobaciones de validez.

Listado 15: Aplicacion de la correccion al estado nominal.

```

for (unsigned int i = 0U; i < 3U; ++i)
{
    siguiente.posicion_ned_m[i] += delta[i];
    siguiente.velocidad_ned_m_s[i] += delta[3U + i];
    siguiente.sesgo_acc_m_s2[i] += delta[9U + i];
    siguiente.sesgo_giro_rad_s[i] += delta[12U + i];
}

/* dq se obtiene del vector de correccion angular. */
fusion_producto_cuaterniones(
    siguiente.q_cuerpo_a_ned, dq, q_corregido
);

```

La función trabaja sobre una copia y conserva el estado original si alguna comprobación falla. Esta operación modifica únicamente el estado nominal: para completar la corrección será necesario actualizar la covarianza.

La covarianza se actualiza mediante la forma de Joseph:

$$P_{\text{Joseph}} = (I_{15} - KH)P^-(I_{15} - KH)^T + KRK^T. \quad (47)$$

Con la que incorporamos la información de la medida a la incertidumbre del estado de error. La ganancia debe corresponder a la misma covarianza previa y a los mismos valores de H y R utilizados en esta actualización.

La función `fusion_corregir_covarianza_escalar` construye las matrices:

$$M = I_{15} - KH, \quad U = KRK^T, \quad (48)$$

y calcula $P_{\text{Joseph}} = MP^-M^T + U$.

Listado 16: Construcción de las matrices de la forma de Joseph.

```
for (unsigned int i = 0U; i < FUSION_DIM_ERROR; ++i)
{
    for (unsigned int j = 0U; j < FUSION_DIM_ERROR; ++j)
    {
        float identidad = (i == j) ? 1.0f : 0.0f;

        M[i][j] = identidad - K[i] * H[j];
        U[i][j] = K[i] * R * K[j];
    }
}
```

La operación conserva el estado nominal y modifica únicamente P , tras comprobar los resultados y limitar las asimetrías de redondeo.

Después de corregir el cuaternión, el error de orientación se mide respecto a una orientación estimada diferente. La covarianza calculada con Joseph todavía utiliza la referencia anterior. El jacobiano de reinicio adapta esa covarianza a la orientación corregida, incluyendo su relación con los errores de posición, velocidad y sesgos.

Para el error local aplicado a la derecha, el jacobiano angular de reinicio se aproxima a primer orden mediante:

$$J_\theta = I_3 - \frac{1}{2}[\delta\hat{\theta}]_\times. \quad (49)$$

Esta aproximación requiere correcciones angulares pequeñas [5].

La transformación completa es:

$$J_{\text{reset}} = \text{diag}(I_3, I_3, J_\theta, I_3, I_3), \quad P^+ = J_{\text{reset}}P_{\text{Joseph}}J_{\text{reset}}^T. \quad (50)$$

Aunque solo el bloque angular del jacobiano difiere de la identidad, esta operación transforma también las covarianzas entre orientación y las demás variables. La función `fusion_reiniciar_covarianza` construye el jacobiano y aplica la transformación sin añadir ruido ni modificar el estado nominal. El Listado 17 muestra la construcción del bloque angular, partiendo de $J_{\text{reset}} = I_{15}$.

Listado 17: Bloque angular del jacobiano de reinicio.

```
/* S contiene [delta_theta]_x.
   J se ha inicializado como identidad. */
for (unsigned int i = 0U; i < 3U; ++i)
{
    for (unsigned int j = 0U; j < 3U; ++j)
    {
        J[6U + i][6U + j] -= 0.5f * S[i][j];
    }
}
```

El reinicio establece en cero la estimación del error residual después de incorporar la corrección del estado. No elimina la incertidumbre ni pone a cero los sesgos estimados. En la implementación, el vector de corrección es temporal y no se vuelve a aplicar.

La función `fusion_corregir_escalor` reúne las operaciones anteriores. Primero calcula la ganancia y la NIS. La medida se rechaza si:

$$\text{NIS} > \gamma, \quad (51)$$

donde γ es un umbral configurable. Su elección seguirá el criterio estadístico del Apéndice A y se revisará mediante las pruebas del estimador.

Si la medida supera la comprobación de compatibilidad, se calcula $\delta\hat{\mathbf{x}} = K\mathbf{r}$, se actualiza la covarianza mediante Joseph, se inyecta la corrección en el estado nominal y se transforma la covarianza mediante el jacobiano de reinicio.

El Listado 18 muestra las llamadas principales una vez aceptada la medida y calculado el vector de corrección.

Listado 18: Aplicacion conjunta de una correccion escalar aceptada.

```
if (!fusion_corregir_covarianza_escalor(
    &siguiente, H, correccion.ganancia, R))
    return FUSION_CORRECCION_ERROR;

if (!fusion_inyectar_error_nominal(&siguiente, delta))
    return FUSION_CORRECCION_ERROR;

if (!fusion_reiniciar_covarianza(
    &siguiente, &delta[6]))
    return FUSION_CORRECCION_ERROR;

*estado = siguiente;
```

Todas las operaciones se realizan sobre una copia, solo sustituida cuando se completa toda la actualización. La función distingue entre corrección aplicada, medida rechazada y error de cálculo.

La innovación y la fila H deben corresponder al estado recibido por la función y al instante de la medida. La comprobación de la NIS no sustituye las verificaciones de validez ni antigüedad de la medida.

Bajo la hipótesis de errores de medida no correlacionados, la función `fusion_corregir_gnss` procesa las componentes:

$$p_N, p_E, p_D, v_N, v_E, v_D. \quad (52)$$

Para cada componente seleccionada, se construye la fila H correspondiente y se calcula la innovación utilizando el estado vigente. Una corrección aceptada modifica el estado y P antes de procesar la siguiente componente. Por tanto, no se reutilizan todas las innovaciones calculadas respecto a la predicción inicial.

El Listado 19 muestra la selección de la variable y la llamada a la corrección escalar dentro del bucle.

Listado 19: Correccion de una componente GNSS con la estimacion vigente.

```
float H[FUSION_DIM_ERROR] = {0};
H[componente] = 1.0f;

float prediccion;

if (componente < 3U)
{
    prediccion =
```

```

        siguiente.posicion_ned_m[componente];
    }
    else
    {
        prediccion =
            siguiente.velocidad_ned_m_s[componente - 3U];
    }

    float innovacion =
        medida->valor[componente] - prediccion;

    FusionResultadoCorreccion resultado =
        fusion_corregir_escalar(
            &siguiente,
            H,
            innovacion,
            medida->varianza[componente],
            umbral_nis
        );

```

Una componente rechazada por la comprobación de la NIS se omite y se continúa con las restantes. Si aparece un error de datos o de cálculo, descarta todas las modificaciones de la llamada. Si termina correctamente, guarda las correcciones aceptadas y devuelve su número, entre cero y seis.

Este procesamiento supone una matriz R_{GNSS} diagonal. Si existen correlaciones relevantes, habrá que usar la covarianza completa en una actualización conjunta, o transformar las medidas y su modelo para eliminar esas correlaciones antes de aplicar las correcciones escalares.

En conjunto, la corrección GNSS compara las componentes disponibles de posición y velocidad con las predichas por el estimador. Para cada componente, calcula la innovación y su incertidumbre, comprueba su compatibilidad y, si se acepta, aplica la corrección al estado nominal. Finalmente, actualiza la covarianza y la adapta al nuevo estado de referencia tras la inyección del error. Así, las medidas GNSS válidas permiten limitar la deriva acumulada durante la predicción inercial. Durante una pérdida de señal GNSS, el estimador continuará con la predicción inercial y la propagación de su incertidumbre, incorporando las demás referencias que estén disponibles y sean válidas.

2.7. Corrección mediante los barómetros

Los barómetros proporcionan medidas de presión que convertiremos en altura mediante un modelo atmosférico. Utilizaremos la rutina de corrección escalar desarrollada en el apartado del GNSS, adaptando la medida, su innovación y su incertidumbre.

Antes del lanzamiento estableceremos la presión de referencia de cada barómetro a partir de varias muestras válidas, con el cohete en reposo. La conversión deberá producir alturas relativas al punto de lanzamiento y utilizarse dentro del rango válido del sensor y del modelo atmosférico.

Para convertir la presión en altura reutilizaremos la función `AlturaDesdePresion`, empleada en el ejercicio 2. En esta implementación se aplicará por separado a cada barómetro, utilizando su presión de referencia. Las alturas obtenidas se incorporarán al estimador antes del filtrado adicional `FiltrarAltura` del programa anterior.

La entrada a la corrección será una altura h_{baro} , expresada en metros, junto con su varianza. La lectura deberá ser nueva, válida y corresponder al instante del estado que se corrige.

Como el eje vertical NED es positivo hacia abajo, el modelo de medida será:

$$z_{\text{baro}} = h_{\text{baro}}, \quad \hat{h}^- = -\hat{p}_D^-. \quad (53)$$

La innovación se obtiene restando la altura predicha a la medida:

$$r_{\text{baro}} = h_{\text{baro}} + \hat{p}_D^-. \quad (54)$$

Para el orden del estado de error definido anteriormente, la matriz de medida será:

$$H_{\text{baro}} = \begin{bmatrix} 0 & 0 & -1 & \underbrace{0 \cdots 0}_{12} \end{bmatrix}. \quad (55)$$

El único elemento no nulo corresponde al error de posición abajo. Su signo es negativo porque un aumento de esta coordenada supone una disminución de la altura.

La incertidumbre de la medida será:

$$R_{\text{baro}} = \sigma_h^2. \quad (56)$$

Esta varianza se expresa en metros cuadrados y debe considerar el ruido de presión, la incertidumbre de la referencia y de la conversión a altura, y los efectos de la instalación. Aumentar su valor reduce la influencia de la medida, pero no elimina los errores sistemáticos.

Cuando ambos barómetros proporcionen alturas nuevas y válidas, referidas al mismo origen y a un instante común, utilizaremos su promedio:

$$h_{\text{baro}} = \frac{h_{5C} + h_{5D}}{2}. \quad (57)$$

La conversión de presión a altura se realiza por separado para cada sensor, antes del promedio. Este promedio asigna el mismo peso a ambas alturas; no supone que sea la combinación óptima para cualquier distribución de incertidumbres.

La varianza de la altura combinada será:

$$\sigma_h^2 = \frac{1}{4} \left(\sigma_{5C}^2 + \sigma_{5D}^2 + 2 \text{Cov}(e_{5C}, e_{5D}) \right). \quad (58)$$

La covarianza representa los errores compartidos por ambas alturas. Su valor deberá justificarse mediante el modelo de errores y las pruebas; no puede obtenerse de una única pareja de lecturas.

La función `fusion_combinar_alturas` calcula el promedio y su varianza. Comprueba que los datos sean finitos, que las varianzas sean positivas y que la covarianza cumpla:

$$|\text{Cov}(e_{5C}, e_{5D})| \leq \sigma_{5C} \sigma_{5D}. \quad (59)$$

El Listado 20 recoge las operaciones principales. Se emplea precisión doble en el cálculo intermedio de la varianza para reducir problemas numéricos.

Listado 20: Combinación de las alturas y sus incertidumbres.

```
float altura =
    0.5f * altura_5C + 0.5f * altura_5D;

float varianza = (float)(
    0.25 * (
        (double)varianza_5C +
        (double)varianza_5D +
        2.0 * (double)covarianza
    )
);
```

Esta función prepara una única medida para el filtro. La corrección se aplicará una sola vez con la altura combinada y su varianza, sin incorporar de nuevo las mismas lecturas por separado.

La función `fusion_corregir_barometros` coordina la disponibilidad de los sensores. Si solo existe una lectura utilizable, emplea directamente su altura y su varianza. Si no existe ninguna, omite la corrección.

Cuando ambas lecturas están disponibles, comprueba su diferencia antes de utilizar el promedio:

$$d = h_{5C} - h_{5D}, \quad \sigma_d^2 = \sigma_{5C}^2 + \sigma_{5D}^2 - 2 \text{Cov}(e_{5C}, e_{5D}). \quad (60)$$

La pareja se descarta para esa actualización si:

$$d^2 > \gamma_d \sigma_d^2, \quad (61)$$

donde γ_d es un umbral configurable. La teoría de su cálculo se encuentra en el anexo A.

Una discrepancia excesiva no permite identificar por sí sola qué sensor falla. En esta implementación se omite la corrección barométrica de ese instante. La comprobación tampoco detecta errores comunes que afecten de forma similar a ambos sensores.

Una vez seleccionada o combinada la altura, la función `fusion_corregir_barometro` prepara la innovación y la matriz de medida definidas anteriormente. Después llama a `fusion_corregir_escalar`, utilizando la varianza correspondiente a esa altura.

El Listado 21 muestra esta adaptación. La misma función sirve para una lectura individual o para el promedio de los dos sensores.

Listado 21: Aplicación de la corrección barométrica.

```
float H[FUSION_DIM_ERROR] = {0};
H[2] = -1.0f;

float innovacion =
    altura_m + estado->posicion_ned_m[2];

return fusion_corregir_escalar(
    estado,
    H,
    innovacion,
    varianza_m2,
    umbral_nis
);
```

La rutina común evalúa la innovación normalizada y, si acepta la medida, realiza la actualización de la covarianza, la inyección de la corrección y el ajuste de la covarianza al nuevo estado de referencia. Si la lectura se rechaza o el cálculo falla, conserva el estado anterior.

La corrección utiliza una altura individual o el promedio de las dos lecturas compatibles. Si no dispone de una medida aceptable, el estimador continúa con la IMU y las demás referencias disponibles.

2.8. Corrección mediante el magnetómetro

El magnetómetro mide las tres componentes del campo magnético en los ejes del cohete. Utilizaremos esta medida como referencia para corregir la orientación que hemos calculado integrando las lecturas del giróscopo.

Necesitaremos una referencia del campo magnético terrestre en el sistema de navegación, denominada $\mathbf{B}^n$. Esta referencia puede obtenerse mediante un modelo geomagnético como el World Magnetic Model (WMM) [7], introduciendo la latitud, longitud, altitud y fecha del vuelo. El modelo proporciona las componentes del campo hacia el norte, este y abajo, con las que se construye el vector $\mathbf{B}^n$, expresado en las mismas unidades que las medidas del magnetómetro.

Para una implementación real, se calculará inicialmente esta referencia para el lugar de lanzamiento y se evaluará su variación a lo largo de la trayectoria prevista. Si dicha variación es relevante para la precisión buscada, la referencia deberá actualizarse durante el vuelo. La altitud introducida deberá ser compatible con la referencia requerida por el modelo; no se utilizará directamente la altura relativa al lanzamiento.

El vector obtenido representa tanto la intensidad como la dirección del campo terrestre. Por ello, no se supondrá que apunta exactamente hacia el norte ni que su componente vertical es nula.

Según la orientación estimada, el campo que debería medir el sensor será:

$$\hat{\mathbf{m}}^b = [C_b^n(\hat{\mathbf{q}}^-)]^T \mathbf{B}^n. \quad (62)$$

La matriz transpuesta realiza la transformación inversa: pasa del sistema de navegación a los ejes del cohete. La función `fusion_predecir_campo_magnetico` realiza esta operación. Reutiliza `fusion_matriz_rotacion` para obtener C_b^n y multiplica su transpuesta por $\mathbf{B}^n$. El resultado contiene las tres componentes que esperamos medir en los ejes del cohete, en las mismas unidades que la referencia. Esta función prepara la predicción, sin modificar el estado del estimador. El siguiente listado muestra el cálculo del campo predicho. Se recorren las columnas de C_b^n para multiplicar por su transpuesta, sin construir otra matriz.

Listado 22: Campo magnético predicho en los ejes del cuerpo.

```
float C[3][3];
fusion_matriz_rotacion(q, C);

float prediccion[3];

for (unsigned int i = 0U; i < 3U; ++i)
{
    prediccion[i] =
        C[0][i] * campo_ned[0]
        + C[1][i] * campo_ned[1]
        + C[2][i] * campo_ned[2];
}
```

Cada componente se obtiene proyectando el campo de referencia sobre uno de los ejes del cohete. Si el cuerpo está alineado con NED, la matriz es la identidad y las componentes del campo permanecen iguales.

Compararemos esta predicción con la medida calibrada del magnetómetro:

$$\mathbf{r}_{\text{mag}} = \mathbf{m}_{\text{medida}}^b - \hat{\mathbf{m}}^b. \quad (63)$$

Si ambas difieren, el filtro evaluará si esa diferencia justifica corregir la orientación, teniendo en cuenta la incertidumbre de la medida y de la predicción.

La matriz H_{mag} relaciona un pequeño error de orientación con el cambio que produciría en las tres componentes del campo medido. Para la convención de error angular que utilizamos:

$$H_{\text{mag}} = \begin{bmatrix} 0_{3 \times 3} & 0_{3 \times 3} & [\hat{\mathbf{m}}^b]_{\times} & 0_{3 \times 3} & 0_{3 \times 3} \end{bmatrix}, \quad (64)$$

donde el bloque correspondiente a la orientación es:

$$[\hat{\mathbf{m}}^b]_{\times} = \begin{bmatrix} 0 & -\hat{m}_z & \hat{m}_y \\ \hat{m}_z & 0 & -\hat{m}_x \\ -\hat{m}_y & \hat{m}_x & 0 \end{bmatrix}. \quad (65)$$

Cada fila corresponde a una componente del magnetómetro. Por ejemplo, la primera indica que el cambio esperado en la componente X será:

$$\delta m_x \simeq -\hat{m}_z \delta \theta_y + \hat{m}_y \delta \theta_z. \quad (66)$$

Así, el filtro puede relacionar la diferencia entre el campo medido y el predicho con un posible error de orientación.

Suponemos constante el campo terrestre en la zona de vuelo. La medida predicha depende directamente de la orientación y los demás bloques de H_{mag} son nulos. Aun así, la corrección puede modificar otras variables mediante las correlaciones de P .

Para construir el bloque angular utilizaremos `fusion_matriz_producto_vectorial`, pasándole las tres componentes del campo predicho $\hat{\mathbf{m}}^b$. La función `fusion_fila_medida_magnetica` prepara la fila de H_{mag} correspondiente a la componente X, Y o Z. Para ello construye el bloque angular a partir del campo predicho y selecciona la fila indicada por `componente`: 0 para X, 1 para Y y 2 para Z.

Listado 23: Construcción de una fila de la matriz de medida magnética.

```
float S[3][3];
fusion_matriz_producto_vectorial(campo_predicho, S);

float fila[FUSION_DIM_ERROR] = {0};

for (unsigned int i = 0U; i < 3U; ++i)
{
    fila[6U + i] = S[componente][i];
}
```

Las posiciones 6, 7 y 8 de `fila` contienen los coeficientes asociados a los tres errores de orientación de $\delta \mathbf{x}$. Esta fila permitirá utilizar la rutina de corrección escalar con una componente magnética.

Para asignar confianza utilizaremos una matriz R_{mag} . Si suponemos errores no correlacionados entre los ejes:

$$R_{\text{mag}} = \text{diag}(\sigma_{m_x}^2, \sigma_{m_y}^2, \sigma_{m_z}^2). \quad (67)$$

Antes de utilizar una lectura comprobaremos su validez, la ausencia de saturación y su coherencia con el campo esperado. Una diferencia importante en la intensidad puede indicar una perturbación:

$$\Delta B = \left| \|\mathbf{m}_{\text{medida}}^b\| - \|\mathbf{B}^n\| \right|. \quad (68)$$

El límite admitido se establecerá mediante la incertidumbre prevista y las pruebas. Esta comprobación no detecta todas las perturbaciones: también revisaremos la innovación, porque la dirección puede alterarse sin que cambie la intensidad. La función `fusion_comprobar_intensidad_magnetica` compara las normas del campo medido y del campo de referencia. No es necesario expresarlos en los mismos ejes para esta comprobación, porque una rotación conserva la norma. Ambos deben utilizar las mismas unidades.

Listado 24: Comprobación de la intensidad del campo magnético.

```
float intensidad_medida = hypotf(
    hypotf(campo_medido[0], campo_medido[1]),
    campo_medido[2]
);

float intensidad_referencia = hypotf(
    hypotf(campo_ned[0], campo_ned[1]),
    campo_ned[2]
);

float diferencia = fabsf(
    intensidad_medida - intensidad_referencia
);

return (diferencia <= limite_diferencia) ? 1 : 0;
```

El listado muestra el cálculo principal. La función completa también comprueba que las entradas sean válidas y descarta una medida de intensidad nula. El límite se recibe como parámetro, en las mismas unidades que el campo. Superar esta comprobación solo indica compatibilidad de intensidad. La saturación y la calidad de la lectura deberán verificarse antes, y la innovación se evaluará al realizar la corrección.

Si la diferencia de intensidad excede el límite, rechazaremos la lectura completa. Si resulta compatible, comprobaremos también la innovación antes de incorporarla. El giróscopo seguirá propagando la orientación, aunque podrá acumular error [4].

Una vez validada la lectura, la función `fusion_corregir_componente_magnetica` prepara la corrección de una componente. Calcula el campo esperado con la orientación vigente, construye la fila de medida y obtiene la innovación.

Listado 25: Preparación y corrección de una componente magnética.

```
fusion_predecir_campo_magnetico(
    siguiente.q_cuerpo_a_ned,
    campo_ned,
    campo_predicho
);

if (!fusion_fila_medida_magnetica(
    campo_predicho, componente, H))
{
    return FUSION_CORRECCION_ERROR;
}

float innovacion =
    medida - campo_predicho[componente];

FusionResultadoCorreccion resultado =
    fusion_corregir_escalar(
        &siguiente,
        H,
        innovacion,
        varianza,
        umbral_nis
    );
```

La rutina `escalar` realiza las operaciones de corrección ya desarrolladas para los sensores anterior-

res. La función `fusion_corregir_magnetometro` coordina la actualización completa. Primero comprueba la intensidad del campo. Si la lectura es compatible, procesa sus componentes X, Y y Z secuencialmente.

Listado 26: Procesamiento secuencial de las componentes magnéticas.

```
for (unsigned int i = 0U; i < 3U; ++i)
{
    FusionResultadoCorreccion resultado =
        fusion_corregir_componente_magnetica(
            &siguiente,
            campo_ned,
            i,
            campo_medido[i],
            varianza[i],
            umbral_nis
        );

    if (resultado == FUSION_CORRECCION_ERROR)
    {
        return -1;
    }

    if (resultado == FUSION_CORRECCION_APLICADA)
    {
        ++aplicadas;
    }
}
```

Cada componente utiliza el estado actualizado por las correcciones anteriores, porque una corrección anterior puede haber cambiado la orientación. Se recalculan el campo predicho y la fila de medida con la orientación vigente. Este tratamiento requiere errores de medida no correlacionados entre ejes.

Una componente rechazada por la NIS se omite y se continúa con la siguiente. El umbral se aplica individualmente: no constituye una comprobación conjunta de las tres componentes. Al tratarse de un modelo no lineal, el procesamiento secuencial puede depender del orden elegido.

La función trabaja sobre una copia del estado. Si aparece un error de datos o de cálculo, descarta los cambios de toda la llamada. En caso contrario, guarda las correcciones aceptadas y devuelve su número, entre cero y tres. La calibración, la comprobación de saturación y la sincronización temporal se realizan antes de invocar esta función.

Recordamos que un giro alrededor de la dirección del campo magnético no modifica la lectura ideal. Por ello seguirá siendo necesario combinarlo con el giróscopo y con otras referencias cuando estén disponibles.

La corrección magnética queda integrada en la misma rutina de actualización utilizada para los otros sensores. Si se descarta la lectura, no se aplica esa corrección y la orientación continúa propagándose con el giróscopo.

2.9. Integración de la predicción y las correcciones

Las funciones desarrolladas actúan sobre un mismo `FusionEstado`. Ahora las conectamos en un ciclo de ejecución: primero avanzamos la estimación con la IMU y después aplicamos las correcciones disponibles para ese instante.

La integración debe evitar que una lectura se utilice dos veces o que una medida antigua corri-

ja el estado actual. Para comprobar esta coordinación trabajaremos inicialmente con sensores sincronizados y sin retrasos de recepción.

2.9.1. Orden temporal de las operaciones y ciclo de ejecución

Las correcciones se programarán en instantes que coincidan con las muestras de la IMU. Esta simplificación permite comprobar la integración del estimador sin introducir todavía un mecanismo de compensación de latencias. En cada instante t_k , primero utilizaremos la muestra de la IMU para propagar el estado y su covarianza desde t_{k-1} hasta t_k . Después incorporaremos las medidas externas disponibles que correspondan a ese mismo instante.

Si coinciden varias medidas, mantendremos un orden de procesamiento fijo: GNSS, barómetros y magnetómetro. Cada actualización utilizará el estado y la covarianza resultantes de las operaciones anteriores. No será necesario disponer de todos los sensores para continuar: se aplicarán únicamente las correcciones cuyas medidas sean nuevas y válidas.

Cada lectura externa se procesará una sola vez. Una medida rechazada también se considerará procesada, evitando volver a evaluarla en los ciclos posteriores como si fuese nueva.

El estimador se inicializará una sola vez antes de comenzar el procesamiento de las muestras. A partir de ese momento, cada ciclo partirá del estado obtenido en el ciclo anterior. El instante inicial del estado, t_0 , servirá como referencia para calcular el primer intervalo de predicción. Para cada nueva muestra IMU, seguiremos esta secuencia:

1. Comprobar la validez de la muestra y calcular $\Delta t_k = t_k - t_{k-1}$. El intervalo deberá ser compatible con las condiciones de funcionamiento previstas.
2. Preparar las medidas de la IMU y ejecutar la predicción mediante `fusion_predecir`. Si la operación termina correctamente, el estado y su covarianza corresponderán al instante t_k .
3. Procesar las medidas nuevas y válidas del GNSS, los barómetros y el magnetómetro que correspondan a t_k , manteniendo el orden establecido.
4. Registrar el estado resultante y los resultados de las correcciones, y conservarlos como punto de partida del siguiente ciclo.

2.9.2. Estado y referencia temporal

Hasta ahora, `FusionEstado` almacena las variables estimadas y su covarianza, pero no indica a qué instante corresponden. Para coordinar la predicción y las correcciones, agrupamos ese estado con una marca temporal:

Listado 27: Estado del estimador y su instante asociado.

```
typedef struct
{
    FusionEstado estado;
    double tiempo_s;
} FusionEstimador;
```

El campo `estado` contiene la estimación del cohete, mientras que `tiempo_s` indica el instante al que corresponde. Todas las marcas temporales se expresarán en segundos y compartirán un mismo origen. Utilizamos `double` para conservar mayor precisión al restar instantes cercanos; las variables del filtro siguen utilizando `float`.

La función `fusion_inicializar_estimador` reutiliza la inicialización ya desarrollada y añade el instante inicial:

Listado 28: Inicialización del estado y de su tiempo.

```
FusionEstimador inicial = {0};

if (!fusion_inicializar(
    &inicial.estado, q_inicial, sigma_inicial))
{
    return 0;
}

inicial.tiempo_s = tiempo_inicial_s;

*estimador = inicial;
```

La expresión `&inicial.estado` entrega a la función existente la dirección del estado que debe inicializar. Si la operación tiene éxito, asignamos el tiempo y copiamos el conjunto al estimador de salida. La función completa comprueba previamente que el puntero de salida sea válido y que el instante inicial sea finito.

Durante la ejecución, el tiempo almacenado avanzará únicamente cuando se complete correctamente una predicción. Las correcciones modificarán el estado y su covarianza en ese mismo instante, sin avanzar el reloj del estimador.

2.9.3. Predicción hasta una nueva muestra IMU

La función `fusion_predecir_hasta` conecta la predicción existente con la referencia temporal del estimador. Recibe el instante de la nueva muestra y calcula el intervalo respecto al estado almacenado:

Listado 29: Cálculo y validación del intervalo de predicción.

```
double intervalo =
    tiempo_muestra_s - estimador->tiempo_s;

if (!isfinite(intervalo) ||
    intervalo <= 0.0 ||
    intervalo > (double)dt_max_s)
{
    return 0;
}
```

Un intervalo nulo corresponde a un instante ya alcanzado, mientras que uno negativo indica una muestra anterior al estado actual. Ambos se rechazan. También establecemos un intervalo máximo configurable para evitar propagar un periodo excesivo con una sola lectura. La implementación actual detecta y rechaza los intervalos excesivos, pero no soluciona la situación. Como el tiempo del estimador permanece sin cambios, las muestras posteriores también se rechazarán.

Una vez validado el intervalo y convertido a `float`, realizamos la predicción:

Listado 30: Predicción y actualización del instante del estado.

```
if (!fusion_predecir(
    &estimador->estado,
    aceleracion_g,
    giro_dps,
    dt,
```

```

        gravedad_m_s2,
        ruido))
{
    return 0;
}

estimador->tiempo_s = tiempo_muestra_s;

```

Si la predicción falla, se conservan el estado y su instante anterior. Si termina correctamente, ambos quedan referidos al instante de la muestra. Esta comprobación evita repetir la predicción con una misma marca temporal; el control de las lecturas externas se incorporará al gestionar las correcciones.

2.9.4. Control de las lecturas ya procesadas

El instante del estado no permite determinar por sí solo si una lectura externa ya se ha utilizado. Varias fuentes pueden corregir el mismo estado, pero una misma lectura no debe incorporarse repetidamente.

Ampliamos `FusionEstimador` con el último instante procesado de cada sensor y un indicador que señala si existe una lectura anterior. Los dos barómetros se controlan por separado, ya que uno puede estar disponible sin el otro.

Listado 31: Seguimiento de las lecturas externas.

```

double ultimo_gnss_s;
double ultimo_baro_s[2];
double ultimo_mag_s;

unsigned char gnss_procesado;
unsigned char baro_procesado[2];
unsigned char mag_procesado;

```

Antes de invocar una corrección, comprobaremos que la medida corresponda al instante del estado y sea posterior a la última lectura procesada de esa fuente:

Listado 32: Condiciones temporales para procesar una medida.

```

if (tiempo_medida_s != tiempo_estado_s)
{
    return 0;
}

if (ya_procesada)
{
    if (!isfinite(ultimo_tiempo_s) ||
        tiempo_medida_s <= ultimo_tiempo_s)
    {
        return 0;
    }
}

```

En esta implementación sincronizada, las medidas coincidentes compartirán exactamente la misma marca temporal. La comprobación no resuelve latencias ni diferencias entre relojes.

Una lectura evaluada y rechazada por los criterios de compatibilidad también se marcará como procesada. Así evitaremos volver a introducirla después de que otras correcciones hayan cambiado el estado.

2.9.5. Integración de las correcciones

La función `fusion_procesar_gnss` conecta la corrección GNSS existente con el seguimiento temporal del estimador. Primero comprueba que la lectura corresponda al instante del estado y no se haya procesado:

Listado 33: Control temporal previo a la corrección GNSS.

```
if (!fusion_medida_procesable(
    tiempo_medida_s,
    estimador->tiempo_s,
    estimador->ultimo_gnss_s,
    estimador->gnss_procesado))
{
    return 0;
}
```

Si cumple estas condiciones, se entrega la medida a la rutina de corrección. Esta conserva las comprobaciones de datos y de compatibilidad mediante la NIS desarrolladas anteriormente.

Listado 34: Corrección GNSS y registro de la lectura procesada.

```
int aplicadas = fusion_corregir_gnss(
    &estimador->estado,
    medida,
    umbral_nis
);

if (aplicadas < 0)
{
    return -1;
}

estimador->ultimo_gnss_s = tiempo_medida_s;
estimador->gnss_procesado = 1U;
```

La marca de lectura procesada se actualiza también cuando ninguna componente supera la comprobación de compatibilidad. Así, una lectura rechazada no vuelve a evaluarse en otra llamada.

El instante del estado permanece constante: la operación ajusta la estimación en ese instante. Si aparece un error de datos o de cálculo, la función devuelve `-1` y conserva tanto el estado como el registro temporal anterior.

En cuanto a los barómetros, su control temporal se realiza de forma independiente para cada uno. Copiamos las medidas recibidas y ajustamos sus indicadores de uso sin modificar los datos originales:

Listado 35: Selección temporal de las lecturas barométricas.

```
FusionMedidasBarometros seleccionadas = *medidas;

/* Dentro del bucle, para cada lectura disponible. */
seleccionadas.usar[i] =
    (unsigned char)fusion_medida_procesable(
        tiempo_medida_s[i],
        estimador->tiempo_s,
        estimador->ultimo_baro_s[i],
        estimador->baro_procesado[i]
    );
```

Solo las lecturas nuevas y correspondientes al instante del estado quedan seleccionadas. Si ambas cumplen las condiciones, la rutina existente comprueba su compatibilidad y combina las alturas. Si solo una las cumple, utiliza esa medida individualmente.

Listado 36: Corrección con las lecturas barométricas seleccionadas.

```
FusionResultadoCorreccion resultado =
    fusion_corregir_barometros(
        &estimador->estado,
        &seleccionadas,
        covarianza_m2,
        umbral_diferencia,
        umbral_nis
    );

if (resultado == FUSION_CORRECCION_ERROR)
{
    return resultado;
}
```

Si no aparece un error, registramos los instantes de las lecturas evaluadas, aunque la corrección se haya rechazado por discrepancia entre sensores o por la NIS:

Listado 37: Registro de las lecturas barométricas procesadas.

```
for (unsigned int i = 0U; i < 2U; ++i)
{
    if (seleccionadas.usar[i])
    {
        estimador->ultimo_baro_s[i] =
            tiempo_medida_s[i];
        estimador->baro_procesado[i] = 1U;
    }
}
```

La función `fusion_procesar_magnetometro` aplica el mismo control temporal utilizado para el GNSS. La lectura deberá corresponder al instante del estado y no haberse procesado anteriormente:

Listado 38: Control temporal de la lectura magnética.

```
if (!fusion_medida_procesable(
    tiempo_medida_s,
    estimador->tiempo_s,
    estimador->ultimo_mag_s,
    estimador->mag_procesado))
{
    return 0;
}
```

Si cumple estas condiciones, reutilizamos la función que comprueba la intensidad del campo y realiza las correcciones de sus tres componentes:

Listado 39: Corrección magnética y registro de la lectura.

```
int aplicadas = fusion_corregir_magnetometro(
    &estimador->estado,
    campo_medido,
    campo_ned,
    varianza,
    limite_intensidad,
```

```

    umbral_nis
);

if (aplicadas < 0)
{
    return -1;
}

estimador->ultimo_mag_s = tiempo_medida_s;
estimador->mag_procesado = 1U;

```

Una lectura rechazada por intensidad o por la NIS también queda registrada como procesada. Solo un error de datos o de cálculo impide actualizar este registro. La corrección no modifica el instante del estado.

2.9.6. Comprobación de la integración

Comprobamos la conexión de las funciones mediante un ciclo sobre un único `FusionEstimador`. Para esta comprobación de software utilizamos reposo ideal y ejes del cuerpo alineados con NED.

Primero propagamos el estado desde $t = 0$ hasta $t = 0,01$ s. Después procesamos, en ese orden, las medidas GNSS, barométricas y magnéticas correspondientes al mismo instante. Todas las llamadas reciben la dirección del mismo estimador:

Listado 40: Ejemplo de llamada integrada a la corrección GNSS.

```

int gnss_aplicadas_integracion = fusion_procesar_gnss(
    &prueba_integracion,
    &gnss_integracion,
    tiempo_ciclo,
    9.0f
);

```

En estas condiciones esperamos que se acepten las seis componentes GNSS, una actualización barométrica y las tres componentes magnéticas.

Finalmente repetimos las llamadas con las mismas marcas temporales. La predicción debe omitirse porque el estado ya ha alcanzado ese instante, y las lecturas externas deben omitirse porque ya figuran como procesadas. Esta comprobación verifica la coordinación de las llamadas.

2.10. Simulación y evaluación del estimador

2.10.1. Condiciones iniciales del estimador

Inicializaremos el estimador antes del despegue, con el cohete en reposo sobre la plataforma de lanzamiento. Como el origen del sistema NED se sitúa en el punto de lanzamiento, tomaremos:

$$\hat{\mathbf{p}}_0 = \mathbf{0}, \quad \hat{\mathbf{v}}_0 = \mathbf{0}.$$

La posición inicial nula responde a la elección del origen, mientras que la velocidad inicial nula representa el reposo respecto al suelo.

La orientación inicial deberá corresponder a la disposición del cohete en la plataforma. Para la trayectoria de referencia 1 adoptaremos un montaje vertical, con x_b hacia arriba, y_b hacia el este y z_b hacia el norte. Esta elección concreta permite definir el cuaternión inicial y comprobar la coherencia de las transformaciones entre cuerpo y NED.

La matriz que transforma vectores del cuerpo a NED será:

$$\mathbf{C}_{b,0}^n = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ -1 & 0 & 0 \end{bmatrix}.$$

Sus columnas expresan las direcciones de x_b , y_b y z_b en coordenadas NED.

Con el convenio de cuaterniones utilizado, ordenados como $[q_0, q_1, q_2, q_3]^T$, un giro de ángulo θ alrededor del eje unitario $\mathbf{u}$ se representa mediante $[\cos(\theta/2), \mathbf{u}^T \sin(\theta/2)]^T$. En este caso, $\theta = \pi/2$ y $\mathbf{u} = [0, 1, 0]^T$, por lo que:

$$\hat{q}_0 = \begin{bmatrix} \cos(\theta/2) \\ u_x \sin(\theta/2) \\ u_y \sin(\theta/2) \\ u_z \sin(\theta/2) \end{bmatrix} = \begin{bmatrix} \cos((\pi/2)/2) \\ 0 \cdot \sin((\pi/2)/2) \\ 1 \cdot \sin((\pi/2)/2) \\ 0 \cdot \sin((\pi/2)/2) \end{bmatrix} = \begin{bmatrix} \cos(\pi/4) \\ 0 \\ \sin(\pi/4) \\ 0 \end{bmatrix} = \begin{bmatrix} 1/\sqrt{2} \\ 0 \\ 1/\sqrt{2} \\ 0 \end{bmatrix}. \quad (69)$$

Esta orientación inicial corresponde a la primera trayectoria. En la segunda utilizaremos la disposición inclinada definida en el apartado 2.12. En la simulación incorporaremos ruido y sesgos, que serán conocidos por el simulador pero no por el estimador. Sus estimaciones iniciales serán nulas, y les asignaremos una cierta incertidumbre inicial.

2.10.2. Generación de las medidas simuladas

El simulador mantendrá una trayectoria de referencia, formada por la posición, la velocidad y la orientación reales del movimiento simulado. Esta información se utilizará para generar las lecturas de los sensores y evaluar posteriormente los errores del estimador, pero no se entregará directamente al filtro.

Para generar la lectura del acelerómetro, partiremos de la aceleración de referencia en NED, $\mathbf{a}_{\text{ref}}^n$. Restaremos la gravedad y transformaremos el resultado a los ejes del cuerpo:

$$\mathbf{f}_{\text{ref}}^b = \left(\mathbf{C}_{b,\text{ref}}^n \right)^T (\mathbf{a}_{\text{ref}}^n - \mathbf{g}^n),$$

donde $\mathbf{C}_{b,\text{ref}}^n$ transforma vectores del cuerpo a NED. La lectura simulada será:

$$\mathbf{f}_m^b = \mathbf{f}_{\text{ref}}^b + \mathbf{b}_{a,\text{real}} + \mathbf{n}_a.$$

Para utilizar la interfaz de preparación de la IMU, expresaremos esta lectura en unidades de g , dividiéndola entre g_0 .

De forma análoga, la lectura del giróscopo será:

$$\boldsymbol{\omega}_m^b = \boldsymbol{\omega}_{\text{ref}}^b + \mathbf{b}_{g,\text{real}} + \mathbf{n}_g.$$

Esta expresión utiliza radianes por segundo; antes de entregarla a la interfaz de la IMU, la convertiremos a grados por segundo.

En esta simulación, los sesgos reales de la IMU serán constantes y el ruido añadido tendrá media nula. El filtro comenzará con sus propias estimaciones de los sesgos y tratará de ajustarlas mediante las correcciones y las relaciones recogidas en la covarianza.

2.11. Trayectoria de referencia 1: ascenso vertical

La primera trayectoria representa un lanzamiento vertical, desde el reposo hasta alcanzar las proximidades del apogeo, manteniendo constante la orientación del cohete. Este escenario permite comprobar el seguimiento de la altura y de la velocidad vertical, así como la integración de las medidas de los distintos sensores en presencia de ruido y sesgos.

Este perfil es una trayectoria sintética de comprobación, no una predicción de las prestaciones del cohete. Consideraremos dos segundos de reposo, cinco segundos de ascenso propulsado y un ascenso balístico posterior. Mantendremos constante la gravedad y despreciaremos la resistencia aerodinámica.

La sencillez de esta trayectoria facilita la interpretación de los resultados, pero limita la evaluación del estimador: no introduce movimiento horizontal ni cambios de orientación durante el vuelo. Por ello, posteriormente utilizaremos una segunda trayectoria tridimensional con orientación variable, destinada a comprobar la estimación del movimiento en los tres ejes y el seguimiento de la orientación.

Ambos escenarios utilizarán el mismo filtro y el mismo procedimiento de generación de medidas simuladas. Cambiarán las referencias de movimiento y orientación a partir de las cuales se obtienen las lecturas de los sensores.

Definimos h positiva hacia arriba y $v_h = \dot{h}$. Durante el reposo:

$$h = 0, \quad v_h = 0, \quad \ddot{h} = 0.$$

Durante la fase propulsada, con $\tau = t - 2$:

$$h = \frac{1}{2}a_p\tau^2, \quad v_h = a_p\tau, \quad \ddot{h} = a_p, \quad a_p = 20 \text{ m/s}^2.$$

Al finalizar esta fase, en $t = 7$ s, la altura y la velocidad son:

$$h_b = 250 \text{ m}, \quad v_b = 100 \text{ m/s}.$$

Durante el ascenso balístico, con $s = t - 7$:

$$h = h_b + v_b s - \frac{1}{2}g_0 s^2, \quad v_h = v_b - g_0 s, \quad \ddot{h} = -g_0.$$

El apogeo teórico de la trayectoria se produce en:

$$t_{\text{apogeo}} = 7 + \frac{v_b}{g_0} \approx 17,20 \text{ s},$$

$$h_{\text{apogeo}} = h_b + \frac{v_b^2}{2g_0} \approx 759,86 \text{ m}.$$

Expresaremos esta trayectoria en NED mediante:

$$\mathbf{p}_{\text{ref}}^n = \begin{bmatrix} 0 \\ 0 \\ -h \end{bmatrix}, \quad \mathbf{v}_{\text{ref}}^n = \begin{bmatrix} 0 \\ 0 \\ -v_h \end{bmatrix}, \quad \mathbf{a}_{\text{ref}}^n = \begin{bmatrix} 0 \\ 0 \\ -\ddot{h} \end{bmatrix}.$$

Mantendremos constante la orientación vertical definida anteriormente, por lo que la velocidad angular real será nula. Las lecturas del giroscopo contendrán, aun así, los sesgos y el ruido añadidos. Este caso permitiría evaluar la deriva de orientación, pero no valida el comportamiento ante giros.

2.11.1. Frecuencia, ruido y sesgos de los sensores

Las frecuencias elegidas para la simulación se muestran en la Tabla 1. Son parámetros de esta evaluación, no especificaciones de los sensores seleccionados.

Tabla 1: Frecuencias de muestreo utilizadas en la simulación.

Fuente	Frecuencia	Periodo
IMU	100 Hz	0,01 s
Cada barómetro	100 Hz	0,01 s
GNSS	10 Hz	0,10 s
Magnetómetro	20 Hz	0,05 s

El bucle principal avanzará con la IMU. Los barómetros aportarán lecturas en cada ciclo, el GNSS cada diez ciclos y el magnetómetro cada cinco. Los sensores que coincidan utilizarán la misma marca temporal. En los ciclos restantes no se volverán a entregar sus lecturas anteriores.

Representaremos los errores de los sensores mediante sesgos y ruido aleatorio, siguiendo un planteamiento habitual en la simulación de sensores inerciales y GNSS [3]. Utilizaremos los valores sintéticos de desviación típica por muestra recogidos en la Tabla 2. Son valores elegidos para esta comprobación y no extraídos de las especificaciones de los sensores. El ruido tendrá media nula y será independiente entre muestras, sensores y componentes.

Tabla 2: Desviaciones típicas del ruido por muestra utilizadas en la simulación.

Magnitud	Desviación típica
Fuerza específica, cada eje	0,10 m/s ²
Velocidad angular, cada eje	0,002 rad/s
Posición GNSS, N/E/D	(2; 2; 3) m
Velocidad GNSS, N/E/D	(0,2; 0,2; 0,3) m/s
Altura de cada barómetro	2 m
Campo magnético, cada eje	0,5 μ T

Los sesgos reales, constantes y expresados en los ejes del cuerpo, serán:

$$\mathbf{b}_{a,\text{real}} = \begin{bmatrix} 0,08 \\ -0,05 \\ 0,03 \end{bmatrix} \text{ m/s}^2, \quad \mathbf{b}_{g,\text{real}} = \begin{bmatrix} 0,001 \\ -0,0007 \\ 0,0005 \end{bmatrix} \text{ rad/s}.$$

Estos sesgos solo se utilizarán para generar las medidas y evaluar los resultados. El estimador comenzará con sesgos estimados nulos e incertidumbres iniciales no nulas. Los valores elegidos no representan una caracterización experimental de la IMU.

Las intensidades continuas del ruido de la IMU se ajustarán al ruido discreto generado mediante:

$$q_a = \sigma_a^2 \Delta t, \quad q_g = \sigma_g^2 \Delta t.$$

Elegimos estos valores para que el filtro asigne a cada paso la misma varianza de ruido en velocidad y ángulo que produce la lectura simulada al integrarse durante Δt .

Como los sesgos reales serán constantes, tomaremos $q_{b_a} = q_{b_g} = 0$. Esto no impide corregir sus estimaciones: su incertidumbre inicial se representa en P , y las medidas externas pueden aportar información sobre ellos.

2.11.2. Generación del ruido gaussiano

El ruido representa las fluctuaciones de una lectura que aparecen incluso cuando la magnitud real permanece constante. A diferencia del sesgo, que mantendremos fijo durante esta simulación, añadiremos un valor de ruido distinto a cada muestra y componente.

Utilizaremos ruido gaussiano de media cero. Primero generaremos un número adimensional z con distribución normal de media cero y varianza uno:

$$z \sim \mathcal{N}(0, 1).$$

Después lo multiplicaremos por la desviación típica del sensor:

$$n = \sigma z.$$

Así obtenemos ruido con las unidades y la dispersión deseadas.

La función `rand()` proporciona enteros pseudoaleatorios entre cero y `RAND_MAX`. Los transformamos en valores estrictamente comprendidos entre cero y uno mediante:

Listado 41: Generación de un valor uniforme entre cero y uno.

```
return ((double)rand() + 1.0)
      / ((double)RAND_MAX + 2.0);
```

La conversión a `double` permite realizar la división con decimales. Los términos añadidos al numerador y al denominador excluyen los extremos cero y uno.

Estos valores uniformes todavía no tienen una distribución gaussiana. Para obtenerla utilizaremos la transformación de Box-Muller:

$$z = \sqrt{-2 \ln u_1} \cos(2\pi u_2),$$

donde u_1 y u_2 son dos valores uniformes independientes en $(0, 1)$. Excluir el cero evita calcular $\ln(0)$.

Listado 42: Generación de ruido gaussiano normalizado.

```
double u1 = simulacion_uniforme();
double u2 = simulacion_uniforme();

double z = sqrt(-2.0 * log(u1))
          * cos(dos_pi * u2);

return (float)z;
```

Cada lectura se construye sumando el valor ideal, el sesgo real y el ruido de esa muestra. Por ejemplo, para el eje X del acelerómetro:

Listado 43: Composición de una lectura simulada del acelerómetro.

```
float lectura_acc_x =
    fuerza_ideal_x
    + SESGO_ACC_REAL[0]
    + SIGMA_ACC * simulacion_normal();
```

Si `simulacion_normal()` devuelve $-1,2$ y $\sigma_a = 0,10 \text{ m/s}^2$, el ruido añadido será $-0,12 \text{ m/s}^2$. Con fuerza ideal nula y sesgo de $0,08 \text{ m/s}^2$, la lectura resultará:

$$f_{m,x} = 0 + 0,08 - 0,12 = -0,04 \text{ m/s}^2.$$

Cada sensor y componente utilizará una llamada nueva al generador. No reutilizaremos el mismo valor de ruido en varios ejes, pues introduciríamos una dependencia no contemplada en el modelo.

2.11.3. Implementación de la trayectoria de referencia 1

Implementamos primero la trayectoria vertical definida anteriormente. Para este caso basta con conocer, en cada instante, la altura, la velocidad vertical y la aceleración vertical: las componentes horizontales del movimiento son nulas y la orientación permanece constante.

Agrupamos estas tres magnitudes en la estructura `SimulacionReferencia`. La altura se mide respecto al punto de lanzamiento y el sentido positivo es hacia arriba. Esta representación simplificada corresponde a la primera trayectoria; la segunda requerirá describir el movimiento en los tres ejes y la evolución de la orientación.

Listado 44: Magnitudes de la trayectoria vertical de referencia.

```
typedef struct
{
    double altura_m;
    double velocidad_vertical_m_s;
    double aceleracion_vertical_m_s2;
} SimulacionReferencia;
```

La función `simulacion_referencia` recibe el tiempo transcurrido desde el inicio de la simulación y devuelve estas tres magnitudes. Durante los dos primeros segundos, el cohete permanece en reposo sobre la plataforma. Entre los instantes 2 y 7 s, aplicamos una aceleración neta constante hacia arriba de 20 m/s^2 .

En el tramo propulsado, la variable `tau` representa el tiempo transcurrido desde el despegue. Como el cohete parte del reposo, calculamos la altura y la velocidad mediante las expresiones del movimiento uniformemente acelerado.

Listado 45: Cálculo de la referencia durante el ascenso propulsado.

```
const double aceleracion = 20.0;
const double tau = tiempo_s - 2.0;

referencia.altura_m =
    0.5 * aceleracion * tau * tau;

referencia.velocidad_vertical_m_s =
    aceleracion * tau;

referencia.aceleracion_vertical_m_s2 =
    aceleracion;
```

Al finalizar este tramo, el cohete alcanza una altura de 250 m y una velocidad ascendente de 100 m/s. Desde ese instante modelamos un ascenso balístico sin resistencia aerodinámica, con aceleración vertical igual a $-g$. La simulación se limitará al ascenso hasta el apogeo.

Para expresar la referencia en el sistema NED, tendremos en cuenta que su eje D apunta hacia abajo. Por tanto, si h , v_h y a_h son los valores devueltos por la función:

$$\mathbf{p}_{\text{ref}}^n = \begin{bmatrix} 0 \\ 0 \\ -h \end{bmatrix}, \quad \mathbf{v}_{\text{ref}}^n = \begin{bmatrix} 0 \\ 0 \\ -v_h \end{bmatrix}, \quad \mathbf{a}_{\text{ref}}^n = \begin{bmatrix} 0 \\ 0 \\ -a_h \end{bmatrix}. \quad (70)$$

En la segunda trayectoria ampliaremos la referencia para incluir los vectores de posición, velocidad y aceleración en NED, junto con el cuaternión de orientación y la velocidad angular expresada en los ejes del cuerpo. Estas magnitudes permitirán generar medidas coherentes con

un movimiento tridimensional y una orientación variable, utilizando los mismos modelos de medida y de ruido.

En ambos casos, la referencia pertenece exclusivamente al simulador y se utiliza para generar las medidas y evaluar los errores. Durante la ejecución, el filtro recibirá únicamente las lecturas simuladas y sus incertidumbres, sin acceder al estado de referencia para realizar la predicción o las correcciones.

2.11.4. Generación de las medidas de la IMU

A partir de la trayectoria de referencia, generamos las lecturas que recibirá el estimador desde el acelerómetro y el giróscopo. Para ello calculamos primero sus valores ideales y después incorporamos los sesgos reales y el ruido.

En la trayectoria de referencia 1 mantenemos una orientación constante, con el eje longitudinal X_b del cohete dirigido hacia arriba. La fuerza específica es la diferencia entre la aceleración del vehículo y la aceleración gravitatoria, expresada en los ejes del cuerpo. Como la aceleración vertical a_h es positiva hacia arriba, obtenemos:

$$\mathbf{f}_{\text{ideal}}^b = \begin{bmatrix} a_h + g \\ 0 \\ 0 \end{bmatrix}. \quad (71)$$

En reposo sobre la plataforma, el acelerómetro ideal mide g en el eje X_b . Durante el ascenso propulsado mide $20 + g$, mientras que en el tramo balístico mide cero, ya que el cohete se encuentra en caída libre. Esto último también sucede durante la parte ascendente del vuelo balístico.

Por otra parte, al mantener constante la orientación, la velocidad angular de referencia es nula en los tres ejes. Las lecturas del giróscopo contendrán, por tanto, únicamente el sesgo real y el ruido simulado.

Para cada eje sumamos estos errores a la lectura ideal. El siguiente fragmento pertenece al bucle que recorre las tres componentes de ambos sensores.

Listado 46: Incorporación de los sesgos y del ruido a las medidas de la IMU.

```
const float fuerza_medida =
    fuerza_ideal[i]
    + SESGO_ACC_REAL[i]
    + SIGMA_ACC * simulacion_normal();

const float giro_medido =
    SESGO_GIRO_REAL[i]
    + SIGMA_GIRO * simulacion_normal();

aceleracion_g[i] = fuerza_medida / G_SIM;

giro_dps[i] = giro_medido * radianes_a_grados;
```

Los sesgos reales permanecen constantes durante la simulación, mientras que generamos una nueva muestra de ruido para cada sensor, eje e instante de muestreo. Las lecturas se entregan en múltiplos de g para el acelerómetro y en grados por segundo para el giróscopo, de acuerdo con la interfaz de entrada del estimador.

El estimador convertirá estas medidas a unidades del SI y descontará sus propios sesgos estimados. Los sesgos reales utilizados para generar las lecturas pertenecen únicamente al simulador y no se proporcionan al algoritmo de fusión.

2.11.5. Generación de las medidas del GNSS

El GNSS proporciona medidas de posición y velocidad que permiten corregir la deriva acumulada durante la predicción mediante la IMU. En esta simulación generamos directamente estas medidas en el sistema NED y respecto al mismo origen que utiliza el estimador.

En la trayectoria de referencia 1, al ser vertical, las componentes norte y este de la posición y de la velocidad son nulas. Las componentes verticales cumplen $p_D = -h$ y $v_D = -v_h$, dado que el eje D apunta hacia abajo.

A cada componente de referencia añadimos una muestra de ruido gaussiano de media cero, escalada mediante la desviación típica correspondiente. Aunque el movimiento horizontal sea nulo, las lecturas horizontales también presentarán ruido.

La estructura `FusionMedidaGnss` almacena primero las tres componentes de posición y después las tres de velocidad:

Listado 47: Generación de las componentes de posición y velocidad del GNSS.

```
medida.valor[i] =  
    posicion_ned[i]  
    + SIGMA_GNSS_POS[i] * simulacion_normal();  
  
medida.valor[i + 3U] =  
    velocidad_ned[i]  
    + SIGMA_GNSS_VEL[i] * simulacion_normal();
```

Además de las lecturas, proporcionamos sus varianzas, que permiten ponderarlas durante la corrección:

Listado 48: Asignación de las varianzas y disponibilidad de las medidas GNSS.

```
medida.varianza[i] =  
    SIGMA_GNSS_POS[i] * SIGMA_GNSS_POS[i];  
  
medida.varianza[i + 3U] =  
    SIGMA_GNSS_VEL[i] * SIGMA_GNSS_VEL[i];  
  
medida.usar[i] = 1U;  
medida.usar[i + 3U] = 1U;
```

Suponemos errores independientes entre las seis componentes y entre instantes de adquisición. En este escenario no añadimos un sesgo persistente al GNSS y mantenemos disponibles todas sus componentes, aunque las comprobaciones de innovación del estimador podrán rechazar medidas.

2.11.6. Generación de las medidas de los barómetros

Los dos barómetros proporcionan una medida de altura respecto al punto de lanzamiento. En esta simulación generamos directamente la altura resultante de la conversión de presión, que es la magnitud utilizada por la función de corrección.

Ambos sensores parten de la misma altura de referencia, pero añadimos una muestra de ruido independiente a cada uno:

$$h_1 = h_{\text{ref}} + \sigma_{\text{baro}} z_1, \quad h_2 = h_{\text{ref}} + \sigma_{\text{baro}} z_2, \quad (72)$$

donde z_1 y z_2 siguen una distribución normal de media cero y varianza uno. En este escenario no introducimos sesgos persistentes ni errores comunes a los dos barómetros.

La función `simulacion_generar_barometros` recorre ambos sensores y almacena sus lecturas, sus varianzas y sus indicadores de disponibilidad.

Listado 49: Generación de las alturas medidas por los dos barómetros.

```
for (unsigned int i = 0U; i < 2U; ++i)
{
    medidas.altura_m[i] =
        (float)referencia->altura_m
        + SIGMA_BARO * simulacion_normal();

    medidas.varianza_m2[i] =
        SIGMA_BARO * SIGMA_BARO;

    medidas.usar[i] = 1U;
}
```

Al generar sus errores de forma independiente, la covarianza entre ambos sensores es nula. Este valor se proporcionará a la función de corrección al procesar las lecturas. Cuando ambas medidas superan las comprobaciones y se combinan mediante su media, la varianza resultante es:

$$\sigma_{\text{media}}^2 = \frac{\sigma_1^2 + \sigma_2^2}{4} = 2 \text{ m}^2. \quad (73)$$

Las alturas se mantienen positivas hacia arriba. La relación con el sistema NED, $h = -p_D$, ya está incorporada en el modelo de medida de la corrección.

Generaremos una lectura de cada barómetro a 100 Hz, coincidiendo con cada paso de la IMU. Las dos lecturas compartirán el instante de adquisición, aunque sus muestras de ruido serán independientes.

2.11.7. Generación de las medidas del magnetómetro

El magnetómetro mide las componentes del campo magnético en los ejes del cuerpo. Para generar sus lecturas, partimos del campo de referencia definido en NED y lo transformamos mediante la orientación real del cohete:

$$\mathbf{B}_{\text{ideal}}^b = \left(\mathbf{C}_{b,\text{ref}}^n \right)^T \mathbf{B}^n. \quad (74)$$

Para ambas simulaciones adoptamos el campo sintético

$$\mathbf{B}^n = \begin{bmatrix} 20 & 0 & 40 \end{bmatrix}^T \mu\text{T}.$$

Este vector permite comprobar las transformaciones y las correcciones magnéticas; no procede de una evaluación del WMM para un emplazamiento real. En la trayectoria de referencia 1, el eje X_b apunta hacia arriba, Y_b hacia el este y Z_b hacia el norte. Por tanto, la transformación se reduce a:

$$\mathbf{B}_{\text{ideal}}^b = \begin{bmatrix} -B_D \\ B_E \\ B_N \end{bmatrix}. \quad (75)$$

Listado 50: Campo magnético ideal expresado en los ejes del cohete.

```
const float campo_ideal[3] = {
    -CAMPO_NED_SIM[2],
    CAMPO_NED_SIM[1],
    CAMPO_NED_SIM[0]
};
```

A cada componente añadimos ruido gaussiano independiente, con una desviación típica de $0.5 \mu\text{T}$. Suponemos que el magnetómetro está calibrado y no introducimos sesgos ni perturbaciones magnéticas adicionales.

Listado 51: Generación de las medidas y varianzas del magnetómetro.

```
for (unsigned int i = 0U; i < 3U; ++i)
{
    campo_medido[i] =
        campo_ideal[i]
        + SIGMA_MAG * simulacion_normal();

    varianza[i] =
        SIGMA_MAG * SIGMA_MAG;
}
```

La generación utiliza la orientación de referencia del simulador. Durante la corrección, el estimador calculará el campo esperado utilizando su propia orientación estimada. La diferencia entre ambos vectores permitirá corregir los errores de orientación observables mediante esta medida.

Generaremos las lecturas a 20 Hz, es decir, una medida del magnetómetro por cada cinco pasos de la IMU.

2.11.8. Inicialización del estimador para la simulación

Una vez implementada la generación de las medidas, configuramos el estado inicial del estimador y su incertidumbre. El cohete parte del reposo en el origen del sistema NED, con el eje longitudinal dirigido hacia arriba.

Utilizamos el cuaternión inicial:

$$\hat{\mathbf{q}}_0 = \begin{bmatrix} 1/\sqrt{2} & 0 & 1/\sqrt{2} & 0 \end{bmatrix}^T, \quad (76)$$

que representa la orientación definida para la trayectoria de referencia.

La posición, la velocidad y la orientación estimadas coinciden inicialmente con sus valores de referencia. Sin embargo, los sesgos estimados del acelerómetro y del giróscopo se inicializan a cero, mientras que las lecturas simuladas incluyen los sesgos reales no nulos definidos anteriormente.

Asignamos una incertidumbre inicial a cada componente del estado de error mediante el siguiente vector de desviaciones típicas:

Listado 52: Desviaciones típicas iniciales del estado de error.

```
const float sigma_inicial[FUSION_DIM_ERROR] = {
    2.0f,  2.0f,  2.0f,
    0.1f,  0.1f,  0.1f,
    0.05f, 0.05f, 0.05f,
    0.2f,  0.2f,  0.2f,
    0.01f, 0.01f, 0.01f
};
```

Los cinco grupos corresponden, respectivamente, a los errores de posición, velocidad, orientación y sesgos del acelerómetro y del giróscopo. Sus unidades son metros, metros por segundo, radianes, metros por segundo al cuadrado y radianes por segundo.

Estos valores representan la incertidumbre que asignamos al estado inicial; no son perturbaciones que añadimos a sus componentes. La función de inicialización construye una matriz de covarianza

diagonal con sus cuadrados:

$$P_{ii}(0) = \sigma_i^2, \quad P_{ij}(0) = 0 \quad \text{si } i \neq j. \quad (77)$$

Configuramos también las intensidades de ruido de proceso del acelerómetro y del giróscopo a partir de las desviaciones típicas por muestra y del intervalo de la IMU.

Listado 53: Configuración del ruido de proceso para la simulación.

```
ruido->q_acc =
    SIGMA_ACC * SIGMA_ACC * (float)DT_SIM;

ruido->q_giro =
    SIGMA_GIRO * SIGMA_GIRO * (float)DT_SIM;

ruido->q_sesgo_acc = 0.0f;
ruido->q_sesgo_giro = 0.0f;
```

Igualamos la varianza del incremento directo de velocidad o ángulo: $\sigma^2 \Delta t^2 = q \Delta t$, de donde $q = \sigma^2 \Delta t$. Este ajuste no iguala por sí solo todos los términos de la covarianza discreta, que dependen de la propagación.

Como los sesgos reales permanecen constantes, no añadimos ruido de proceso a su evolución. Esto no fija sus estimaciones: su incertidumbre inicial y las relaciones entre errores permiten que las correcciones actualicen los sesgos estimados.

2.11.9. Programación de las correcciones

Las correcciones se ejecutan después de la predicción de la IMU, respetando la frecuencia de adquisición de cada sensor. Con un paso de simulación de 0.01 s, procesamos los dos barómetros en cada iteración, el GNSS cada diez iteraciones y el magnetómetro cada cinco.

Listado 54: Procesamiento del GNSS cada diez pasos de la IMU.

```
if (paso % 10U == 0U)
{
    FusionMedidaGnss medida =
        simulacion_generar_gnss(referencia);

    int resultado = fusion_procesar_gnss(
        estimador,
        &medida,
        tiempo_s,
        umbral_nis);

    if (resultado < 0)
    {
        return 0;
    }
}
```

Todas las medidas externas utilizadas para corregir en una iteración corresponden al instante alcanzado por la predicción. La trayectoria de referencia utilizada para generarlas se evalúa también en ese instante. Suponemos, por tanto, sensores sincronizados y ausencia de retrasos de comunicación.

Cuando coinciden varias medidas, aplicamos primero la corrección del GNSS, después la de los barómetros y finalmente la del magnetómetro. Cada corrección utiliza el estado y la covarianza resultantes de la operación anterior, sin avanzar el tiempo del estimador.

Para las correcciones escalares utilizamos un umbral de NIS igual a 9. En los barómetros fijamos también en 9 el umbral de la NIS entre sensores y utilizamos una covarianza entre sus errores igual a cero. Para el magnetómetro admitimos una diferencia máxima de intensidad respecto al campo de referencia de $5 \mu\text{T}$.

2.11.10. Bucle principal de la simulación

El bucle principal genera las medidas simuladas y llama a las funciones del estimador. Antes de comenzar, inicializamos el generador pseudoaleatorio con `srand(1U)` y configuramos el estado inicial y el ruido de proceso.

Cada iteración avanza la estimación 0,01 s. Calculamos el instante correspondiente multiplicando el número de paso por ese intervalo, para evitar acumular los redondeos de sumarlo repetidamente.

Para avanzar desde t_{k-1} hasta t_k , necesitamos una lectura de la IMU representativa del movimiento durante ese intervalo. La generamos evaluando la referencia en su punto medio:

$$t_{\text{medio}} = \frac{t_{k-1} + t_k}{2}.$$

Por ejemplo, para predecir desde 0,01 hasta 0,02 s, generamos la lectura de la IMU con la referencia de 0,015 s.

Una vez completada la predicción, el estado corresponde a t_k . Generamos entonces las medidas externas disponibles con la referencia de ese instante y las utilizamos para corregir la estimación.

Listado 55: Generación de la IMU para el intervalo de predicción.

```
const double tiempo_medio_s =
    0.5 * (estimador.tiempo_s + tiempo_s);

SimulacionReferencia referencia_imu =
    simulacion_referencia(tiempo_medio_s);

simulacion_generar_imu(
    &referencia_imu,
    aceleracion_g,
    giro_dps);
```

Con estas lecturas ejecutamos la predicción hasta t_k . A continuación, evaluamos la trayectoria de referencia en ese instante final y generamos las medidas de corrección que correspondan según las frecuencias de los sensores.

Listado 56: Correcciones en el instante alcanzado por la predicción.

```
SimulacionReferencia referencia =
    simulacion_referencia(tiempo_s);

if (!simulacion_aplicar_correcciones(
    &estimador,
    &referencia,
    paso))
{
    fprintf(
        stderr,
        "Error de correccion en t = %.3f s.\n",
        tiempo_s);
}
```

```

    return EXIT_FAILURE;
}

```

La primera simulación termina en la última muestra anterior al apogeo, en $t = 17,19$ s. Por ello, la velocidad vertical de referencia todavía es ligeramente positiva. Comparamos los valores estimados y los de referencia en ese mismo instante.

2.11.11. Registro de los resultados

Para evaluar la evolución del estimador, almacenamos los resultados en un archivo CSV. Registramos el estado inicial y, posteriormente, una fila al finalizar cada paso, una vez completadas la predicción y las correcciones disponibles.

Cada fila contiene el instante, la altura y la velocidad vertical de referencia, sus estimaciones, los errores correspondientes y las seis componentes de los sesgos estimados del acelerómetro y del giróscopo.

Para presentar los resultados utilizamos magnitudes verticales positivas hacia arriba:

$$\hat{h} = -\hat{p}_D, \quad \hat{v}_h = -\hat{v}_D. \quad (78)$$

Definimos los errores de evaluación como el valor de referencia menos la estimación, siguiendo la convención de los errores aditivos del filtro:

$$e_h = h_{\text{ref}} - \hat{h}, \quad e_{v_h} = v_{h,\text{ref}} - \hat{v}_h. \quad (79)$$

Listado 57: Cálculo de los errores verticales para el registro.

```

const double altura_estimada =
    -(double)estado->posicion_ned_m[2];

const double velocidad_estimada =
    -(double)estado->velocidad_ned_m_s[2];

const double error_altura =
    referencia->altura_m - altura_estimada;

const double error_velocidad =
    referencia->velocidad_vertical_m_s
    - velocidad_estimada;

```

Los sesgos estimados se registran en los ejes del cuerpo, en metros por segundo al cuadrado para el acelerómetro y en radianes por segundo para el giróscopo. Su evolución se comparará con los valores reales constantes utilizados al generar las medidas.

La función `simulacion_guardar_resultado` escribe una fila y devuelve un indicador de éxito. Si se produce un error de escritura, detenemos la simulación e informamos del fallo. Al finalizar cerramos el archivo, comprobando también el resultado de esta operación.

Con este registro queda preparada la evaluación de la primera trayectoria. A continuación definiremos un segundo escenario con movimiento tridimensional y orientación variable, utilizando el mismo estimador. Ampliaremos el registro para evaluar también las componentes horizontales, el error de orientación y la incertidumbre, y analizaremos conjuntamente los resultados de ambas simulaciones.

2.12. Trayectoria de referencia 2: movimiento tridimensional y orientación variable

Conservamos la simulación anterior y añadimos un ascenso tridimensional hasta un apogeo próximo a 10 km, inspirado en el cohete ISPIDA [8]. El cohete parte del reposo, asciende con empuje y continúa sin propulsión hasta el apogeo. Calcularemos su movimiento considerando la gravedad, el empuje dirigido según el eje longitudinal y la resistencia aerodinámica, con masa y densidad atmosférica variables.

La novedad principal será la orientación: introduciremos una inclinación progresiva y un giro alrededor del eje longitudinal. Fijaremos cómo cambia la orientación para generar un movimiento conocido con el que probar el estimador. No calcularemos ese giro a partir de los momentos aerodinámicos y de la inercia del cohete.

Definiremos una velocidad angular de referencia e integraremos la ecuación cinemática del cuaternión mediante Runge-Kutta de cuarto orden. La lectura del giróscopo será esa velocidad angular más el sesgo y el ruido. Para generar el acelerómetro y el magnetómetro, transformaremos la fuerza específica y el campo magnético desde NED a los ejes del cuerpo usando la orientación de referencia.

El estimador recibirá únicamente las medidas simuladas, manteniendo las frecuencias y el procedimiento anteriores. El cuaternión de referencia se utilizará para generar las medidas y evaluar el error de orientación. Durante los dos primeros segundos imponemos reposo sobre la plataforma, cuya reacción compensa el peso. Una vez iniciado el vuelo libre, la aceleración de referencia se calculará mediante:

$$\mathbf{a}_{\text{ref}}^n = \frac{T}{m} C_{b,\text{ref}}^n \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} + \mathbf{g}^n - \frac{\rho C_D S}{2m} \|\mathbf{v}_{\text{rel}}^n\| \mathbf{v}_{\text{rel}}^n, \quad (80)$$

donde T es el empuje, m la masa, ρ la densidad atmosférica, C_D el coeficiente de resistencia, S el área frontal y $\mathbf{v}_{\text{rel}}^n$ la velocidad respecto al aire. Tras el apagado del motor, $T = 0$. Integraremos esta aceleración para obtener la velocidad y la posición.

La velocidad angular prescrita $\boldsymbol{\omega}_{\text{ref}}^b$ determinará la evolución del cuaternión y, con él, la matriz $C_{b,\text{ref}}^n$. Las medidas simuladas serán:

$$\mathbf{f}_m^b = \left(C_{b,\text{ref}}^n\right)^T (\mathbf{a}_{\text{ref}}^n - \mathbf{g}^n) + \mathbf{b}_{a,\text{real}} + \mathbf{n}_a, \quad (81)$$

$$\boldsymbol{\omega}_m^b = \boldsymbol{\omega}_{\text{ref}}^b + \mathbf{b}_{g,\text{real}} + \mathbf{n}_g, \quad (82)$$

$$\mathbf{B}_m^b = \left(C_{b,\text{ref}}^n\right)^T \mathbf{B}^n + \mathbf{n}_B. \quad (83)$$

Así, las lecturas serán coherentes con el movimiento y la orientación del cohete. No proporcionaremos una medida directa del cuaternión: el estimador reconstruirá la orientación mediante la predicción con el giróscopo y las correcciones disponibles.

Tomaremos los parámetros de la Tabla 3. Aproximaremos el empuje por su valor medio durante la combustión y la disminución de masa por una relación lineal.

La masa inicial, la masa seca, el diámetro, la duración de combustión y el impulso total proceden de ISPIDA [8]. Como simplificación, identificaremos la masa final con la masa seca publicada y utilizaremos un coeficiente de resistencia constante. La altura máxima será un resultado de la simulación, sin imponer que alcance exactamente 10 km.

La orientación inicial tendrá una inclinación de 2° respecto a la vertical, hacia un azimut de 30° medido desde el norte hacia el este. Durante el vuelo aumentaremos la inclinación a razón de $\kappa = 0,2^\circ/\text{s}$ e introduciremos un giro longitudinal de $\Omega = 10^\circ/\text{s}$. Estos valores definen nuestro escenario de prueba y no proceden de mediciones de ISPIDA.

Tabla 3: Parámetros del escenario tridimensional.

Parámetro	Valor
Masa inicial	47 kg
Masa final adoptada	29,6 kg
Diámetro	0,152 m
Duración de la combustión	9,5 s
Impulso total	32,1 kN s
Empuje medio	3379 N
Coefficiente de resistencia adoptado	0,5

Si τ es el tiempo desde el despegue, el ángulo de giro longitudinal será $\varphi(\tau) = \Omega\tau$. Para mantener el aumento de inclinación en el mismo plano mientras gira el cuerpo, prescribiremos:

$$\boldsymbol{\omega}_{\text{ref}}^b(\tau) = \begin{bmatrix} \Omega \\ -\kappa \cos \varphi(\tau) \\ \kappa \sin \varphi(\tau) \end{bmatrix}. \quad (84)$$

En el código expresaremos los ángulos en radianes y las velocidades angulares en rad/s.

Integraremos esta velocidad angular para obtener el cuaternión de referencia. Así, el escenario combinará desplazamiento horizontal, inclinación variable y giro longitudinal, permitiendo evaluar la orientación y su influencia en la estimación de posición y velocidad.

La nueva simulación se implementa en `simulacion_3d.c`, utilizando las mismas funciones de `fusion.c` y `fusion.h`. La referencia se integra mediante Runge-Kutta de cuarto orden con pasos de 0,001 s, mientras que el estimador mantiene su intervalo de 0,01 s.

Adoptamos aire en reposo, gravedad constante y una densidad atmosférica aproximada por $\rho(h) = 1,225 \exp(-h/8500)$, con h en metros y ρ en kg/m^3 . El cohete permanece inicialmente en reposo durante dos segundos y la simulación termina en la última muestra anterior al apogeo.

La Figura 2 resume el perfil de referencia utilizado en esta simulación: la evolución de la masa, el empuje, la rapidez y la inclinación del eje longitudinal respecto a la vertical.

Durante la combustión, el empuje se mantiene constante y la masa disminuye. Tras el apagado, la masa permanece constante y la rapidez se reduce por la acción de la gravedad y de la resistencia aerodinámica. La inclinación aumenta durante el vuelo según la ley prescrita.

El archivo `resultados_simulacion_3d.csv` recoge la referencia, las estimaciones, los errores y las desviaciones típicas obtenidas de la diagonal de P , que utilizaremos para analizar los resultados.

Es importante mencionar que en este escenario mantenemos disponibles las referencias barométricas sintéticas durante todo el ascenso. No se simula el rango de presión de los LPS22DF ni la pérdida de sus medidas a gran altitud.

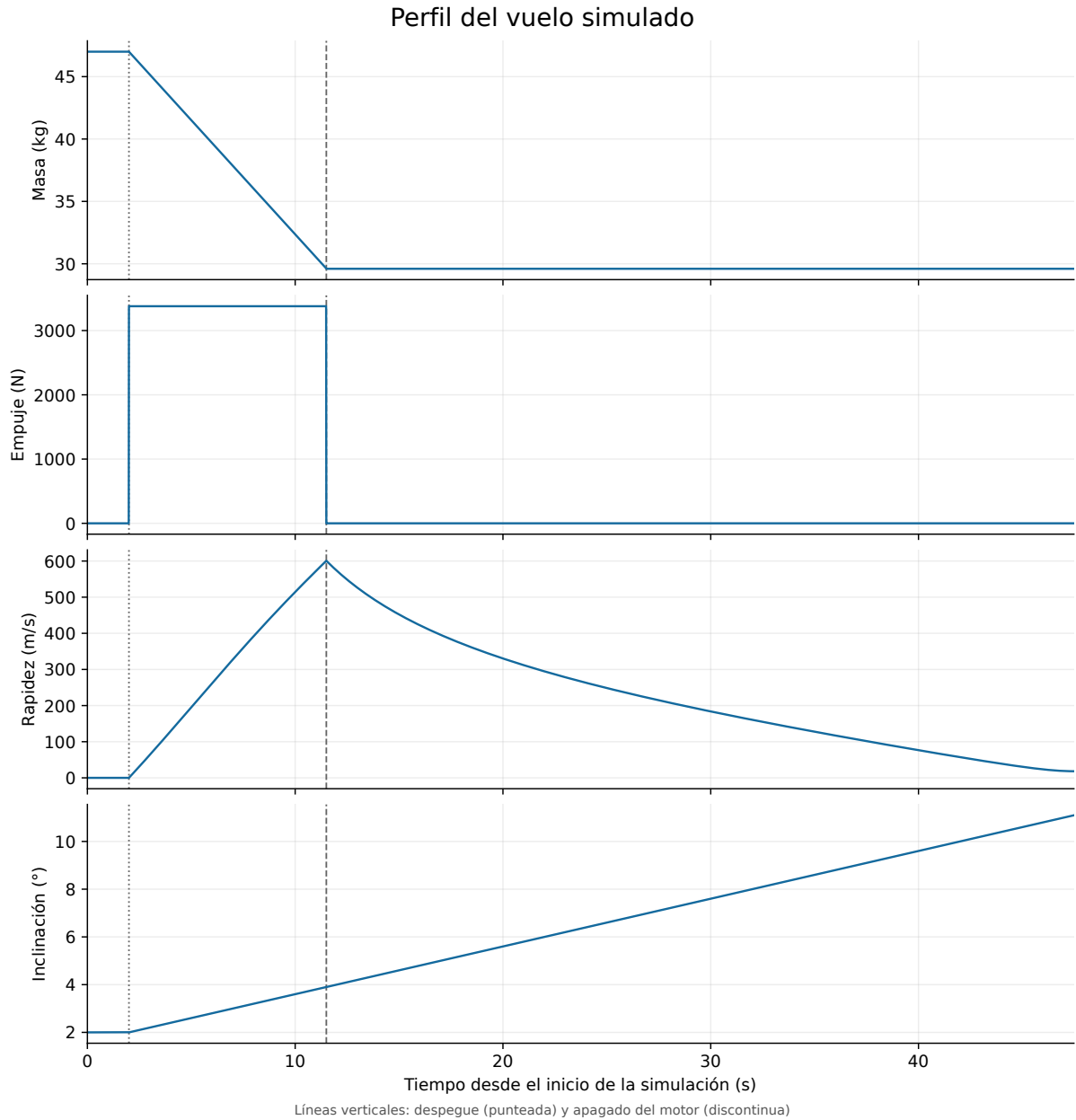


Figura 2: Perfil de referencia de la segunda trayectoria. Las líneas verticales señalan el despegue y el final de la propulsión.

2.13. Resultados e interpretación de las simulaciones

Analizaremos primero cada trayectoria y después compararemos los resultados. La primera comprueba el ascenso vertical con orientación constante; la segunda añade desplazamiento horizontal y giros. Compararemos las estimaciones y los sesgos con sus referencias, y en la segunda evaluaremos también los errores angulares y la incertidumbre.

2.13.1. Resultados de la trayectoria 1: ascenso vertical

Esta prueba permite examinar el seguimiento de la altura y la velocidad vertical, sus errores y los sesgos estimados. El registro contiene 1720 muestras, incluida la inicial, y termina en $t = 17,19$ s, antes del apogeo. Las líneas verticales señalan el comienzo del ascenso ($t = 2$ s) y el final de la propulsión ($t = 7$ s).

Seguimiento de la trayectoria vertical. La Figura 3 compara la altura y la velocidad vertical estimadas con sus referencias. Ambas magnitudes se representan positivas hacia arriba, por lo que se obtienen de las componentes NED mediante $h = -p_D$ y $v_h = -v_D$.

Trayectoria vertical: referencia y estimación

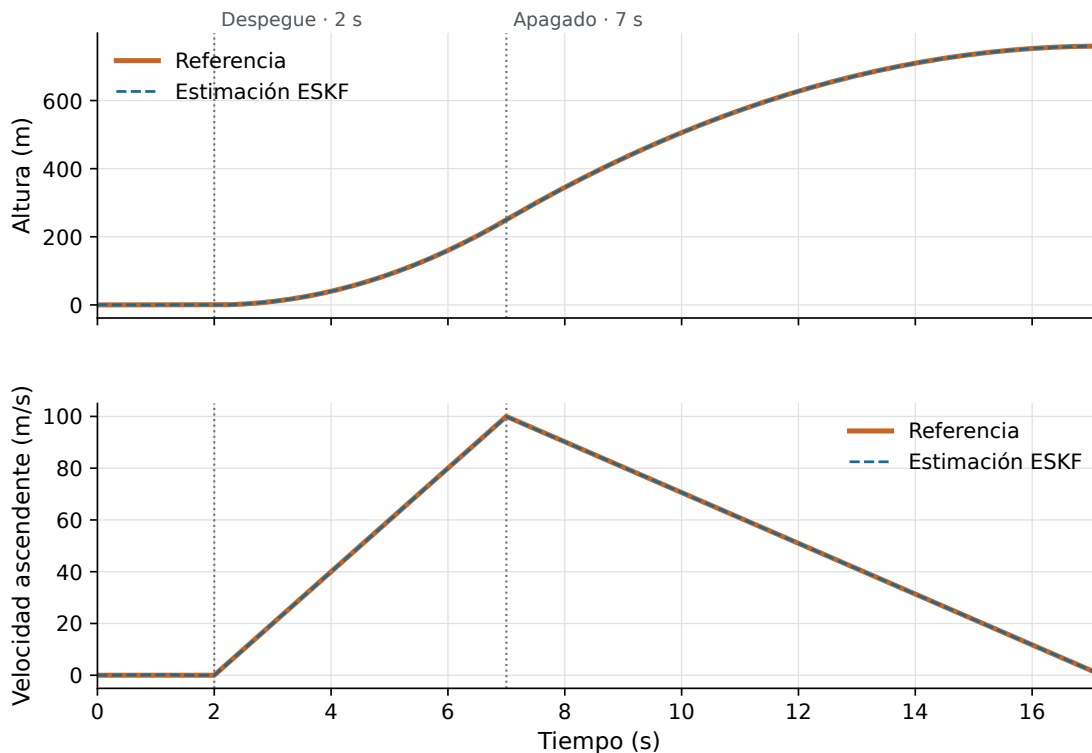


Figura 3: Altura y velocidad vertical de referencia y estimadas en la trayectoria 1. Las líneas verticales señalan el comienzo del ascenso y el final de la propulsión.

Durante los primeros dos segundos, el cohete permanece en reposo. Las pequeñas variaciones de la estimación alrededor de cero proceden de las medidas simuladas con ruido y de las correcciones aplicadas por el filtro.

Entre 2 y 7 s, la aceleración vertical de referencia es constante e igual a 20 m/s^2 . Por ello, la velocidad aumenta linealmente y la altura presenta una evolución cuadrática. Al finalizar esta fase, la referencia alcanza una altura de 250 m y una velocidad vertical de 100 m/s. Las estimaciones correspondientes son 249,730 m y 99,928 m/s.

Tras finalizar la propulsión, el modelo de esta trayectoria considera únicamente la acción de la gravedad. La velocidad vertical disminuye, pero continúa siendo positiva hasta las proximidades del apogeo. Por tanto, la altura sigue aumentando, aunque cada vez más lentamente.

En el último instante registrado, la altura de referencia es 759,858 m y la estimada, 759,762 m. Las velocidades verticales son, respectivamente, 0,070 m/s y 0,047 m/s. El estimador reproduce así las distintas fases del movimiento y la aproximación al apogeo.

A esta escala, las curvas prácticamente se superponen. Para valorar las diferencias entre ellas, examinamos a continuación sus errores.

Errores de altura y velocidad vertical. En ambas simulaciones definimos los errores aditivos como la referencia menos la estimación:

$$e_h = h_{\text{ref}} - \hat{h}, \quad e_{v_h} = v_{h,\text{ref}} - \hat{v}_h. \quad (85)$$

Esta convención coincide con la utilizada para los errores aditivos del filtro. Un error positivo indica que la estimación queda por debajo de la referencia.

Como $h = -p_D$ y $v_h = -v_D$, los errores verticales cumplen $e_h = -\delta p_D$ y $e_{v_h} = -\delta v_D$. El cambio de signo se debe a que el eje Down es positivo hacia abajo, mientras que la altura y la velocidad ascendente son positivas hacia arriba.

La Figura 4 muestra la evolución temporal de ambas diferencias.

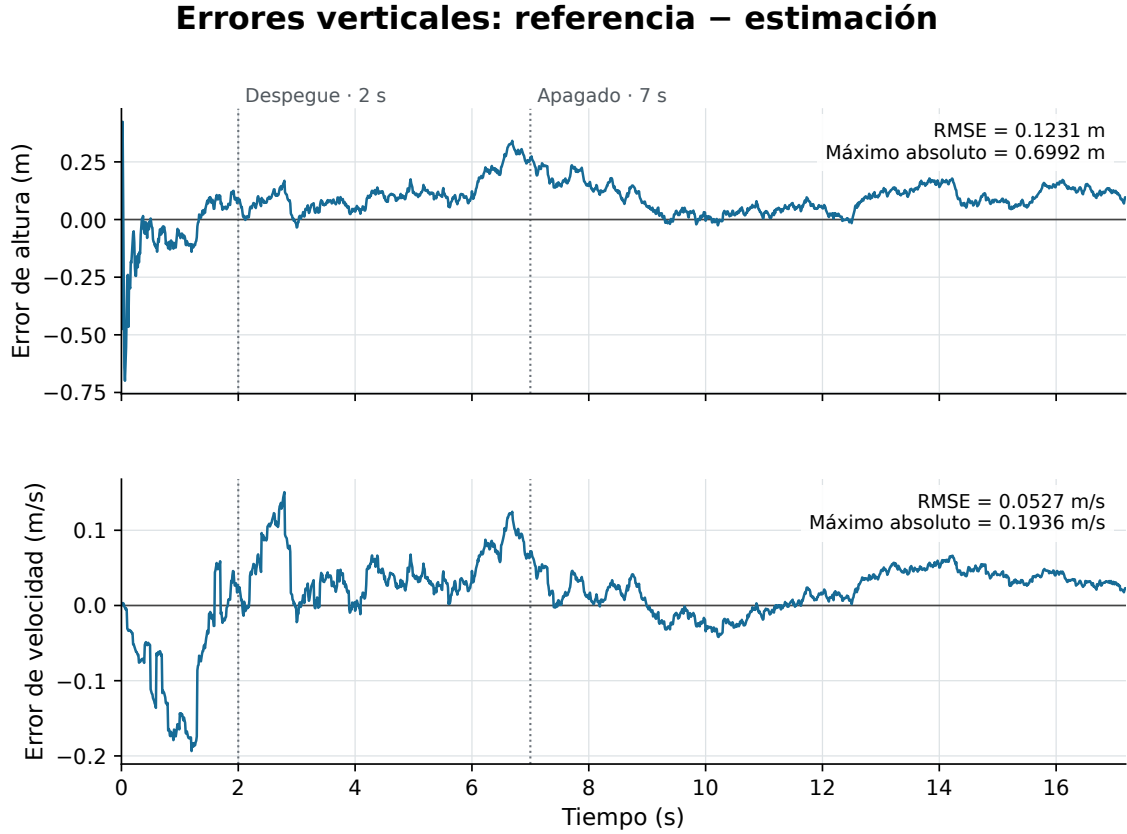


Figura 4: Errores de altura y velocidad vertical en la trayectoria 1, calculados como referencia menos estimación.

Los mayores errores absolutos aparecen durante el reposo inicial: el de altura alcanza 0,699 m en $t = 0,06$ s y el de velocidad vertical, 0,194 m/s en $t = 1,20$ s. Aunque la posición y la velocidad iniciales coinciden con sus referencias, las medidas posteriores contienen ruido y desplazan temporalmente la estimación.

Durante la propulsión, el error de altura es principalmente positivo y se aproxima a 0,3 m al apagarse el motor. Después se reduce, pero sigue siendo positivo la mayor parte del tiempo: la altura estimada suele quedar por debajo de la referencia. El error de velocidad oscila sin un crecimiento sostenido en este registro.

Para resumir el comportamiento utilizamos la raíz del error cuadrático medio, calculada sobre las 1720 muestras, incluida la inicial:

$$\text{RMSE}(e) = \sqrt{\frac{1}{M} \sum_{k=0}^{M-1} e_k^2}, \quad M = 1720. \quad (86)$$

La Tabla 4 recoge este indicador, el máximo error absoluto y el error en la última muestra.

Tabla 4: Errores de seguimiento vertical en la trayectoria 1.

Indicador	Altura (m)	Velocidad vertical (m/s)
RMSE	0,123	0,0527
Máximo error absoluto	0,699	0,194
Error final	0,0958	0,0229

Estos resultados muestran un seguimiento próximo a la referencia en esta ejecución. Los errores finales son reducidos, pero no describen por sí solos todo el vuelo: el RMSE y los máximos permiten incluir también las desviaciones que aparecen en instantes anteriores.

Estimación de los sesgos del acelerómetro. La Figura 5 compara las tres componentes del sesgo estimado con los valores constantes utilizados para generar las lecturas:

$$\mathbf{b}_{a,\text{ref}} = \begin{bmatrix} 0,08 \\ -0,05 \\ 0,03 \end{bmatrix} \text{ m/s}^2. \quad (87)$$

Las estimaciones parten de cero. El filtro debe ajustarlas a partir de las discrepancias entre las predicciones y las medidas externas. Estas medidas no proporcionan directamente los sesgos: su corrección es posible a través de las relaciones entre los errores que recoge la matriz de covarianza.

La estimación de la componente x_b aumenta desde cero, supera temporalmente su referencia y termina próxima a ella. La componente y_b presenta una evolución más irregular, con desviaciones transitorias que se reducen hacia el final. La componente z_b muestra un ajuste inicial pronunciado y posteriormente se aproxima a su valor de referencia.

En la última muestra se obtiene aproximadamente:

$$\hat{\mathbf{b}}_a(t_f) = \begin{bmatrix} 0,08028 \\ -0,05218 \\ 0,03352 \end{bmatrix} \text{ m/s}^2. \quad (88)$$

Los errores absolutos finales son, respectivamente, 0,00028, 0,00218 y 0,00352 m/s². Por tanto, las tres componentes terminan próximas a los valores introducidos en la simulación, aunque su ajuste no es monótono ni igualmente rápido.

Las variaciones de las curvas corresponden a cambios en las estimaciones. Los sesgos reales permanecen constantes durante toda esta simulación.

Estimación de los sesgos del acelerómetro

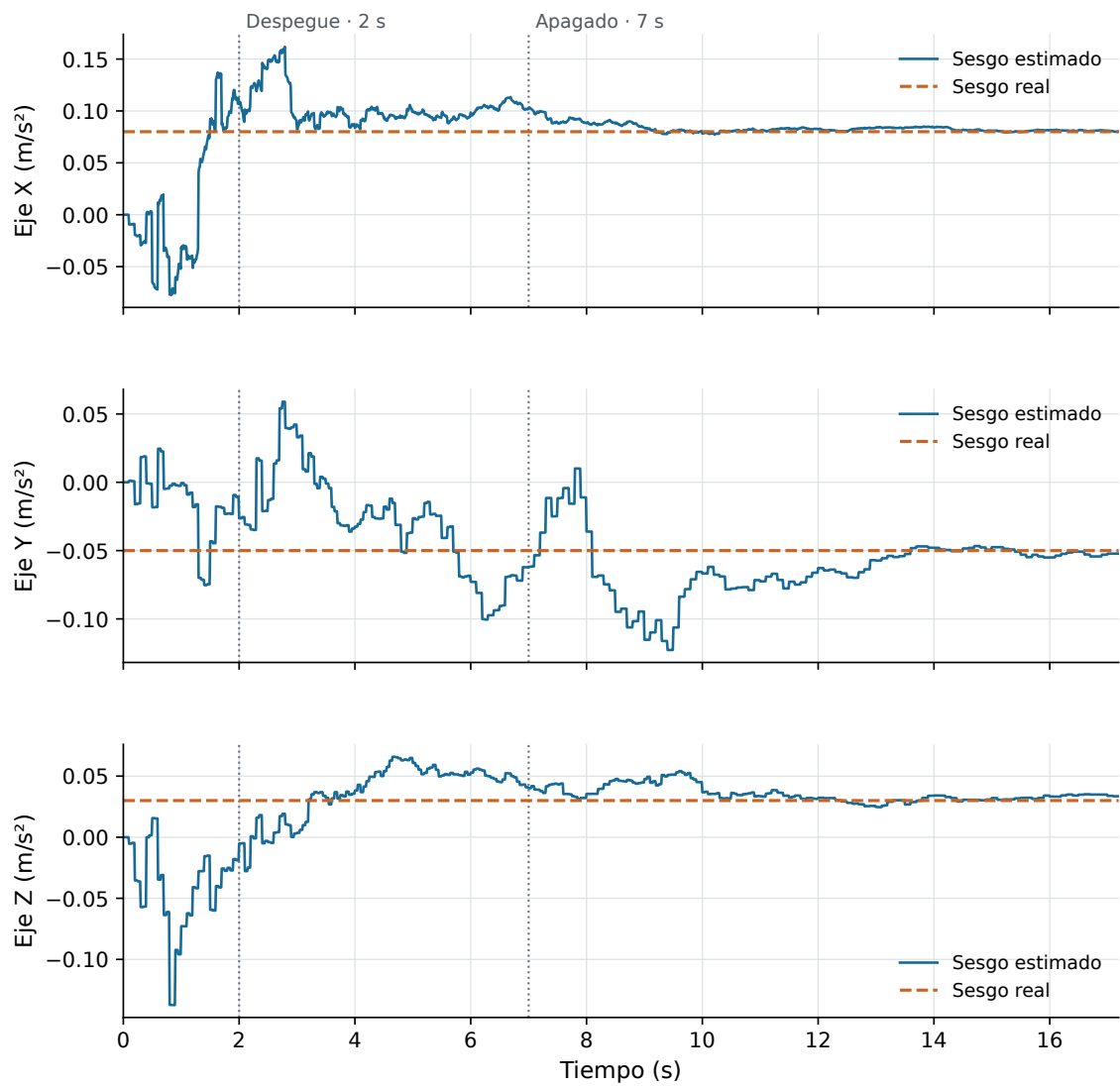


Figura 5: Sesgos estimados del acelerómetro y valores de referencia en la trayectoria 1, expresados en los ejes del cuerpo.

Estimación de los sesgos del gir6scopo. La Figura 6 presenta la misma comparaci6n para el gir6scopo, cuyos sesgos de referencia son:

$$\mathbf{b}_{g,\text{ref}} = \begin{bmatrix} 0,001 \\ -0,0007 \\ 0,0005 \end{bmatrix} \text{ rad/s.} \quad (89)$$

Como la orientaci6n de referencia permanece constante, la velocidad angular real es nula. Las lecturas del gir6scopo contienen, por tanto, los sesgos introducidos y el ruido de medida.

Estimaci6n de los sesgos del gir6scopo

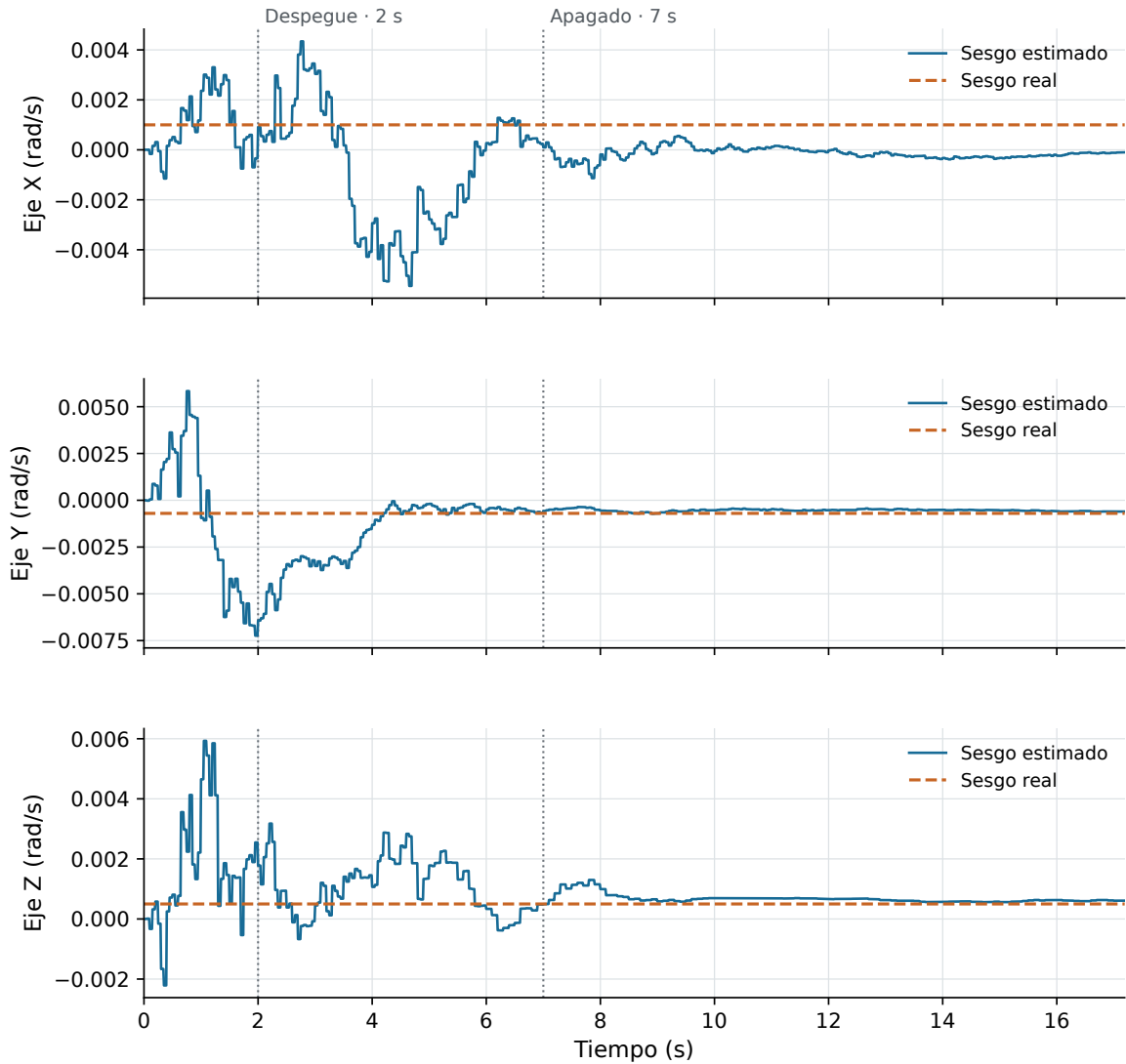


Figura 6: Sesgos estimados del gir6scopo y valores de referencia en la trayectoria 1, expresados en los ejes del cuerpo.

Las estimaciones presentan oscilaciones iniciales antes de estabilizarse. Sin embargo, estabilizarse no implica necesariamente alcanzar el valor real: las tres componentes muestran resultados diferentes.

El sesgo estimado de y_b termina en $-0,000603 \text{ rad/s}$, frente a una referencia de $-0,000700 \text{ rad/s}$.

Para z_b , el valor final es 0,000610 rad/s y la referencia, 0,000500 rad/s.

En cambio, la componente x_b termina alrededor de $-0,00010$ rad/s, alejada de su referencia de 0,001 rad/s. Su error final es aproximadamente 0,00110 rad/s, por lo que esta componente no se recupera correctamente en la ejecución analizada.

Este resultado muestra que un buen seguimiento de la altura y la velocidad vertical puede coexistir con una estimación imprecisa de alguno de los sesgos del gir6scopo. Con esta prueba no podemos determinar si la diferencia se debe a la falta de informaci3n sobre ese giro, al tiempo disponible para ajustar el sesgo o al comportamiento del filtro. Harían falta pruebas específicas para separar esas causas.

Balance de la primera trayectoria. En este escenario, el estimador combina la predicci3n mediante la IMU con las correcciones de los sensores externos y reproduce el ascenso vertical con un RMSE de 0,123 m en altura y 0,0527 m/s en velocidad vertical. Las estimaciones de los sesgos del aceler3metro terminan pr3ximas a sus referencias, mientras que los resultados del gir6scopo son menos uniformes.

La prueba permite comprobar el seguimiento vertical con medidas ruidosas y sesgos inicialmente desconocidos, pero mantiene una orientaci3n de referencia constante y parte de una orientaci3n inicial conocida. Por ello, no evalúa el seguimiento de giros durante el vuelo.

Además, el registro de esta primera simulaci3n no incluye los errores de orientaci3n ni la covarianza necesaria para contrastar los errores con la incertidumbre estimada. Estas comprobaciones se incorporan al análisis de la segunda trayectoria, que aña de desplazamiento horizontal y orientaci3n variable.

2.13.2. Resultados de la trayectoria 2: ascenso con orientaci3n variable

La segunda simulaci3n permite evaluar el estimador durante un ascenso con desplazamiento horizontal, inclinaci3n variable y giro longitudinal. Además del seguimiento de posici3n y velocidad, analizaremos la orientaci3n, los sesgos y la relaci3n entre los errores y la incertidumbre calculada.

El registro contiene 4749 muestras, incluida la inicial, y termina en $t = 47,48$ s, en la última muestra anterior al apogeo. La altura final de referencia es aproximadamente 10 700,8 m.

Trayectoria y desplazamiento horizontal. La Figura 7 muestra la trayectoria de referencia y la estimada, junto con su proyecci3n sobre el plano horizontal. La altura se representa positiva hacia arriba, mediante $h = -p_D$.

El cohete gana altura mientras se desplaza hacia el norte y el este. A la escala del vuelo, las curvas de referencia y estimaci3n prácticamente se superponen. La proyecci3n horizontal sigue aproximadamente una recta. Aunque la posici3n cambia en las tres componentes NED, el movimiento de referencia permanece en un plano vertical.

Este escenario amplía la evaluaci3n al seguimiento de las tres componentes de posici3n y de una orientaci3n variable, aunque no representa una trayectoria con cambios arbitrarios de rumbo.

Seguimiento de la posici3n y la velocidad. Las Figuras 8 y 9 comparan las componentes de posici3n y velocidad estimadas con sus referencias en el sistema NED.

Las posiciones norte y este aumentan conforme el cohete se aleja horizontalmente del punto

Trayectoria de referencia y estimación

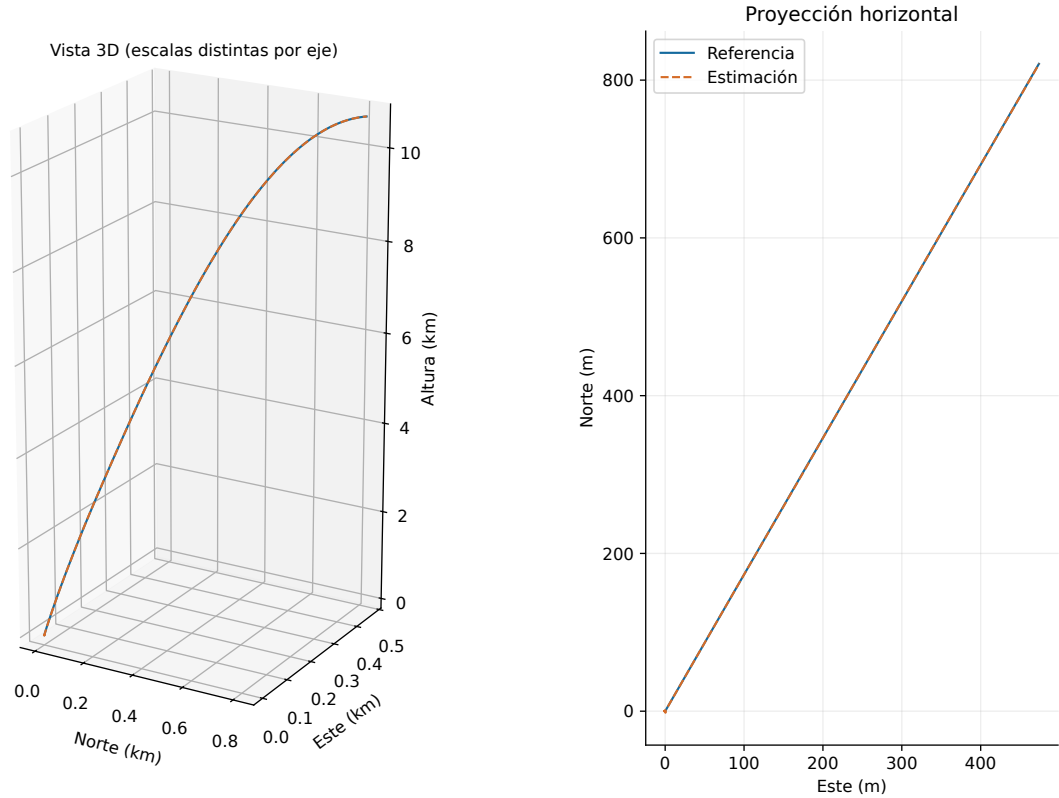


Figura 7: Trayectoria de referencia y estimada en el segundo escenario, junto con su proyección horizontal. La vista tridimensional utiliza escalas distintas en sus ejes.

de lanzamiento. La componente norte es mayor que la este, de acuerdo con el azimut de 30° utilizado para definir la dirección del ascenso.

La componente Down se hace progresivamente más negativa porque el cohete gana altura. El módulo de su pendiente disminuye al aproximarse al apogeo, cuando la velocidad vertical tiende a cero.

Durante el reposo inicial, las velocidades de referencia son nulas. A partir del despegue, en $t = 2$ s, aparecen componentes horizontales positivas y una componente Down negativa, correspondiente al movimiento ascendente.

La propulsión termina en $t = 11,5$ s. A partir de ese instante, la gravedad y la resistencia aerodinámica reducen la velocidad de ascenso. La componente Down se aproxima a cero desde valores negativos, mientras que las componentes horizontales disminuyen por efecto de la resistencia.

El seguimiento de las tres componentes permite comprobar la respuesta del estimador durante las distintas fases del vuelo. Podemos ver cómo las gráficas de la referencia y la estimación se superponen a escalas del vuelo.

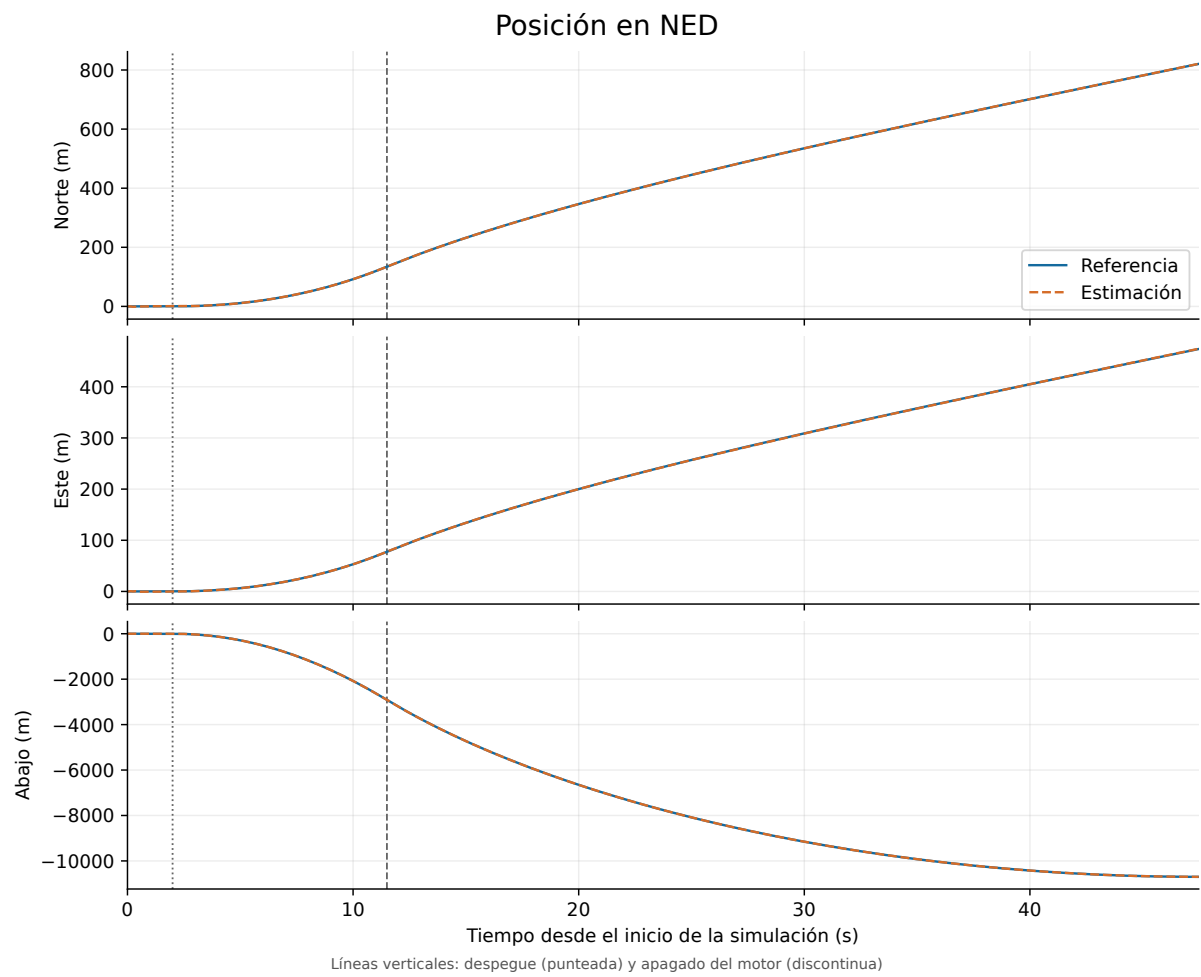


Figura 8: Posición de referencia y estimada en las componentes norte, este y abajo durante la segunda trayectoria.

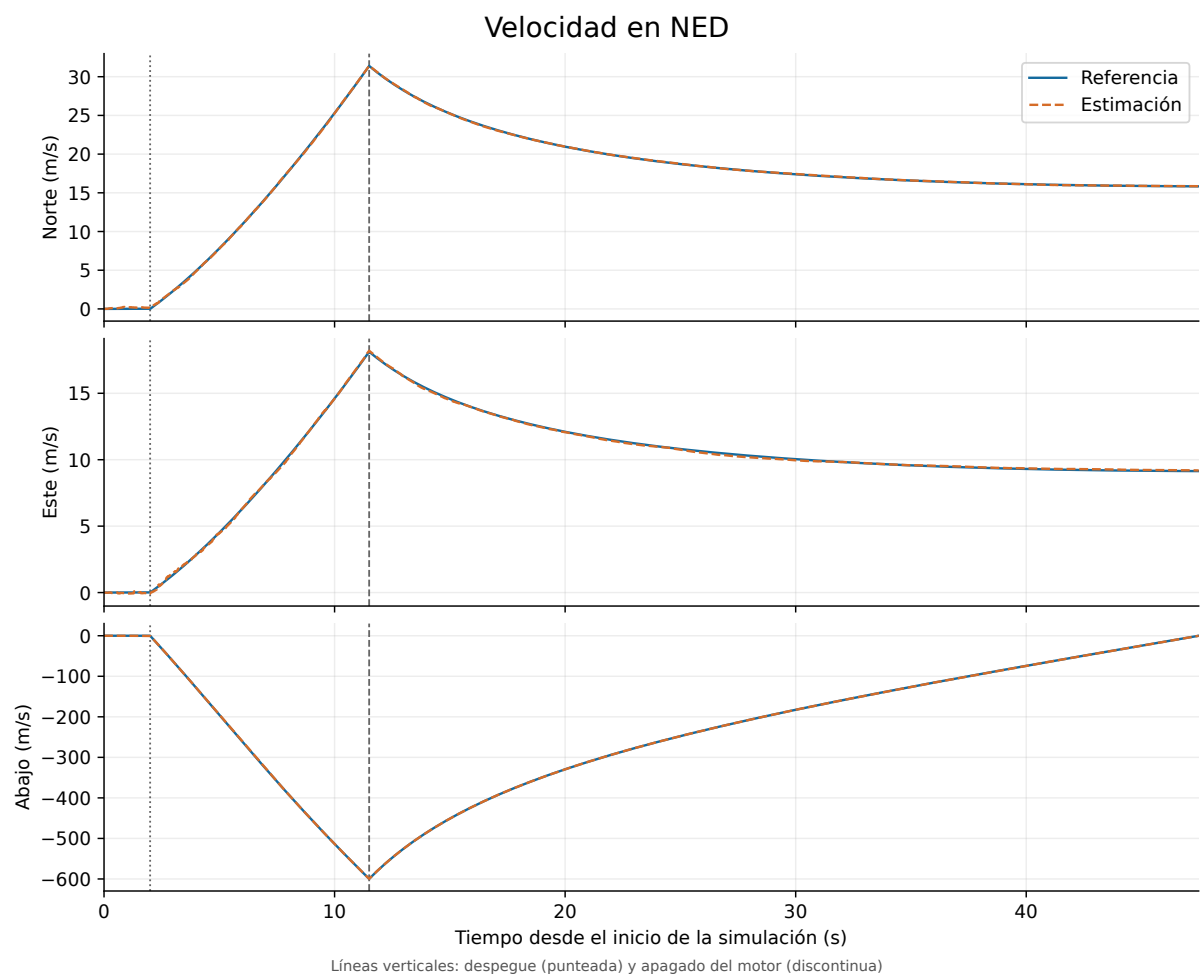


Figura 9: Velocidad de referencia y estimada en las componentes norte, este y abajo durante la segunda trayectoria.

Errores de posición y velocidad e incertidumbre estimada. Las siguientes figuras muestran los errores de posición y velocidad, calculados como referencia menos estimación, junto con las bandas $\pm 3\sigma$ obtenidas de la covarianza del filtro.

Para cada componente del estado de error, calculamos:

$$\sigma_i(t) = \sqrt{P_{ii}(t)}. \quad (90)$$

La curva de error muestra la diferencia real respecto a la referencia; la banda representa la incertidumbre que el estimador atribuye a esa componente. Su comparación permite comprobar si el filtro está asignando una incertidumbre acorde con los errores observados.

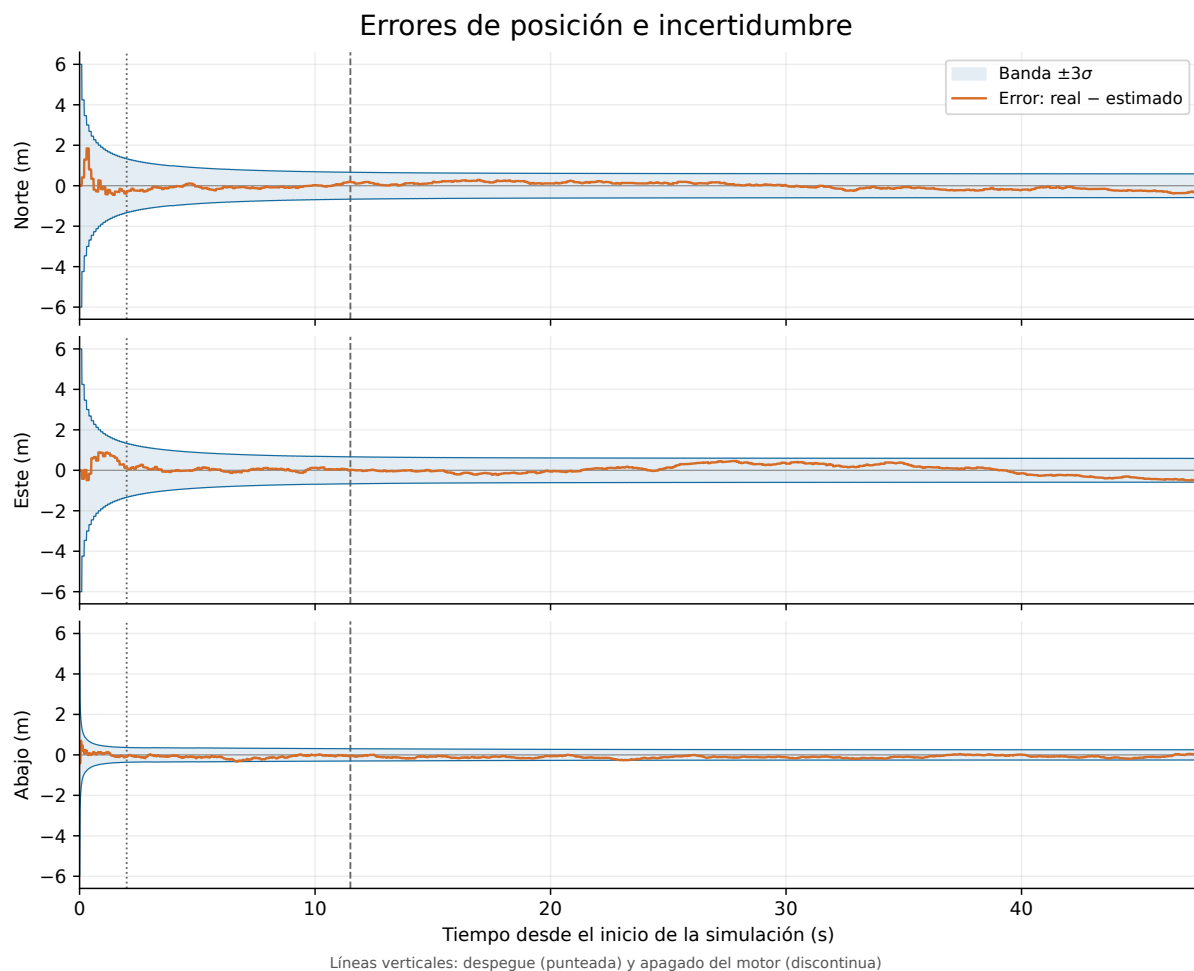


Figura 10: Errores de posición en NED y bandas $\pm 3\sigma$ obtenidas de la covarianza del estimador en la segunda trayectoria.

En la Figura 10, las bandas de posición se estrechan rápidamente durante los primeros segundos, a medida que se incorporan las medidas externas. La componente vertical alcanza una banda más estrecha que las horizontales, coherente con la disponibilidad de las correcciones barométricas además del GNSS.

Los máximos errores absolutos son aproximadamente 1,85 m en norte, 0,886 m en este y 0,699 m en Down. Tras el ajuste inicial, los errores se mantienen en el orden de unas décimas de metro.

Las componentes norte y este permanecen dentro de sus bandas durante todo el registro. La componente Down presenta algunas salidas puntuales, aunque la mayor parte de sus muestras también queda dentro de ellas.

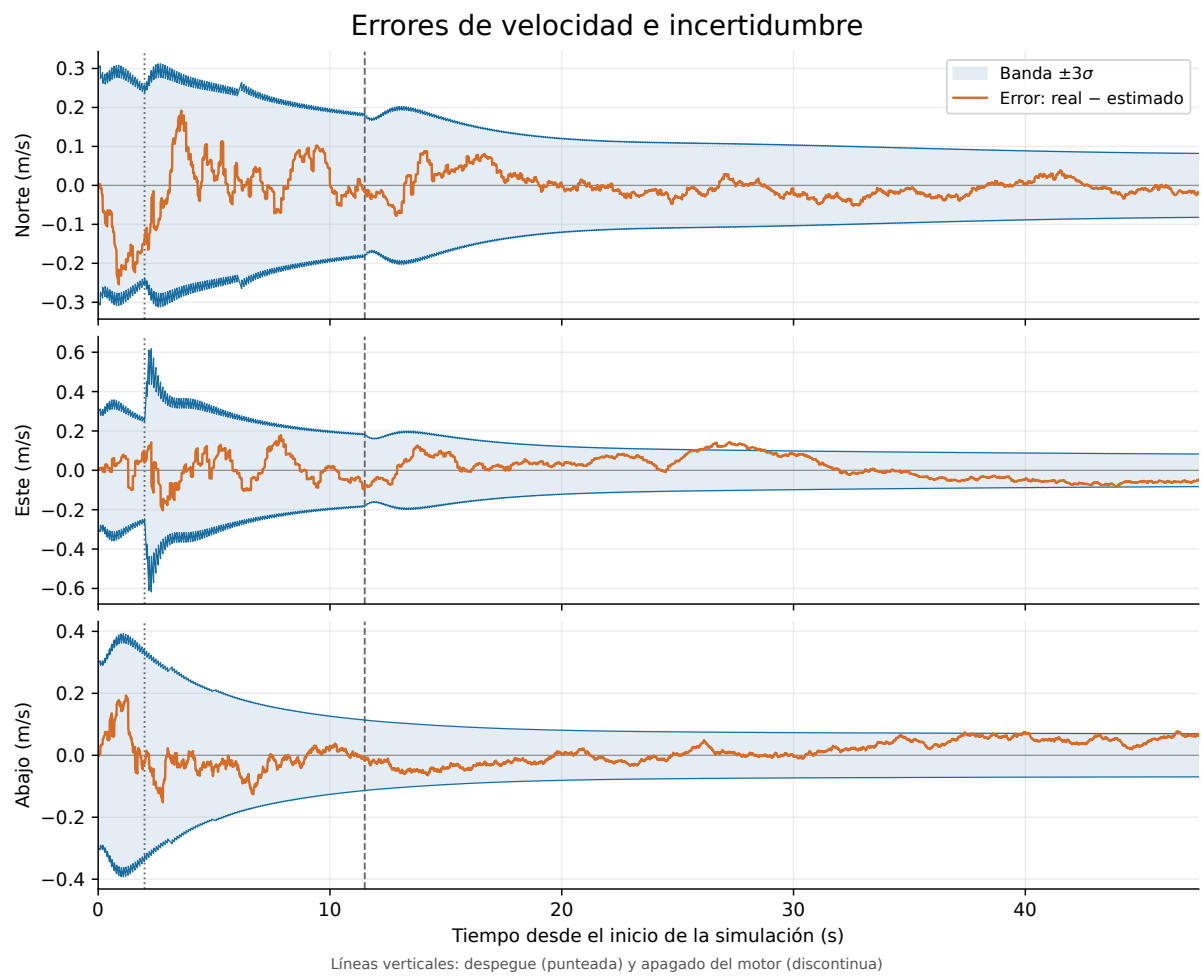


Figura 11: Errores de velocidad en NED y bandas $\pm 3\sigma$ obtenidas de la covarianza del estimador en la segunda trayectoria.

En la Figura 11, las bandas no disminuyen de manera monótona. Se observan variaciones alrededor del despegue y del final de la propulsión. La incertidumbre de velocidad depende también de los errores de orientación y de los sesgos, cuya influencia se transmite mediante el modelo de propagación.

Los máximos errores absolutos de velocidad son aproximadamente 0,254 m/s en norte, 0,204 m/s en este y 0,193 m/s en Down. Las oscilaciones más pronunciadas se concentran en la parte inicial del vuelo y posteriormente se reducen.

La componente norte permanece dentro de su banda durante todo el registro. En cambio, el error de velocidad este supera la banda superior aproximadamente entre 25,9 y 28,8 s. También aparecen salidas puntuales en la componente Down durante la parte final.

Seguimiento de la orientación. La Figura 12 compara las cuatro componentes del cuaternión estimado con las de referencia. Su evolución recoge tanto el giro longitudinal como el cambio de inclinación del cohete.

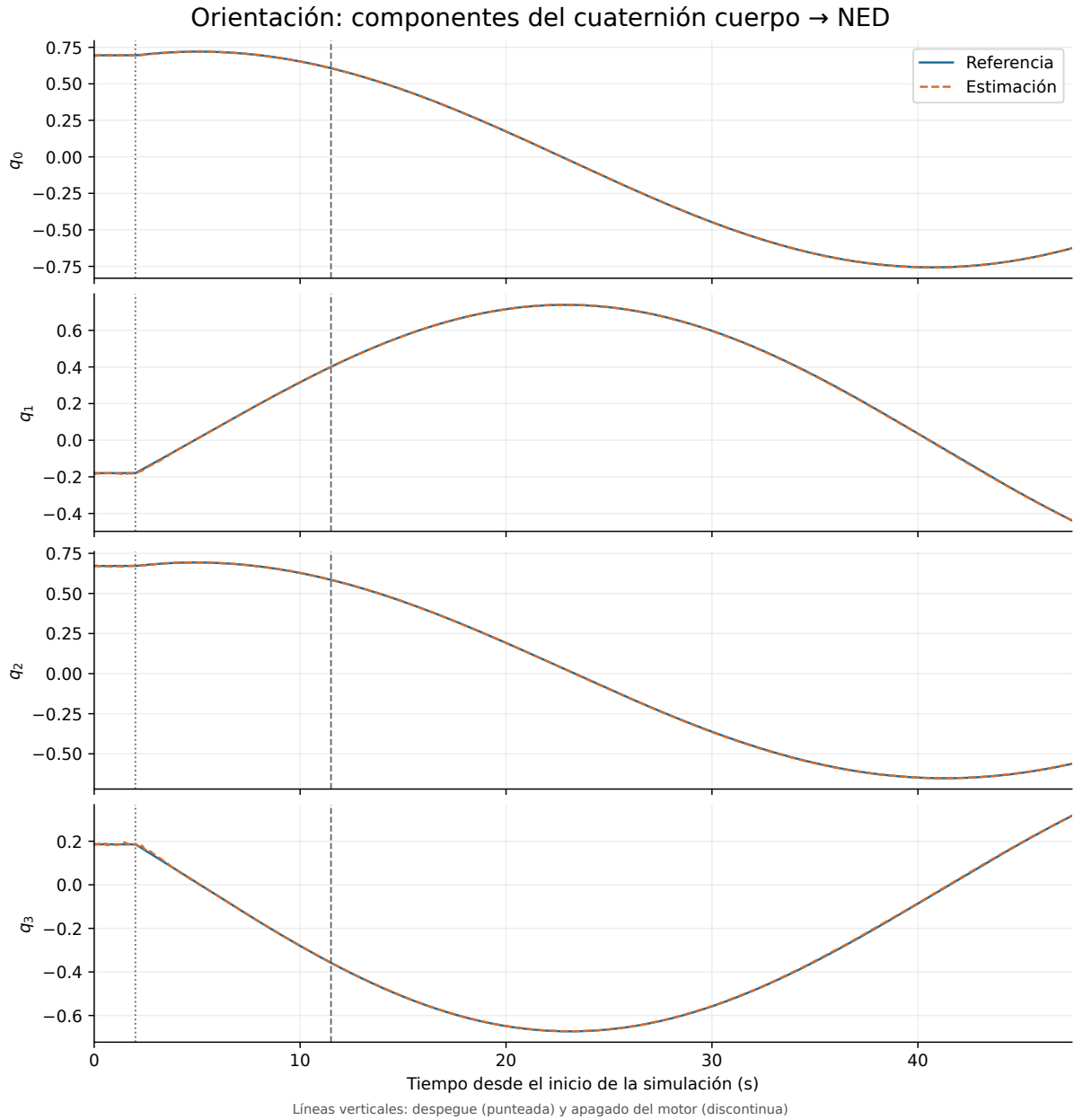


Figura 12: Componentes del cuaternión de referencia y estimado, que representa la orientación del cuerpo respecto a NED.

Las componentes estimadas siguen de cerca sus referencias durante todo el vuelo. Los cambios de signo de las componentes forman parte de la evolución del cuaternión y no representan, por sí mismos, saltos en la orientación.

Como $\mathbf{q}$ y $-\mathbf{q}$ representan la misma rotación, al dibujar las curvas se elige el signo del cuaternión estimado que permite compararlo con el de referencia. Esta operación afecta únicamente a su representación gráfica. Para cuantificar la diferencia entre ambas orientaciones utilizamos el ángulo de la rotación relativa que las separa.

Calculamos el cuaternión de error local mediante:

$$\delta q = \hat{q}^{-1} \otimes q_{\text{ref}} = \begin{bmatrix} s \\ \mathbf{u} \end{bmatrix}. \quad (91)$$

Para cuaterniones unitarios, la inversa coincide con el conjugado. Si $s < 0$, cambiamos el signo de todo el cuaternión para seleccionar la rotación de menor ángulo.

El error angular total y su vector local se obtienen como:

$$\varepsilon_\theta = 2 \operatorname{atan2}(\|\mathbf{u}\|, s), \quad \delta\boldsymbol{\theta} = \frac{\varepsilon_\theta}{\|\mathbf{u}\|} \mathbf{u}. \quad (92)$$

Para $\|\mathbf{u}\|$ próximo a cero utilizamos $\delta\boldsymbol{\theta} \approx 2\mathbf{u}$. Estas magnitudes se calculan en radianes y se convierten a grados para las gráficas.

La Figura 13 muestra este error y compara la inclinación del eje longitudinal respecto a la vertical.

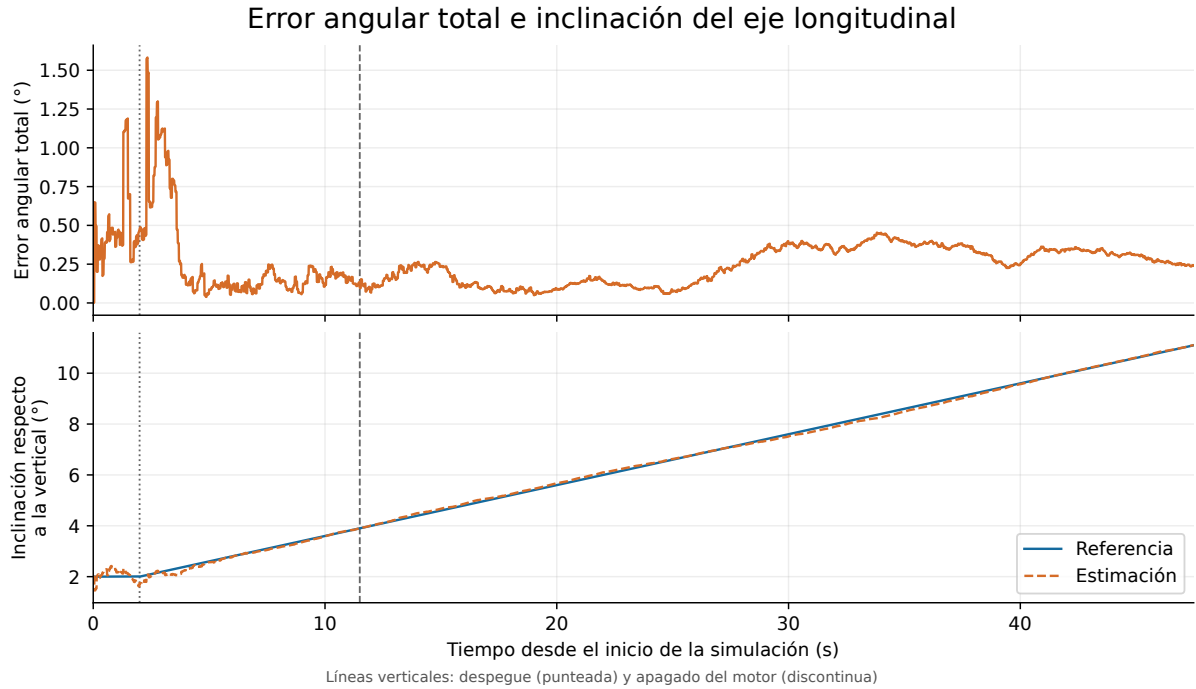


Figura 13: Error angular total de orientación e inclinación del eje longitudinal respecto a la vertical en la segunda trayectoria.

El error angular presenta sus mayores oscilaciones durante los primeros segundos y alcanza un máximo de aproximadamente $1,58^\circ$ en $t = 2,34\text{s}$, poco después del despegue.

Aunque la orientación inicial se proporciona correctamente al estimador, el ruido de las medidas y los sesgos inicialmente desconocidos producen diferencias durante la evolución posterior. Por tanto, partir de la orientación correcta no implica mantener un error nulo.

La inclinación de referencia aumenta desde 2° hasta aproximadamente $11,10^\circ$. La estimación reproduce esta evolución, con pequeñas diferencias que vemos especialmente durante los primeros segundos.

Estas gráficas permiten comprobar el seguimiento de una orientación variable, aspecto que la primera trayectoria no ponía a prueba. A continuación examinaremos las tres componentes del error angular local y su relación con la incertidumbre estimada.

Error angular local e incertidumbre. La Figura 14 descompone la diferencia de orientación en las tres componentes del error angular local $\delta\theta$ y las compara con sus bandas $\pm 3\sigma$.

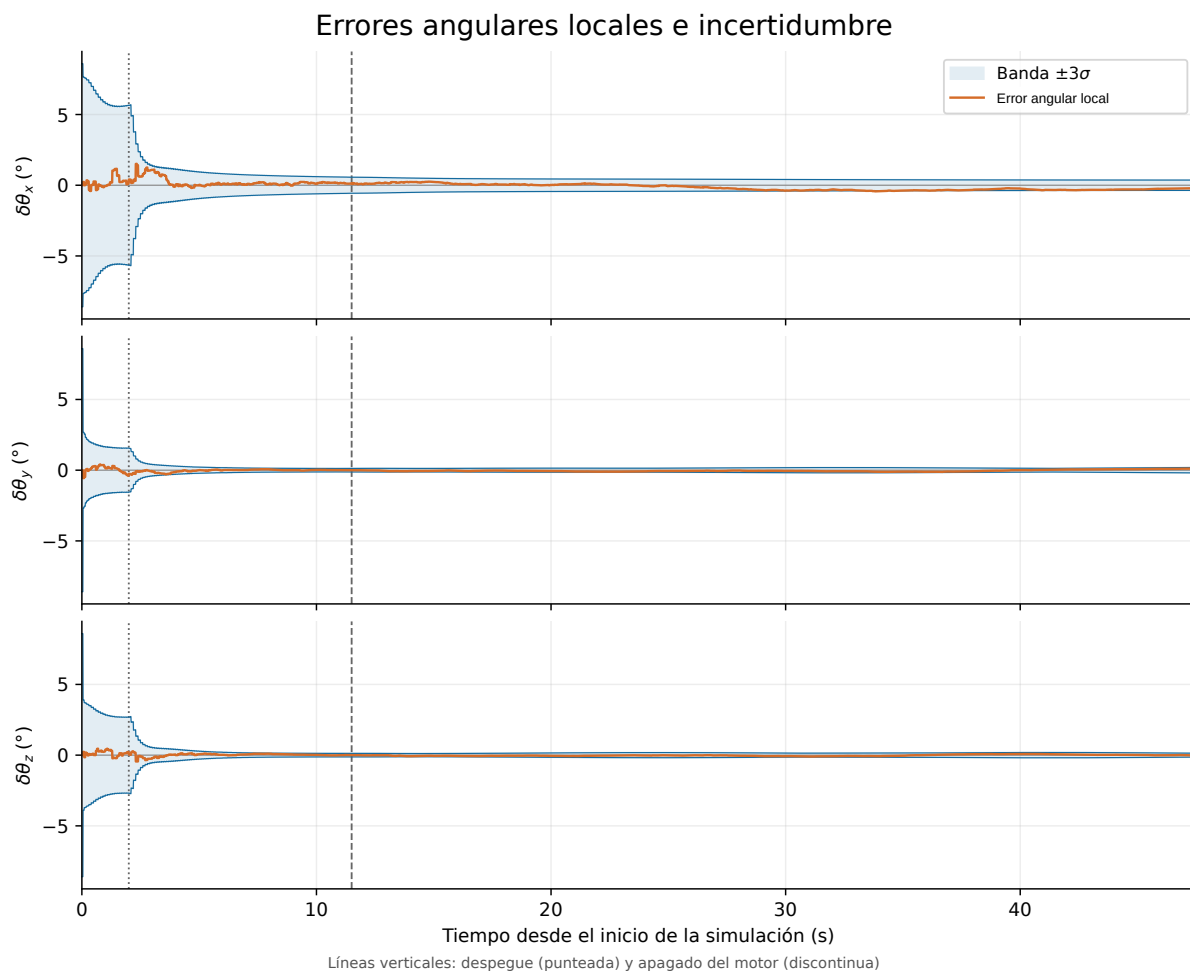


Figura 14: Componentes del error angular local y bandas $\pm 3\sigma$ en la segunda trayectoria. Los errores y las desviaciones típicas se convierten de radianes a grados para su representación.

Estas componentes describen la pequeña rotación que relaciona la orientación estimada con la de referencia, expresada en los ejes locales del cuerpo según la convención adoptada.

Las bandas se estrechan durante los primeros segundos, especialmente después del despegue. Esta reducción indica que el estimador atribuye una incertidumbre progresivamente menor a la orientación al incorporar las medidas disponibles.

La componente $\delta\theta_x$, asociada al eje longitudinal del cuerpo, presenta el mayor error y conserva una banda más ancha que las componentes transversales. Su máximo absoluto es aproximadamente $1,51^\circ$, frente a $0,562^\circ$ y $0,458^\circ$ en las componentes y_b y z_b , respectivamente.

Esta diferencia es coherente con la geometría del problema: una pequeña rotación alrededor del

eje longitudinal modifica poco la dirección de una fuerza específica predominantemente alineada con ese eje. Por ello, su efecto sobre la predicción de velocidad puede ser menor que el de un error de inclinación. La información magnética también contribuye a corregir la orientación, según la dirección del campo respecto al cuerpo.

Las componentes $\delta\theta_y$ y $\delta\theta_z$ permanecen dentro de sus bandas durante todo el registro. En cambio, $\delta\theta_x$ presenta algunas salidas por el límite inferior entre $t = 33,4$ s y $t = 36,2$ s aproximadamente.

El error angular total es reducido durante buena parte del vuelo, pero el error alrededor del eje longitudinal supera en algunos instantes la banda prevista por el filtro. Conviene comprobar si esto se repite al cambiar el ruido y las condiciones iniciales.

Al finalizar la simulación, las componentes del error angular local son aproximadamente:

$$\delta\theta(t_f) = \begin{bmatrix} -0,210 \\ 0,113 \\ -0,021 \end{bmatrix}^\circ. \quad (93)$$

Su módulo corresponde al error angular total final de aproximadamente $0,240^\circ$ mostrado anteriormente.

Estimación de los sesgos del acelerómetro. La Figura 15 compara los sesgos estimados del acelerómetro con los valores constantes utilizados para generar las medidas. Conservamos los mismos sesgos de referencia que en la primera trayectoria: $0,08$, $-0,05$ y $0,03$ m/s² en los ejes x_b , y_b y z_b , respectivamente. Las tres estimaciones parten de cero.

La componente x_b presenta oscilaciones iniciales y supera temporalmente su referencia. Posteriormente se aproxima a ella y mantiene variaciones reducidas durante el resto del vuelo.

La componente y_b muestra un ajuste más lento. Durante la propulsión alcanza valores más negativos que el sesgo real y, tras el apagado, se aproxima progresivamente a su referencia. La componente z_b también presenta desviaciones y termina ligeramente por debajo de su valor real. Estos cambios corresponden al ajuste de las estimaciones mediante las correcciones del filtro.

En el último instante se obtiene aproximadamente:

$$\hat{\mathbf{b}}_a(t_f) = \begin{bmatrix} 0,07809 \\ -0,05157 \\ 0,02603 \end{bmatrix} \text{ m/s}^2. \quad (94)$$

Los errores finales, calculados como referencia menos estimación, son aproximadamente $0,00191$, $0,00157$ y $0,00397$ m/s². Las tres componentes terminan próximas a sus referencias, aunque ninguna tiene por qué alcanzar exactamente su valor real en una ejecución con ruido.

La Figura 16 permite comparar estos errores con la incertidumbre que les asigna el estimador.

Las bandas se estrechan a medida que el filtro incorpora información. La reducción es más rápida en la componente longitudinal, mientras que las componentes transversales conservan una incertidumbre mayor durante la propulsión y muestran una reducción adicional tras el apagado.

En esta ejecución, los errores de las tres componentes permanecen dentro de sus bandas durante todo el registro. Las estimaciones finales próximas a la referencia y la reducción de sus errores muestran que el filtro consigue ajustar los sesgos inicialmente desconocidos en este escenario.

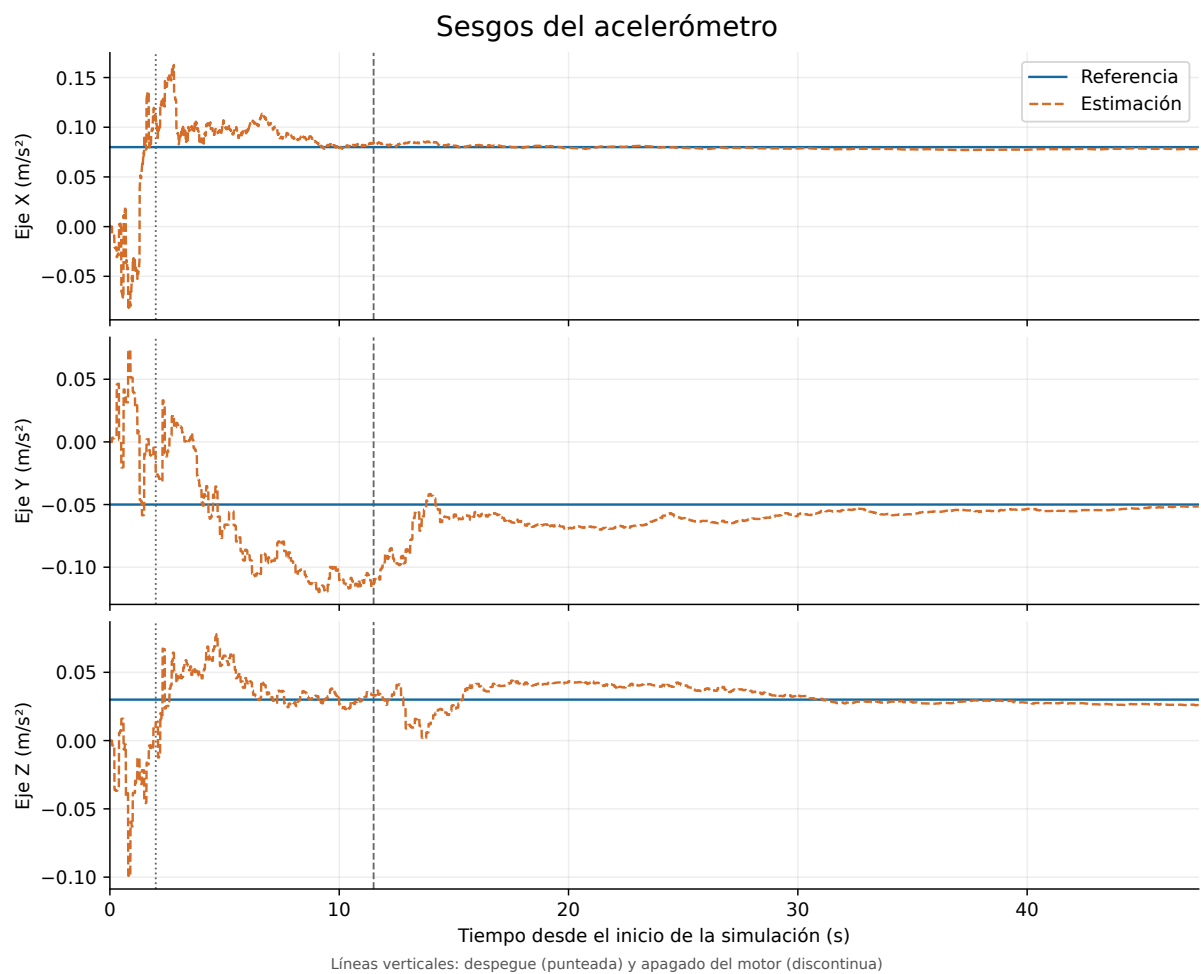


Figura 15: Sesgos estimados del acelerómetro y valores de referencia en la segunda trayectoria, expresados en los ejes del cuerpo.

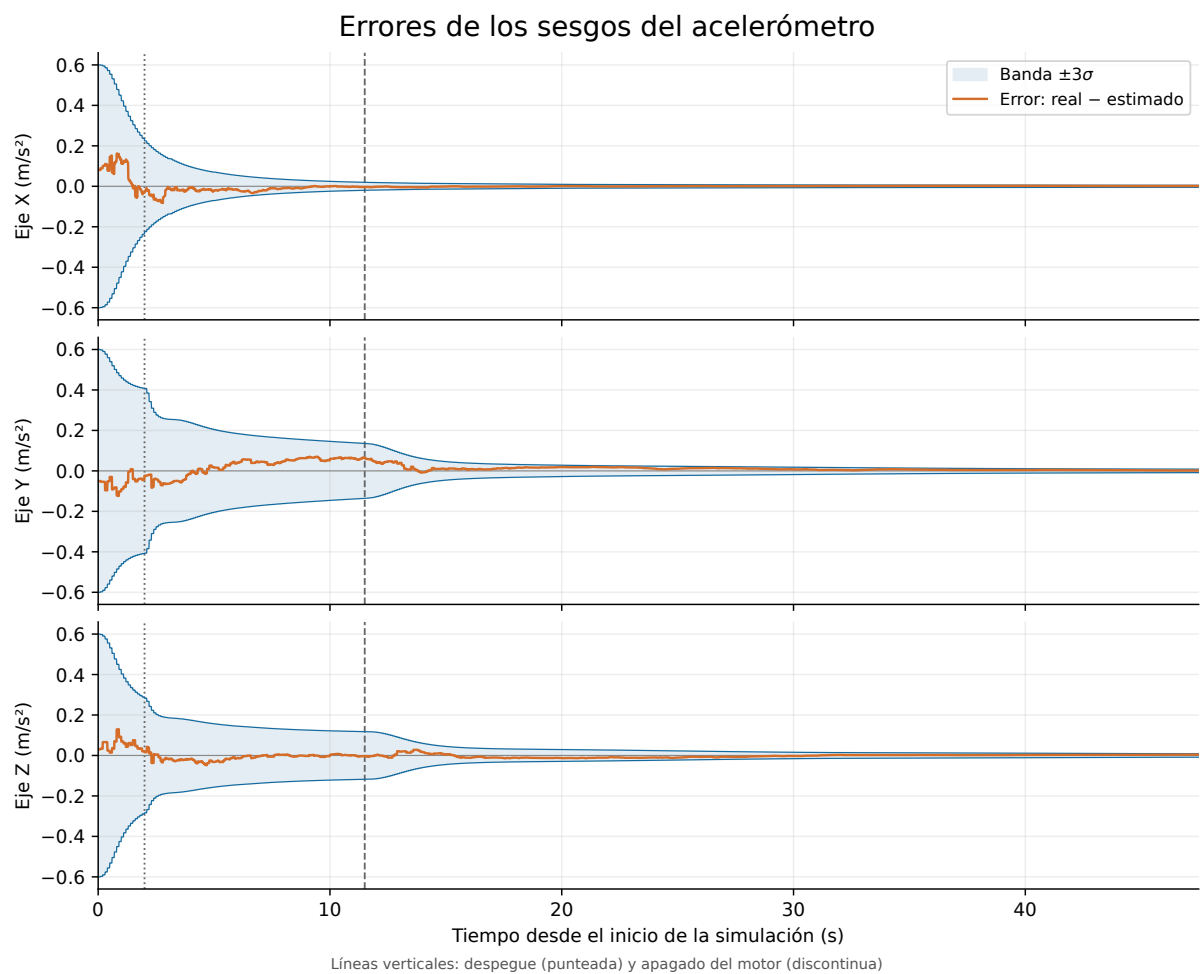


Figura 16: Errores de estimación de los sesgos del acelerómetro y bandas $\pm 3\sigma$ en la segunda trayectoria.

Estimación de los sesgos del giróscopo. La Figura 17 compara los sesgos estimados del giróscopo con sus referencias constantes: 0,001, -0,0007 y 0,0005 rad/s en los ejes x_b , y_b y z_b , respectivamente.

A diferencia de la primera trayectoria, las lecturas contienen ahora una velocidad angular real distinta de cero durante el vuelo. El estimador debe seguir ese movimiento y ajustar simultáneamente los sesgos.

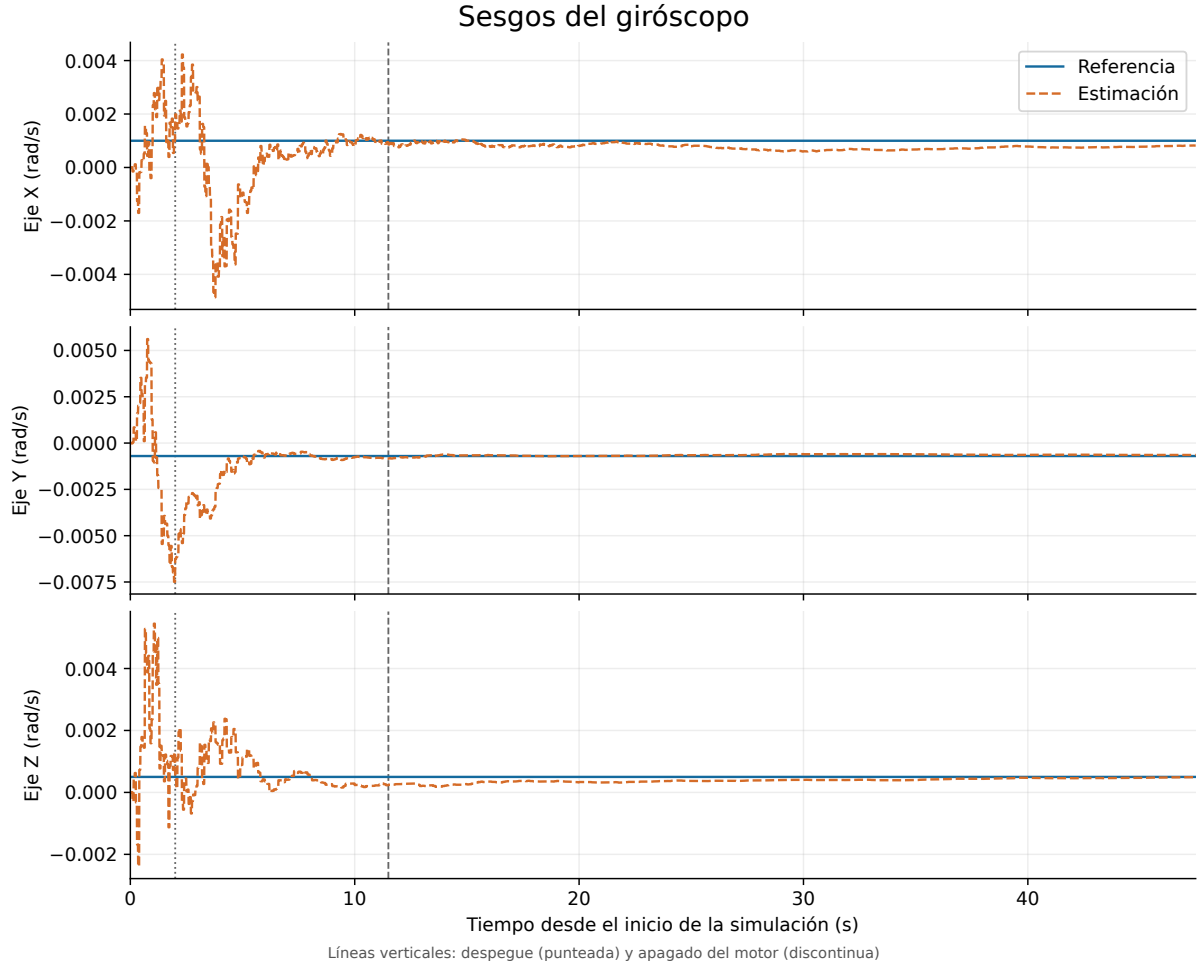


Figura 17: Sesgos estimados del giróscopo y valores de referencia en la segunda trayectoria, expresados en los ejes del cuerpo.

Las tres componentes presentan oscilaciones iniciales de amplitud superior a los propios sesgos reales. Posteriormente, las estimaciones se aproximan a sus referencias y sus variaciones se reducen.

La componente x_b supera inicialmente su referencia, toma después valores negativos y vuelve a aproximarse al valor real. Durante la parte final mantiene una pequeña diferencia respecto a este. La componente y_b se estabiliza cerca de su referencia tras los instantes iniciales, mientras que la componente z_b continúa ajustándose gradualmente durante el vuelo.

Las estimaciones finales son aproximadamente:

$$\hat{\mathbf{b}}_g(t_f) = \begin{bmatrix} 0,0008225 \\ -0,0006431 \\ 0,0004906 \end{bmatrix} \text{ rad/s.} \quad (95)$$

Los errores finales, definidos como referencia menos estimación, son:

$$\delta \mathbf{b}_g(t_f) \approx \begin{bmatrix} 1,78 \times 10^{-4} \\ -5,69 \times 10^{-5} \\ 9,44 \times 10^{-6} \end{bmatrix} \text{ rad/s.} \quad (96)$$

La componente longitudinal conserva el mayor error final, mientras que la componente z_b termina especialmente próxima a su referencia. Para valorar estos errores respecto a la incertidumbre calculada, utilizamos la Figura 18.

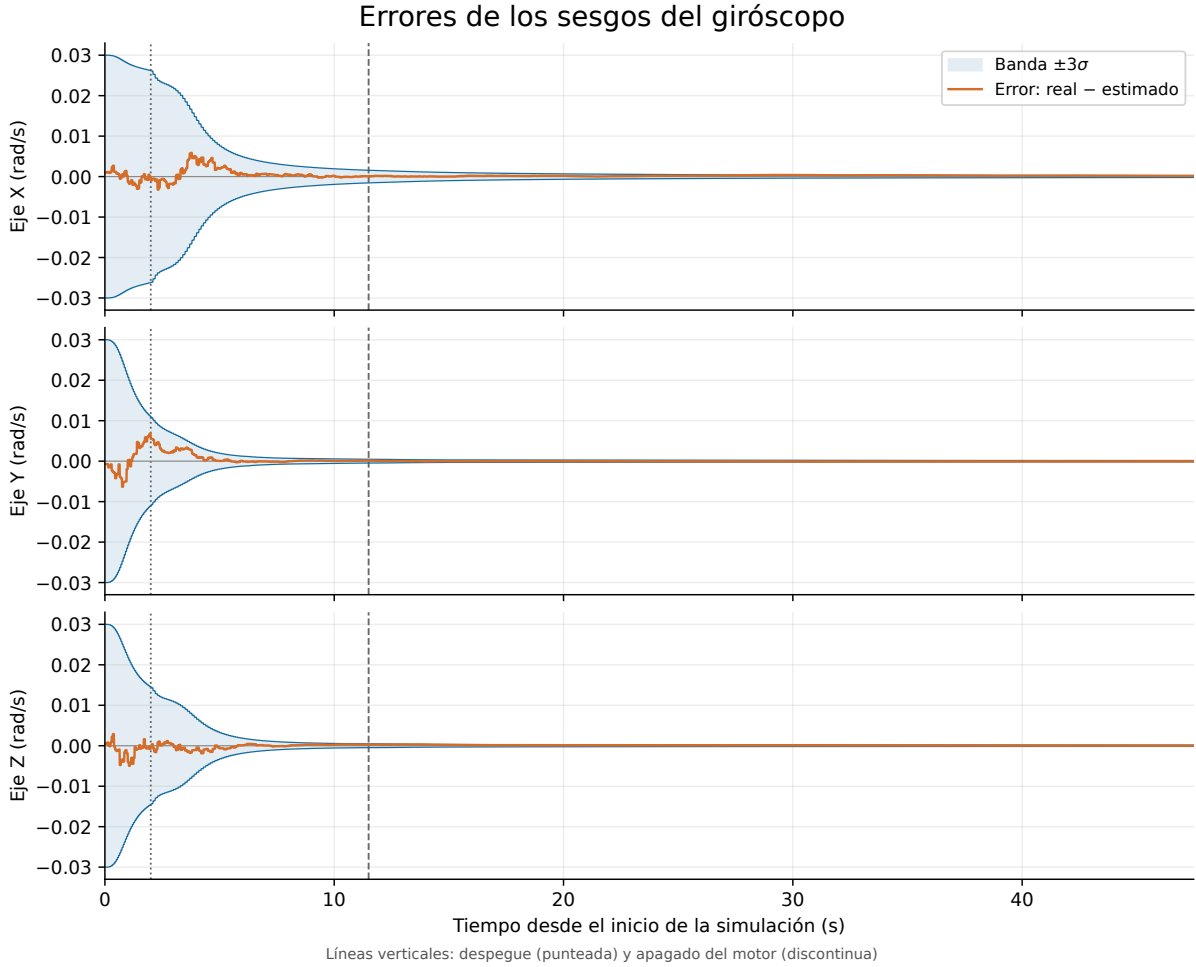


Figura 18: Errores de estimación de los sesgos del gir6scopo y bandas $\pm 3\sigma$ en la segunda trayectoria.

Las bandas se estrechan conforme avanza la simulación. Los errores de las componentes y_b y z_b permanecen dentro de sus bandas durante todo el registro.

En cambio, el error del sesgo longitudinal supera el límite superior en distintos intervalos comprendidos aproximadamente entre 29,1 y 44,7 s. En conjunto, estas salidas afectan a cerca del 12 por ciento de las muestras registradas.

Evolución conjunta de la incertidumbre. La Figura 19 reúne las desviaciones típicas de los quince componentes del estado de error. Representamos $\sigma_i = \sqrt{P_{ii}}$, en lugar de las bandas $3\sigma_i$ utilizadas en las figuras anteriores.

El eje vertical de cada panel utiliza una escala logarítmica para mostrar la evolución de magni-

tudes que cambian varios órdenes de magnitud durante el vuelo. Las curvas representan incertidumbres calculadas por el filtro, no errores medidos respecto a la referencia.

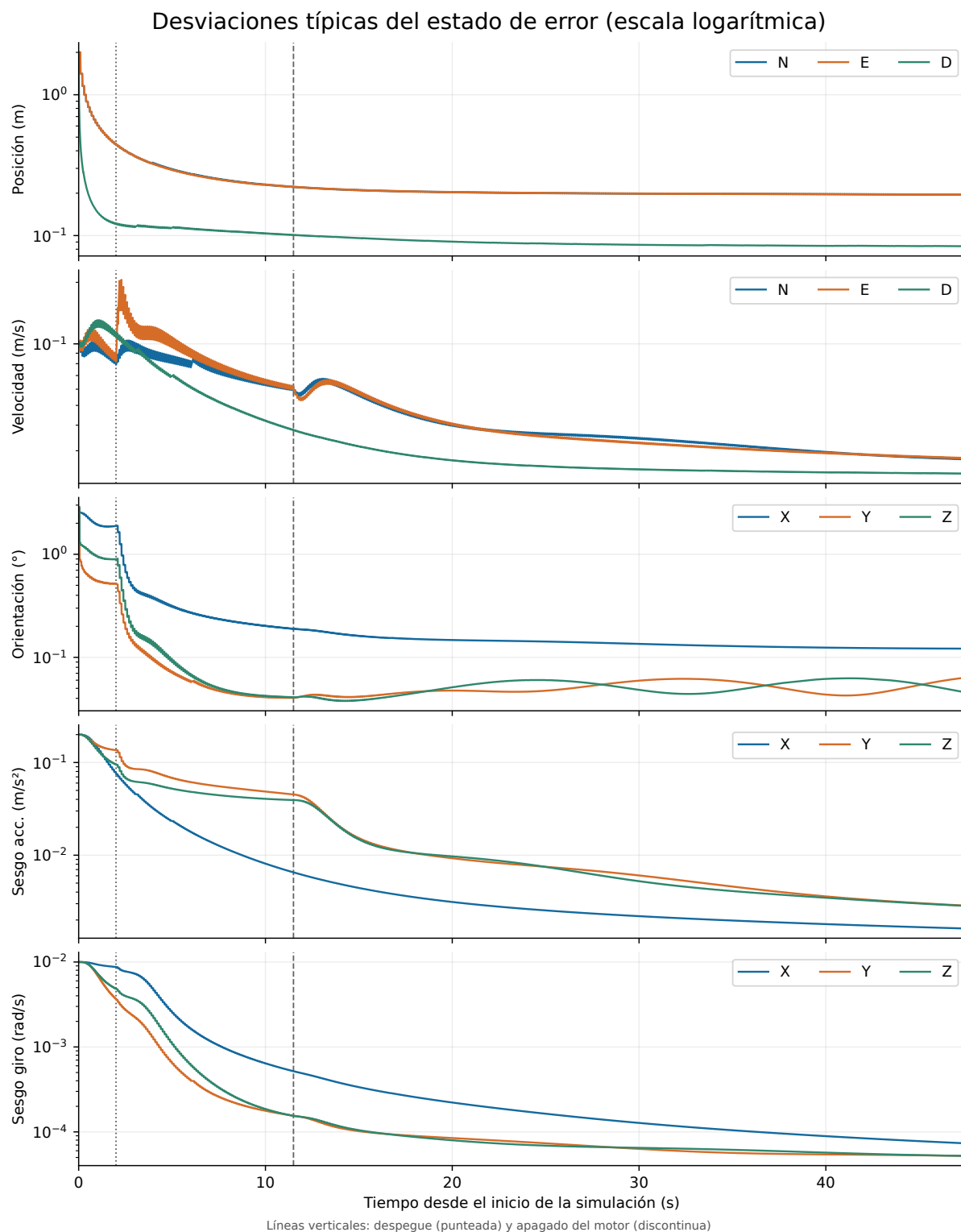


Figura 19: Evolución de las desviaciones típicas de posición, velocidad, error angular y sesgos en la segunda trayectoria. Cada panel utiliza una escala vertical logarítmica y las unidades indicadas.

La incertidumbre de posición disminuye rápidamente al inicio y después se aproxima a un nivel prácticamente estable. La componente vertical mantiene una desviación típica menor que las horizontales, en presencia de las referencias barométricas y del GNSS.

En velocidad aparecen aumentos transitorios, especialmente alrededor del despegue y después del apagado del motor. En orientación, la componente longitudinal conserva una incertidumbre mayor que las transversales. Estas últimas presentan oscilaciones durante la fase sin propulsión.

Las desviaciones típicas de los sesgos disminuyen progresivamente. En esta simulación los sesgos reales son constantes y no se introduce ruido de proceso asociado a su variación, por lo que el filtro puede acumular información sobre ellos a lo largo del registro.

Que las bandas se estrechen significa que el filtro asigna más confianza a sus estimaciones. Para comprobar si esa confianza está justificada debemos compararlas con los errores, como muestran las salidas observadas en el sesgo longitudinal del giróscopo.

Resumen de los errores de seguimiento. Para valorar el error de posición en una sola magnitud, calculamos la distancia entre la posición estimada y la de referencia. Aplicamos el mismo criterio al vector de velocidad:

$$E_p = \|\delta \mathbf{p}\| = \sqrt{\delta p_N^2 + \delta p_E^2 + \delta p_D^2}, \quad E_v = \|\delta \mathbf{v}\| = \sqrt{\delta v_N^2 + \delta v_E^2 + \delta v_D^2}. \quad (97)$$

Estas magnitudes son siempre no negativas y permiten valorar el error conjunto sin que las componentes de distinto signo se compensen. En particular, E_p representa la distancia entre la posición estimada y la de referencia.

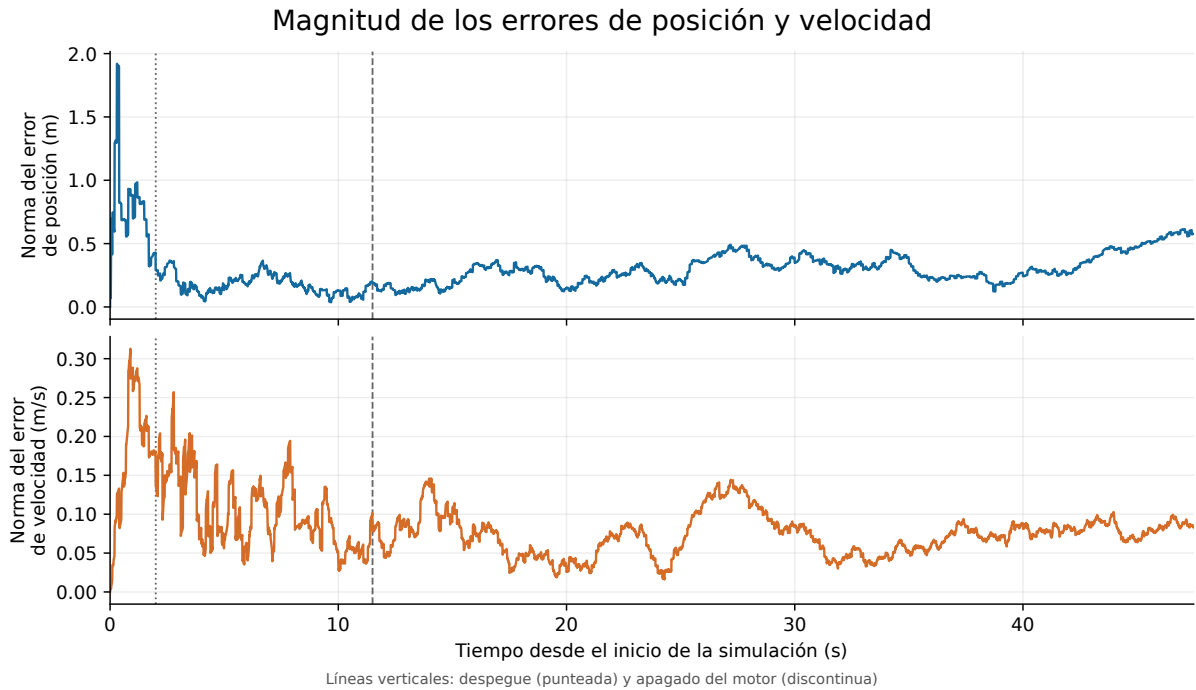


Figura 20: Normas de los errores de posición y velocidad en la segunda trayectoria.

La Figura 20 muestra que los máximos aparecen durante el reposo inicial. El error de posición alcanza aproximadamente 1,92 m en $t = 0,30$ s y el de velocidad, 0,313 m/s en $t = 0,89$ s.

Tras el transitorio inicial, ambos errores se reducen y presentan oscilaciones a lo largo del vuelo. El error de posición aumenta durante el tramo final y termina en aproximadamente 0,577 m. El de velocidad finaliza en 0,0830 m/s.

La Tabla 5 resume estos resultados junto con el error angular total. Para mantener el criterio utilizado en la primera trayectoria, los valores RMS se calculan sobre todas las muestras, incluida la inicial. En posición y velocidad se utiliza la raíz de la media de E_p^2 y E_v^2 , respectivamente.

Tabla 5: Resumen de los errores de la segunda trayectoria, calculado sobre las 4749 muestras del registro.

Magnitud	RMS	Máximo	Valor final
E_p (m)	0,342	1,920	0,577
E_v (m/s)	0,0932	0,313	0,0830
Error angular total ($^\circ$)	0,314	1,582	0,240

Balance de la segunda trayectoria. El estimador sigue el movimiento en las tres componentes NED y reproduce la orientación variable durante el ascenso. Los sesgos estimados terminan próximos a sus referencias, aunque presentan transitorios y diferencias entre ejes.

El registro de la covarianza permite además identificar aspectos que no podían evaluarse con los resultados guardados en la primera simulación. Algunas componentes presentan errores que superan sus bandas de incertidumbre, especialmente la velocidad este y el sesgo longitudinal del giróscopo.

Por tanto, esta prueba muestra un seguimiento preciso dentro del escenario simulado, pero también intervalos en los que la incertidumbre calculada resulta demasiado reducida respecto al error observado. Ambos aspectos se tendrán en cuenta al comparar las trayectorias y delimitar el alcance de los resultados.

2.14. Comparación y conclusiones

Las dos trayectorias permiten comprobar aspectos distintos del estimador. La primera facilita detectar errores de signos, unidades e integración, porque el movimiento es vertical y la orientación permanece constante. La segunda exige transformar correctamente las medidas mientras el cohete cambia de orientación y se desplaza horizontalmente. Utilizar el mismo filtro en ambas permite comprobar su comportamiento sin modificar los algoritmos para adaptarlos a cada trayectoria.

Comparación de los resultados. La Tabla 6 compara los errores verticales mediante su RMSE, calculado sobre todas las muestras registradas, incluida la inicial.

Tabla 6: Comparación de los errores verticales de ambas trayectorias.

Magnitud	Trayectoria 1	Trayectoria 2
RMSE de altura (m)	0,1231	0,1194
RMSE de velocidad vertical (m/s)	0,0527	0,0437

El RMSE de altura es de unos 0,12 m en ambas pruebas. El de velocidad vertical también es similar, aunque algo menor en la segunda. Estos valores resumen el registro completo; durante el vuelo aparecen errores mayores, especialmente en los primeros segundos. Por tanto, introducir cambios de orientación y desplazamiento horizontal no deteriora apreciablemente el seguimiento vertical en estas ejecuciones. Hay que tener en cuenta que ambos escenarios mantienen las correcciones barométricas y GNSS disponibles durante todo el ascenso.

La segunda trayectoria permite comprobar además el seguimiento de la orientación: el error angular total alcanza un máximo de $1,58^\circ$ durante el transitorio inicial y termina en aproximadamente $0,24^\circ$. Las gráficas muestran que el estimador sigue tanto la inclinación como el giro longitudinal, aunque las componentes del error no se comportan igual.

Algo parecido sucede con los sesgos. En la primera trayectoria, el sesgo longitudinal del gir6scopo termina alejado de su valor real pese al buen seguimiento vertical. En la segunda, su error final disminuye de aproximadamente $1,10 \times 10^{-3}$ a $1,78 \times 10^{-4}$ rad/s. Sin embargo, los sesgos del aceler6metro no mejoran en todos los ejes. No podemos atribuir estas diferencias 6nicamente al movimiento, porque tambi6n cambian la duraci6n de la prueba y las condiciones en las que act6a el ruido.

El principal aspecto que debemos revisar. Varios errores superan las bandas de $\pm 3\sigma$. El sesgo longitudinal del gir6scopo lo hace en cerca del 12% de las muestras. Estas est6n correlacionadas: el porcentaje describe esta ejecuci6n, pero no constituye por s6 solo una prueba estadística de consistencia.

El punto a revisar es, por tanto, si el filtro est6 atribuyendo demasiada confianza a ciertas estimaciones. Antes de aumentar los par6metros de ruido, comprobaríamos la correspondencia entre el ruido simulado y su discretizaci6n, las aproximaciones del modelo de error y el efecto de las correcciones secuenciales. Aumentar las covarianzas podría ensanchar las bandas, pero no explicaría por s6 solo el origen de la discrepancia. Repetiríamos primero la simulaci6n con distintas semillas de ruido y errores iniciales. As6 comprobaríamos si las salidas de las bandas se repiten en los mismos ejes y fases del vuelo. Tambi6n registraríamos las innovaciones y las medidas rechazadas para identificar qu6 actualizaciones coinciden con esas desviaciones.

Qu6 falta para representar un lanzamiento real. La segunda trayectoria introduce m6s exigencia al estimador, pero conserva simplificaciones importantes. La orientaci6n se prescribe en lugar de obtenerse de los momentos que act6an sobre el cohete. El movimiento permanece en un plano vertical, no se introduce viento y se utilizan aproximaciones sencillas para el empuje, la p6rdida de masa y la resistencia aerodin6mica. Tampoco se simulan el descenso ni la recuperaci6n.

Desde el punto de vista de la fusi6n, la siguiente prueba m6s 6til sería retirar temporalmente alguna referencia externa y comprobar cu6nto crece el error y c6mo se recupera el filtro. Tambi6n introduciríamos errores de orientaci6n inicial, sesgos variables, retrasos y saturaciones. Estas situaciones pondrían a prueba aspectos que las dos trayectorias actuales, con sensores siempre disponibles y sincronizados, no permiten evaluar.

Aplicaci6n a Faraday. Para unos 9 km, comprobaremos los l6mites de los sensores con la altitud real de la rampa y la presi6n en el apogeo. Los LPS22DF miden entre 26 y 126 kPa [6]. La segunda simulaci6n alcanza unos 10,7 km sobre su origen y conserva bar6metros sint6ticos durante todo el ascenso: sus errores verticales no representan directamente las prestaciones de estos sensores.

La instalaci6n tambi6n influir6 en las medidas. Habría que calibrar y alinear la IMU, evaluar las vibraciones y comprobar que las tomas barom6tricas proporcionan una presi6n representativa. En el magnet6metro sería necesario medir las perturbaciones introducidas por la electr6nica y los materiales del cohete. La separaci6n entre la antena GNSS y la IMU debería incorporarse al modelo cuando produzca diferencias relevantes durante los giros.

En software, una prioridad sería resolver la recuperaci6n tras p6rdidas de muestras. Actualmente, un intervalo excesivo se rechaza y el reloj del estimador no avanza, de modo que las muestras posteriores pueden seguir siendo rechazadas. Este comportamiento es admisible como detecci6n de un problema en la prueba, pero necesita una estrategia de recuperaci6n antes de utilizarse en vuelo.

Diferencias para un vuelo al espacio. Para responder al alcance espacial del enunciado, sería necesario revisar la gravedad constante y la aproximación de navegación local. La importancia de la curvatura y de la rotación terrestre dependerá de la trayectoria, su duración y la precisión requerida. Si estos efectos resultan relevantes, deberán incorporarse a la propagación, utilizando coordenadas terrestres globales o inerciales cuando sea conveniente.

También cambiarían las referencias disponibles. La corrección barométrica desaparecería al salir del rango útil y el GNSS debería evaluarse para las condiciones concretas de la misión. Durante la caída libre ideal, el acelerómetro no proporciona una referencia de verticalidad. Habría que determinar durante cuánto tiempo puede mantenerse la precisión de orientación con el giróscopo y las referencias restantes, y si son necesarios otros sensores.

Conclusión. El trabajo permite comprobar la integración de la predicción inercial y las correcciones externas en un mismo estimador. Las dos pruebas muestran un seguimiento próximo a la referencia, pero también dejan una conclusión práctica: un error pequeño en altura o posición puede coexistir con errores en los sesgos o con una incertidumbre demasiado optimista en algunas componentes.

El siguiente paso sería revisar esa incertidumbre y someter el filtro a pérdidas de referencias y errores de inicialización, antes de pasar a ensayos con sensores reales. Con esas comprobaciones y la evaluación en el microcontrolador podríamos determinar qué prestaciones conserva la implementación fuera de las condiciones favorables de estas simulaciones.

Referencias

- [1] MathWorks. *IMU and GPS Fusion for Inertial Navigation*. Navigation Toolbox. <https://www.mathworks.com/help/nav/ug/imu-and-gps-fusion-for-inertial-navigation.html>. Consulta: 6 de octubre de 2026.
- [2] Smalling, K. M., y Eure, K. W. (2015). *A Short Tutorial on Inertial Navigation System and Global Positioning System Integration*. NASA/TM-2015-218803. <https://ntrs.nasa.gov/citations/20150018921>.
- [3] MathWorks. *Model IMU, GPS, and INS/GPS*. Sensor Fusion and Tracking Toolbox. <https://www.mathworks.com/help/fusion/gs/model-imu-gps-and-ins-gps.html>. Consulta: 6 de octubre de 2026.
- [4] MathWorks. *Determine Orientation Using Inertial Sensors*. Sensor Fusion and Tracking Toolbox. <https://www.mathworks.com/help/fusion/gs/determine-orientation-through.html>. Consulta: 6 de octubre de 2026.
- [5] Solà, J. (2017). *Quaternion kinematics for the error-state Kalman filter*. arXiv:1711.02508. <https://arxiv.org/abs/1711.02508>.
- [6] STMicroelectronics (2025). *LPS22DF: Low-power and high-precision MEMS nano pressure sensor*. DS13316, revisión 3. <https://www.st.com/resource/en/datasheet/lps22df.pdf>.
- [7] NOAA NCEI Geomagnetic Modeling Team y British Geological Survey (2024). *World Magnetic Model 2025*. NOAA NCEI. <https://doi.org/10.25921/aqfd-sd83>. Consulta: 7 de octubre de 2026.
- [8] Aerospace Team Graz. *ISPIDA: EuRoC 2025 Competition Rocket*. <https://astg.at/projects/ispida>.

A. Fundamento del filtro de Kalman extendido

Este anexo resume el EKF con actualización aditiva [2]. En el ESKF implementado, la covarianza describe quince errores; la orientación se corrige mediante la inyección y el reinicio del apartado 2.6. Las expresiones generales siguientes deben interpretarse con esa diferencia.

A.1. Modelo del estado y de las medidas

El estado contiene las variables que queremos estimar. Su evolución y las medidas de los sensores se representan como:

$$\mathbf{x}_k = f(\mathbf{x}_{k-1}, \mathbf{u}_k) + \mathbf{w}_k, \quad (98)$$

$$\mathbf{z}_k = h(\mathbf{x}_k) + \mathbf{v}_k. \quad (99)$$

La función f describe la evolución del estado, y $\mathbf{u}_k$ contiene las entradas utilizadas para predecirlo. La función h relaciona el estado con las medidas que esperamos recibir. Los términos $\mathbf{w}_k$ y $\mathbf{v}_k$ representan, respectivamente, el ruido del proceso y el de las mediciones.

Estas expresiones suponen ruidos aditivos de media cero y no correlacionados entre sí. Si el ruido entra de otra forma en el modelo, su contribución a la covarianza debe transformarse mediante la matriz correspondiente.

A.2. Representación de la incertidumbre

Para una variable escalar, la varianza se define como:

$$\sigma^2 = \text{E} \left[(x - \text{E}[x])^2 \right]. \quad (100)$$

El operador E representa el valor esperado. La desviación típica es σ , mientras que la varianza tiene las unidades de la variable al cuadrado.

La covarianza del error de estimación es:

$$P_k = \text{E} \left[(\mathbf{x}_k - \hat{\mathbf{x}}_k)(\mathbf{x}_k - \hat{\mathbf{x}}_k)^T \right]. \quad (101)$$

Aquí, $\mathbf{x}_k$ es el estado real y $\hat{\mathbf{x}}_k$ su estimación. Los elementos diagonales de P_k representan las varianzas de los errores de cada componente. Los elementos restantes recogen la relación entre los errores de distintas componentes. Esta expresión supone que el error de estimación tiene media nula bajo el modelo del filtro; en el caso general, la covarianza se define restando también la media del error.

Las covarianzas del ruido del proceso y de las mediciones son:

$$Q_k = \text{E} \left[\mathbf{w}_k \mathbf{w}_k^T \right], \quad (102)$$

$$R_k = \text{E} \left[\mathbf{v}_k \mathbf{v}_k^T \right]. \quad (103)$$

Por ejemplo, para una medida de posición GNSS con errores no correlacionados entre sus tres componentes:

$$R_{\text{GNSS},p} = \begin{bmatrix} \sigma_{p_N}^2 & 0 & 0 \\ 0 & \sigma_{p_E}^2 & 0 \\ 0 & 0 & \sigma_{p_D}^2 \end{bmatrix}. \quad (104)$$

Esta forma diagonal solo es adecuada cuando la hipótesis de ausencia de correlación resulta razonable. Los sesgos deben calibrarse o incorporarse al modelo; no se eliminan por aumentar la covarianza asignada.

A.3. Predicción y linealización

El filtro predice el estado mediante:

$$\hat{\mathbf{x}}_k^- = f(\hat{\mathbf{x}}_{k-1}^+, \mathbf{u}_k). \quad (105)$$

Los superíndices $-$ y $+$ indican los valores antes y después de incorporar una corrección. El EKF aproxima localmente las funciones no lineales mediante sus matrices de derivadas, denominadas jacobianos:

$$F_k = \left. \frac{\partial f}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_{k-1}^+, \mathbf{u}_k}, \quad (106)$$

$$H_k = \left. \frac{\partial h}{\partial \mathbf{x}} \right|_{\hat{\mathbf{x}}_k^-}. \quad (107)$$

Estas matrices describen cómo pequeños cambios del estado afectan a su evolución y a las medidas. La covarianza predicha se calcula como:

$$P_k^- = F_k P_{k-1}^+ F_k^T + Q_k. \quad (108)$$

En esta expresión, Q_k es la contribución del ruido durante el intervalo de predicción, expresada en las coordenadas del estado.

A.4. Corrección con una medida

La innovación es la diferencia entre la medida recibida y la que esperaba el filtro:

$$\mathbf{r}_k = \mathbf{z}_k - h(\hat{\mathbf{x}}_k^-). \quad (109)$$

Su covarianza combina la incertidumbre de la predicción y la de la medida:

$$S_k = H_k P_k^- H_k^T + R_k. \quad (110)$$

La ganancia de Kalman es:

$$K_k = P_k^- H_k^T S_k^{-1}. \quad (111)$$

Con ella se actualiza el estado:

$$\hat{\mathbf{x}}_k^+ = \hat{\mathbf{x}}_k^- + K_k \mathbf{r}_k. \quad (112)$$

La covarianza corregida puede calcularse mediante la forma de Joseph:

$$P_k^+ = (I - K_k H_k) P_k^- (I - K_k H_k)^T + K_k R_k K_k^T. \quad (113)$$

La matriz I es la identidad. Esta forma ayuda a conservar las propiedades de la covarianza frente a los errores de redondeo del cálculo numérico.

Las expresiones anteriores presentan el caso de un estado con actualización aditiva. Si la orientación se representa mediante un cuaternión unitario, su corrección deberá respetar esa representación. Las ecuaciones concretas dependerán de cómo se formule el estado y su error.

A.5. Comprobación de la innovación

Para valorar una discrepancia respecto a su incertidumbre se utiliza:

$$d_k^2 = \mathbf{r}_k^T S_k^{-1} \mathbf{r}_k. \quad (114)$$

Una medida será candidata a rechazo si:

$$d_k^2 > \gamma. \quad (115)$$

Bajo las hipótesis gaussianas y una aproximación local adecuada, el umbral puede elegirse como:

$$\gamma = \chi_{m, 1-\alpha}^2, \quad (116)$$

donde m es el número de componentes independientes de la medida y α es la probabilidad nominal de rechazar una medida compatible con el modelo.

Superar el umbral indica una discrepancia, pero no identifica automáticamente su causa. Puede deberse a una medida incorrecta, a un error del modelo o a una incertidumbre mal ajustada.

B. Transformación de los ejes del cuerpo al sistema NED

La función `fusion_matriz_rotacion` convierte la orientación almacenada como cuaternión en una matriz de rotación. Esta matriz permite expresar las medidas del acelerómetro en norte, este y abajo, aunque los ejes del sensor estén inclinados respecto a esas direcciones.

B.1. Convención y matriz de rotación

Utilizamos un cuaternión unitario de Hamilton, con componente escalar primero:

$$\mathbf{q} = [w \ x \ y \ z]^T. \quad (117)$$

Las letras x , y y z son componentes del cuaternión, no coordenadas de posición. La correspondencia con la notación del cuerpo del documento es $w = q_0$, $x = q_1$, $y = q_2$ y $z = q_3$.

El cuaternión representa la transformación desde los ejes del cuerpo al sistema NED. Con esta convención, la matriz correspondiente es:

$$\mathbf{C}_b^n = \begin{bmatrix} 1 - 2(y^2 + z^2) & 2(xy - wz) & 2(xz + wy) \\ 2(xy + wz) & 1 - 2(x^2 + z^2) & 2(yz - wx) \\ 2(xz - wy) & 2(yz + wx) & 1 - 2(x^2 + y^2) \end{bmatrix}. \quad (118)$$

Las filas corresponden a las componentes norte, este y abajo del resultado. Las columnas corresponden a las componentes X, Y y Z del vector de entrada, expresado en el cuerpo.

Por ejemplo, el elemento de la segunda fila y primera columna indica cuánto contribuye la componente X del cuerpo a la componente este. En C se accede a ese elemento mediante `C[1][0]`, porque los índices comienzan en cero.

B.2. Implementación en C

El Listado 58 muestra la función completa. Requiere un cuaternión válido y normalizado, además de memoria disponible para la matriz de salida. La función no modifica el estado del estimador ni adquiere medidas.

Listado 58: Conversión del cuaternión unitario a la matriz de rotación cuerpo a NED.

```
void fusion_matriz_rotacion(
    const float q[4],
    float C[3][3]
)
{
    /* Componentes del cuaternion: escalar, x, y, z. */
    const float w = q[0];
    const float x = q[1];
    const float y = q[2];
    const float z = q[3];

    /* Fila norte. */
    C[0][0] = 1.0f - 2.0f*(y*y + z*z);
    C[0][1] = 2.0f*(x*y - w*z);
    C[0][2] = 2.0f*(x*z + w*y);

    /* Fila este. */
    C[1][0] = 2.0f*(x*y + w*z);
    C[1][1] = 1.0f - 2.0f*(x*x + z*z);
    C[1][2] = 2.0f*(y*z - w*x);

    /* Fila abajo. */
    C[2][0] = 2.0f*(x*z - w*y);
    C[2][1] = 2.0f*(y*z + w*x);
    C[2][2] = 1.0f - 2.0f*(x*x + y*y);
}
```

B.3. Integración en la predicción

La función se utilizará después de calcular y normalizar la orientación representativa del intervalo. Para integrar la aceleración emplearemos la orientación del punto medio, conforme al criterio descrito en el cuerpo del documento.

La transformación de la fuerza específica corregida será:

$$\mathbf{f}_c^n = \mathbf{C}_b^n \mathbf{f}_c^b. \quad (119)$$

Por ejemplo, su componente norte se obtiene como:

$$f_{c,N} = C_{11}f_{c,x} + C_{12}f_{c,y} + C_{13}f_{c,z}. \quad (120)$$

Aquí los índices matemáticos comienzan en uno; los índices de C comienzan en cero.

Esta operación cambia los ejes en los que expresamos el vector. No incorpora todavía la gravedad ni convierte por sí sola la fuerza específica en aceleración del movimiento.

Como comprobación sencilla, el cuaternión $[1, 0, 0, 0]^T$ produce la matriz identidad. En ese caso, los ejes del cuerpo están alineados con NED y las componentes del vector no cambian. Para una orientación distinta, la matriz distribuye las componentes entre las direcciones correspondientes.